

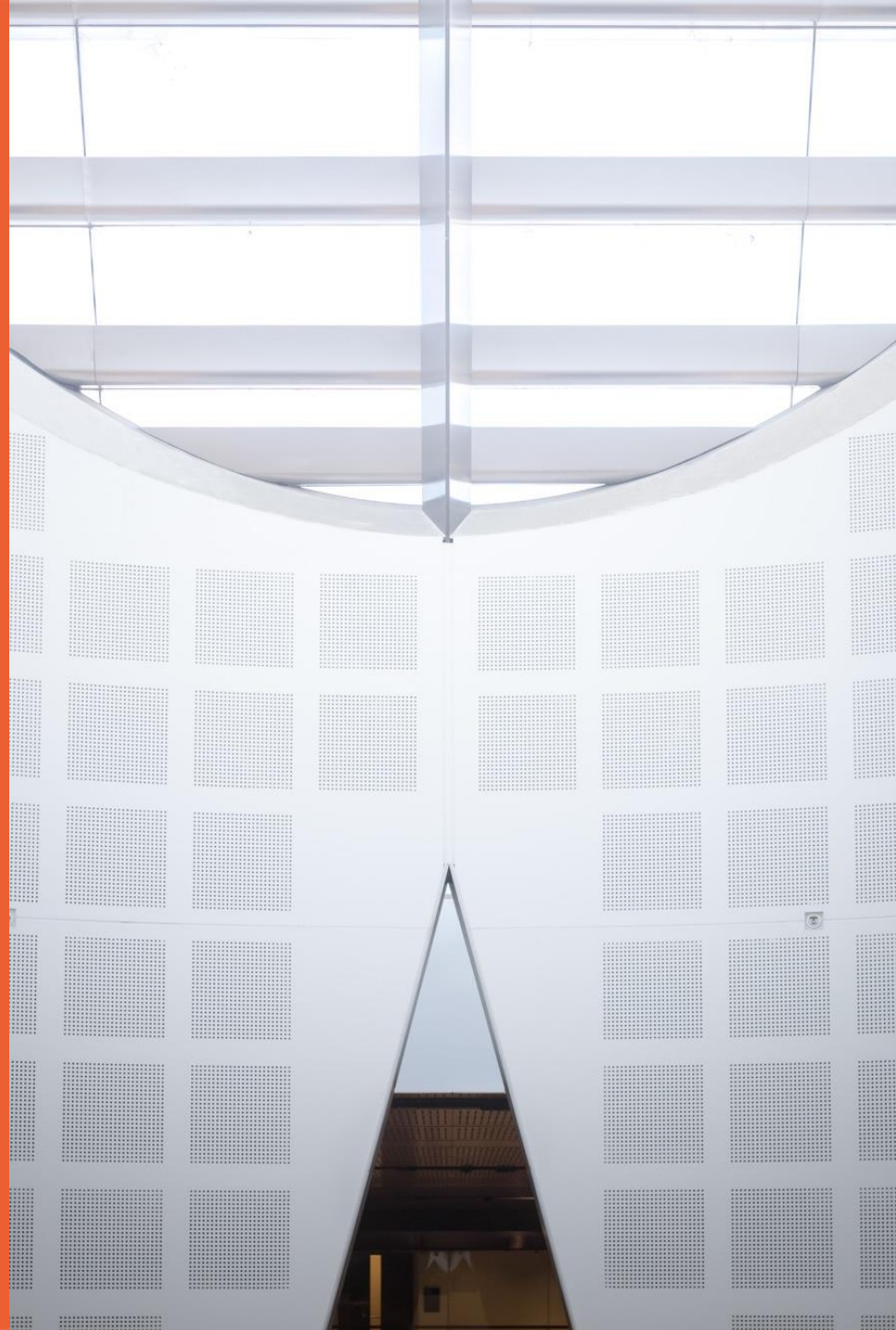
Convolutional Neural Networks and Network Architectures

Qianyang Chen

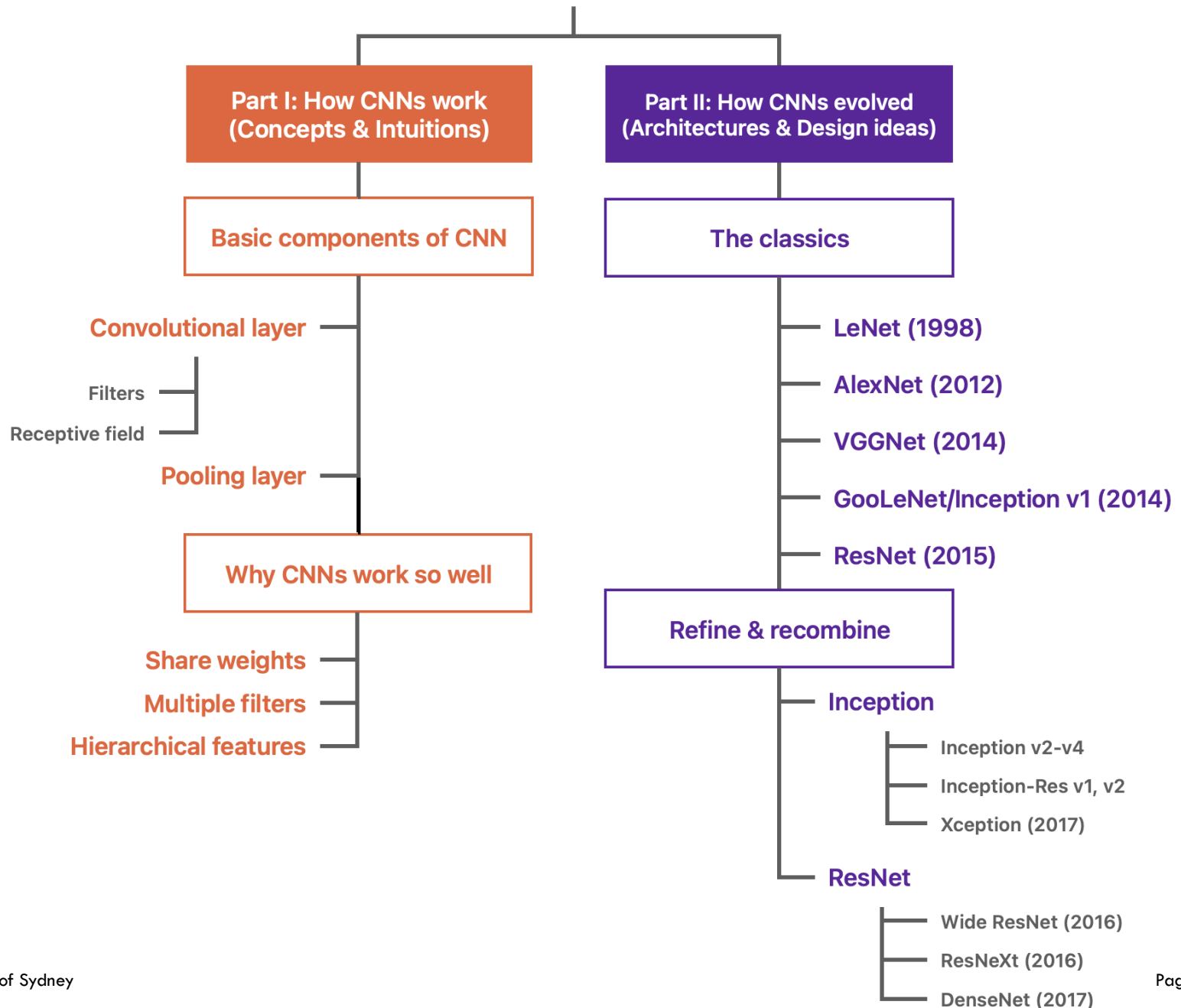
School of Computer Science



THE UNIVERSITY OF
SYDNEY

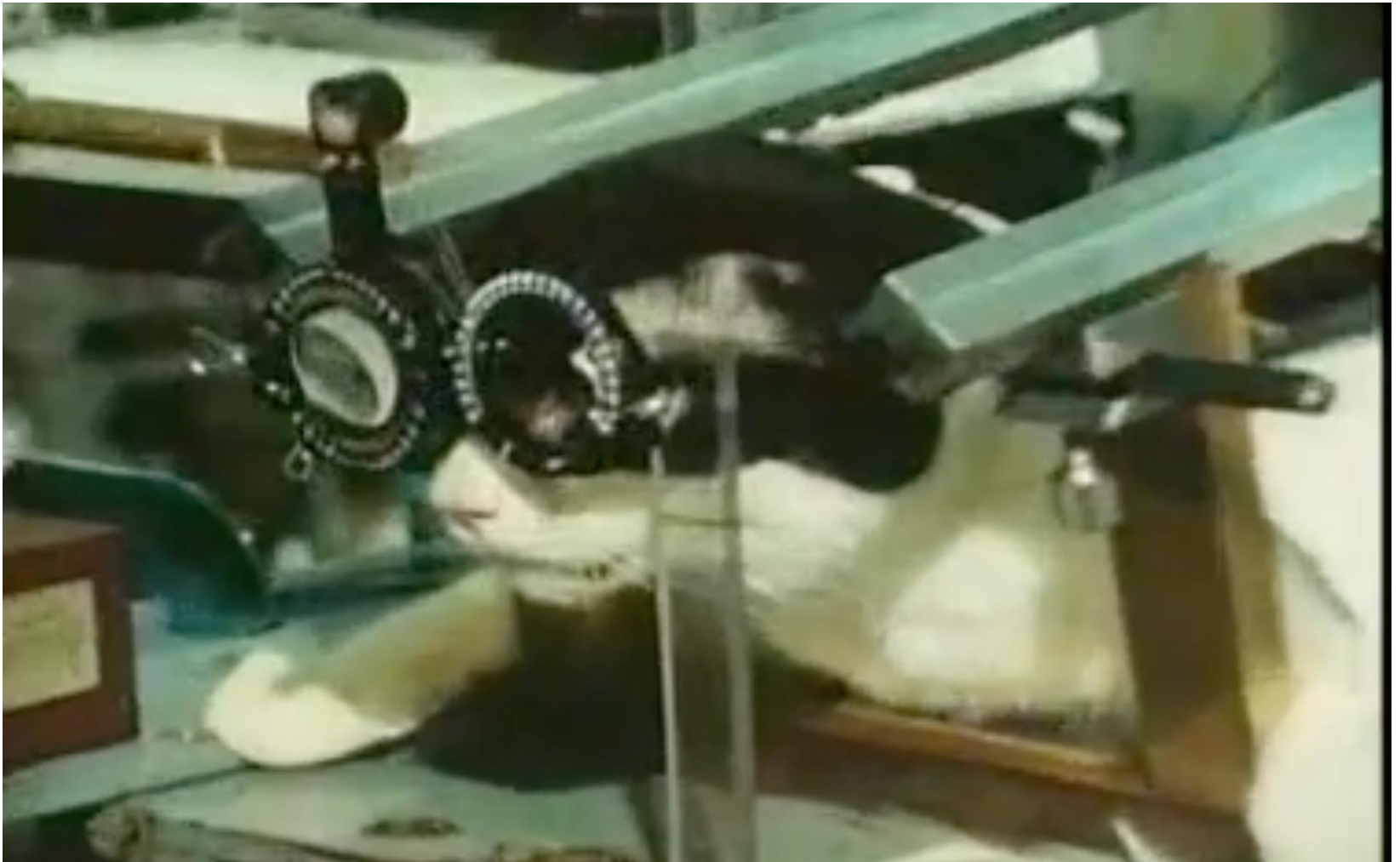


Convolutional Neural Network



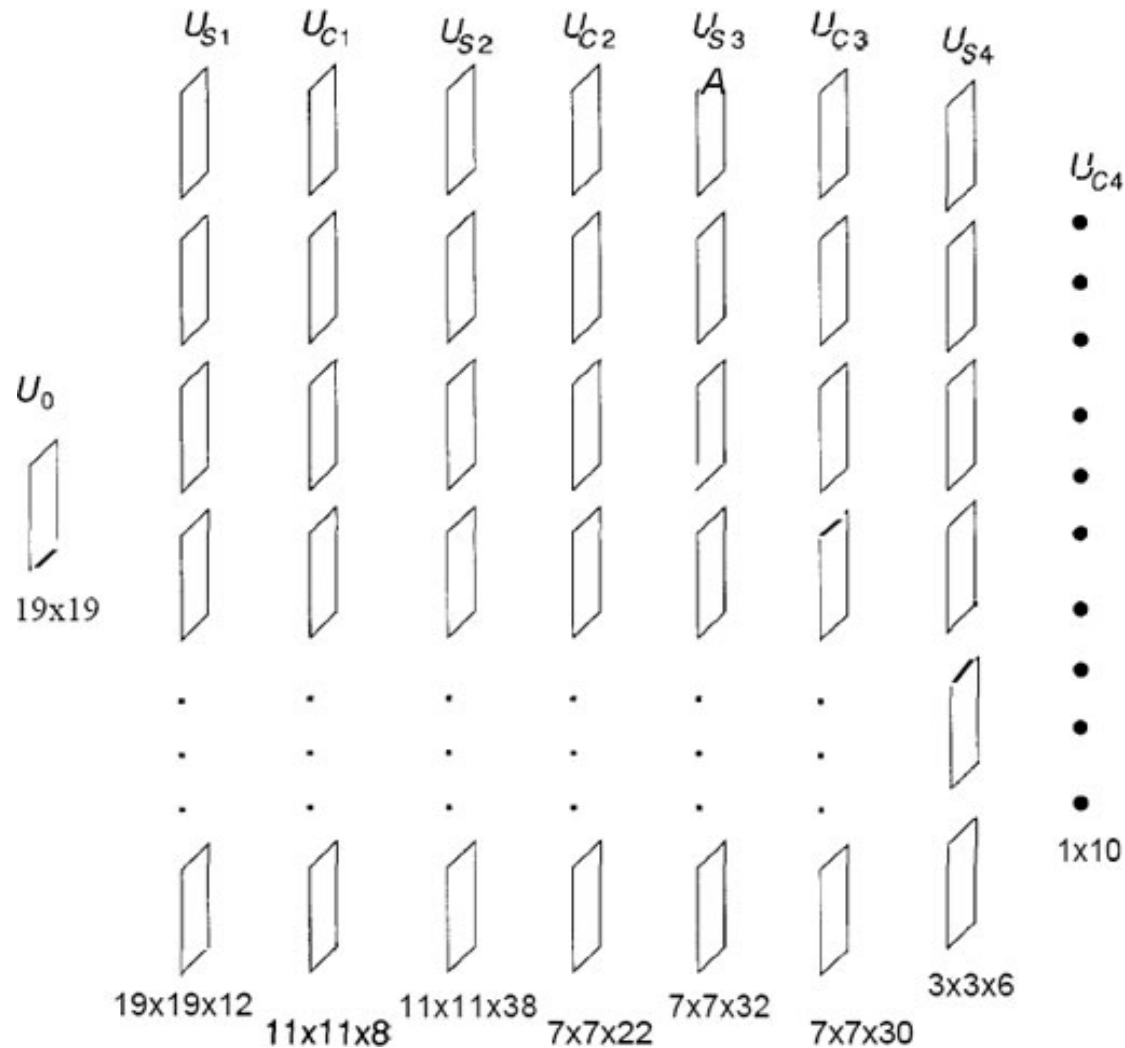
History of CNNs

Neocognitron (Kunihiko Fukushima, 1980)



History of CNNs

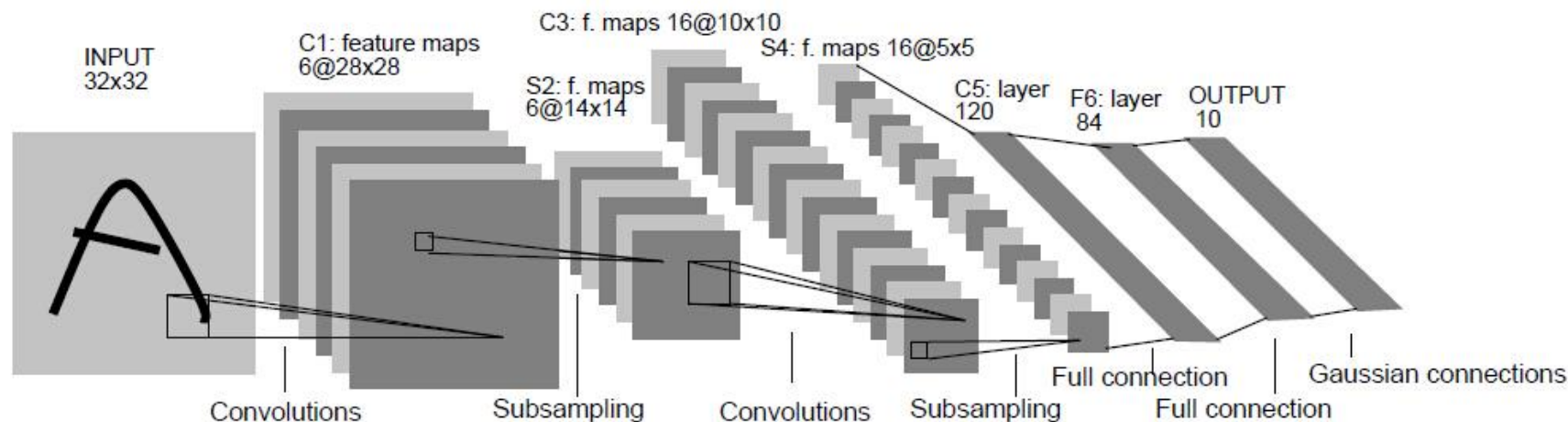
Neocognitron (Kunihiko Fukushima, 1980)



History of CNNs

LeNet-5 (LeCun et al, 1998)

- Built the modern framework of CNNs: **Convolutional Layer**, **Pooling Layer**, and **Fully-Connected Layer**
- Trained with the **backpropagation algorithm**
- Classify handwritten digits. However, it can not perform well on more complex problems, e.g., large-scale image and video classification



History of CNNs

Linear classifier:	8% ~ 12% error
K-nearest-neighbor:	1.x% ~ 5% error
Support Vector Machine:	0.6% ~ 1.4% error
(Conventional) Neural Nets:	1% ~ 5% error



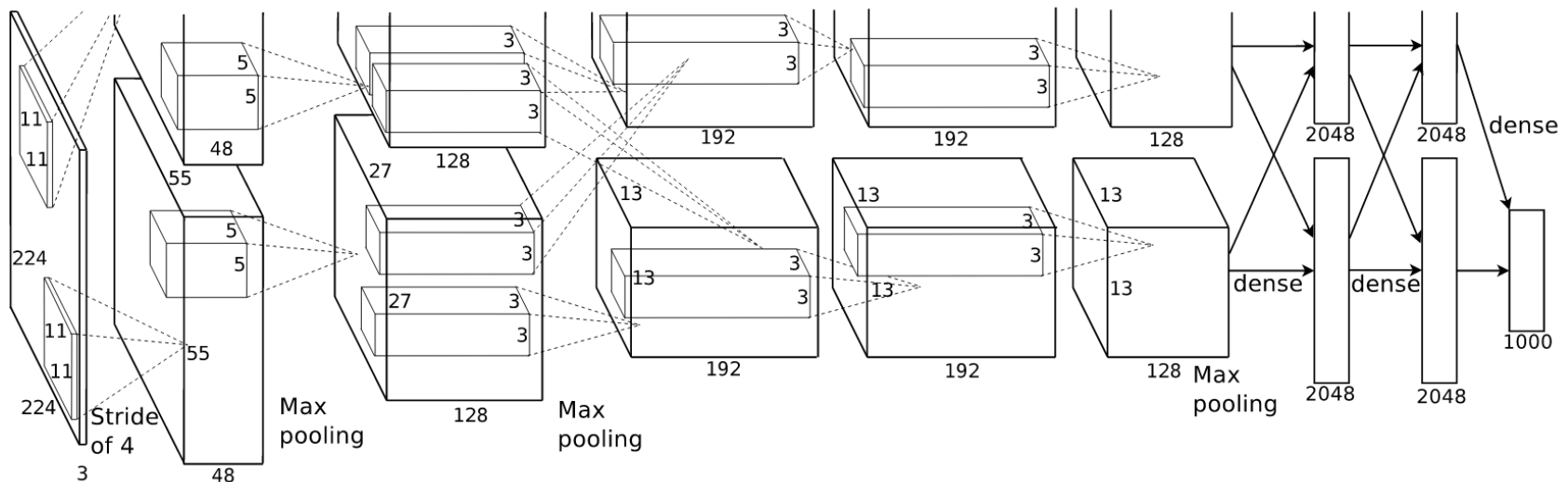
“The MNIST Database”

History of CNNs

AlexNet (Krizhevsky et al, 2012)

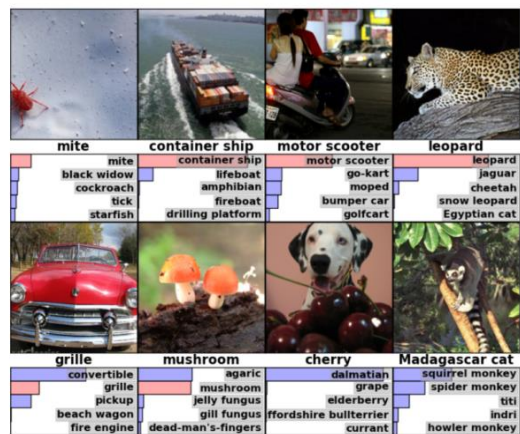
- Significant improvements on the image classification task, ImageNet 2012
- The network achieved a top-5 error of 15.3%, more than 10.8 percentage points ahead of the runner up.
- Basic framework of CNNs with a deeper structure
- Benefit from ImageNet dataset, GPUs, ReLU, Dropout ...

5 convolutional layers and 3 fully connected layers

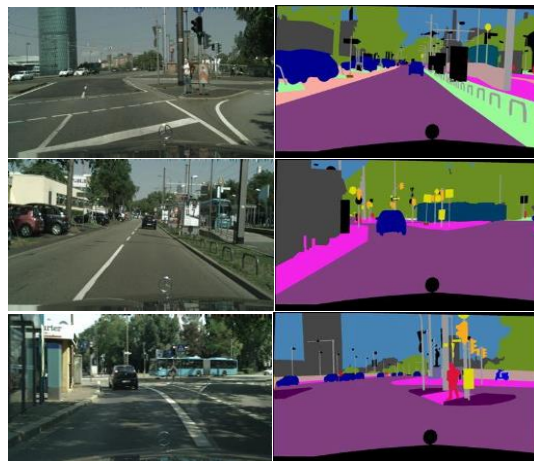


Today, CNNs are everywhere

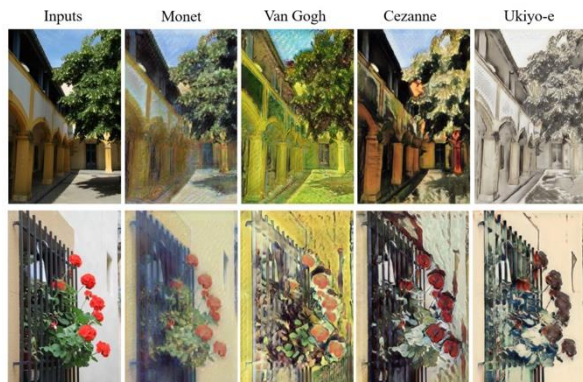
- Image classification, Image segmentation, Pose estimation, Style transfer, Image detection, Image caption ...



(Krizhevsky et al, 2012)



(Shaoli et al, 2017)



(Xinyuan et al, 2018)



Large black and white dog jumping hurdle

(Jianfeng et al, 2017)

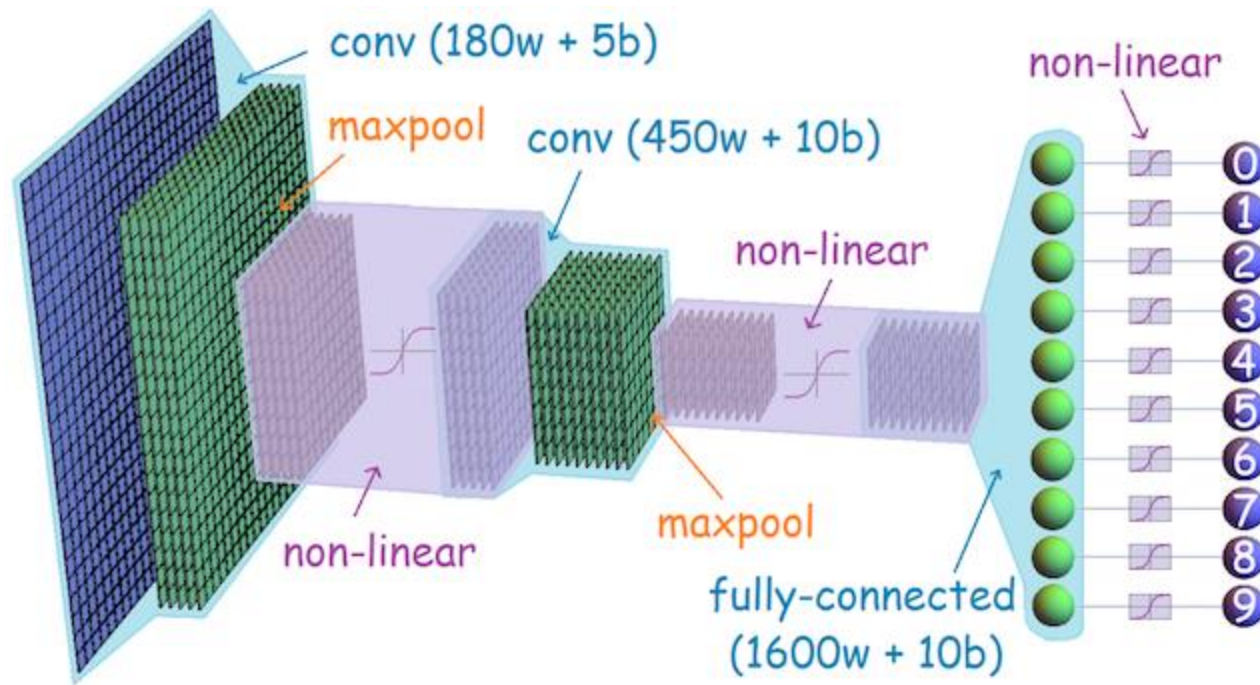


A man in a red shirt is playing a saxophone outside a business

Basic CNNs Components

A general CNN

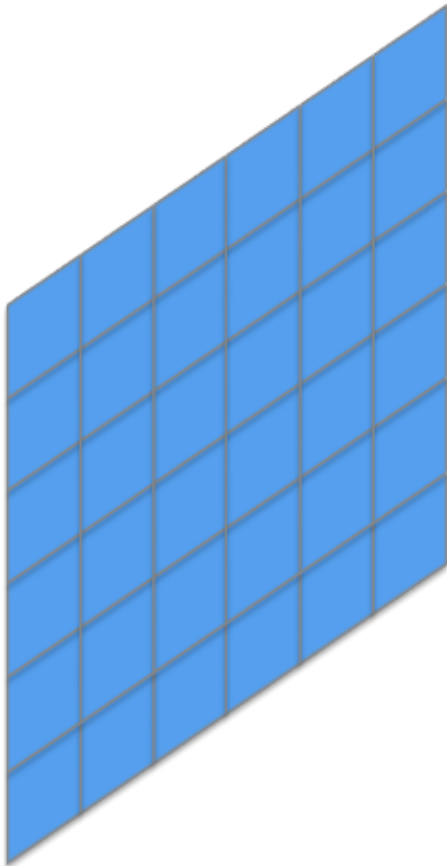
- Convolutional Layer
- Pooling
- Fully-connected Layer



(<https://leonardoaraujosantos.gitbooks.io>)

Convolutional Layer

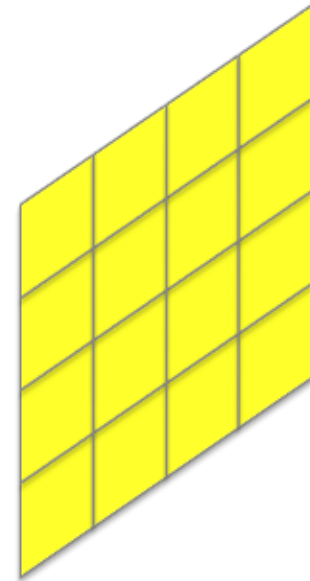
- Give a simple example: take a grayscale image as input



Grayscale Image: X

$w_{0,1}$	$w_{0,2}$	$w_{0,3}$
$w_{1,1}$	$w_{1,2}$	$w_{1,3}$
$w_{2,1}$	$w_{2,2}$	$w_{2,3}$

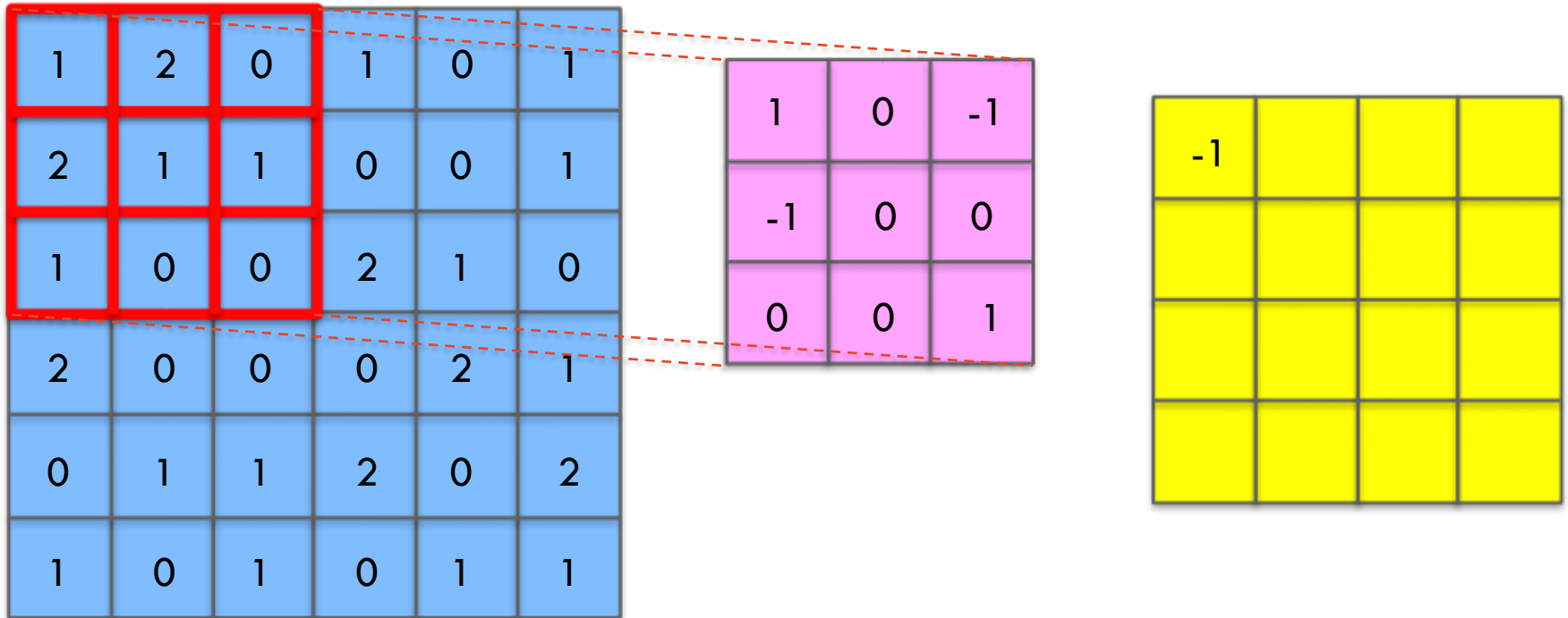
Filter: w



Feature

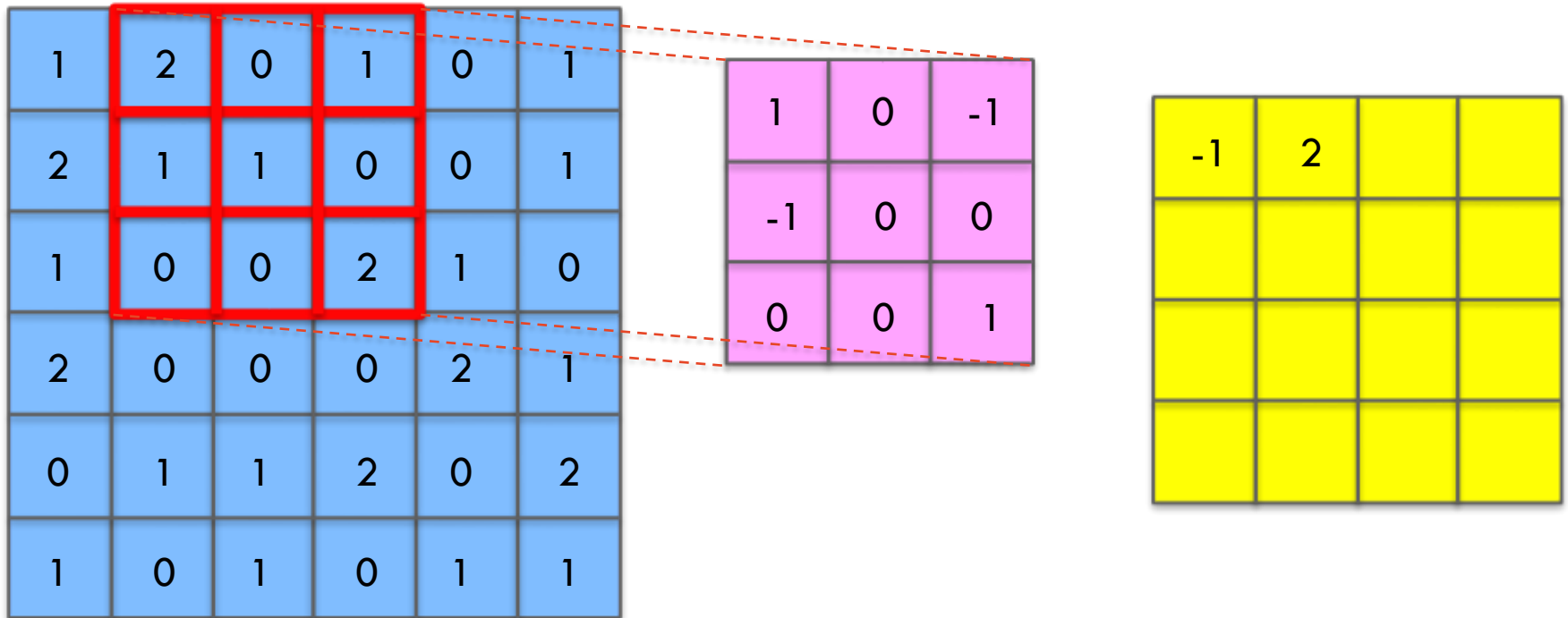
Convolutional Layer

– Convolution



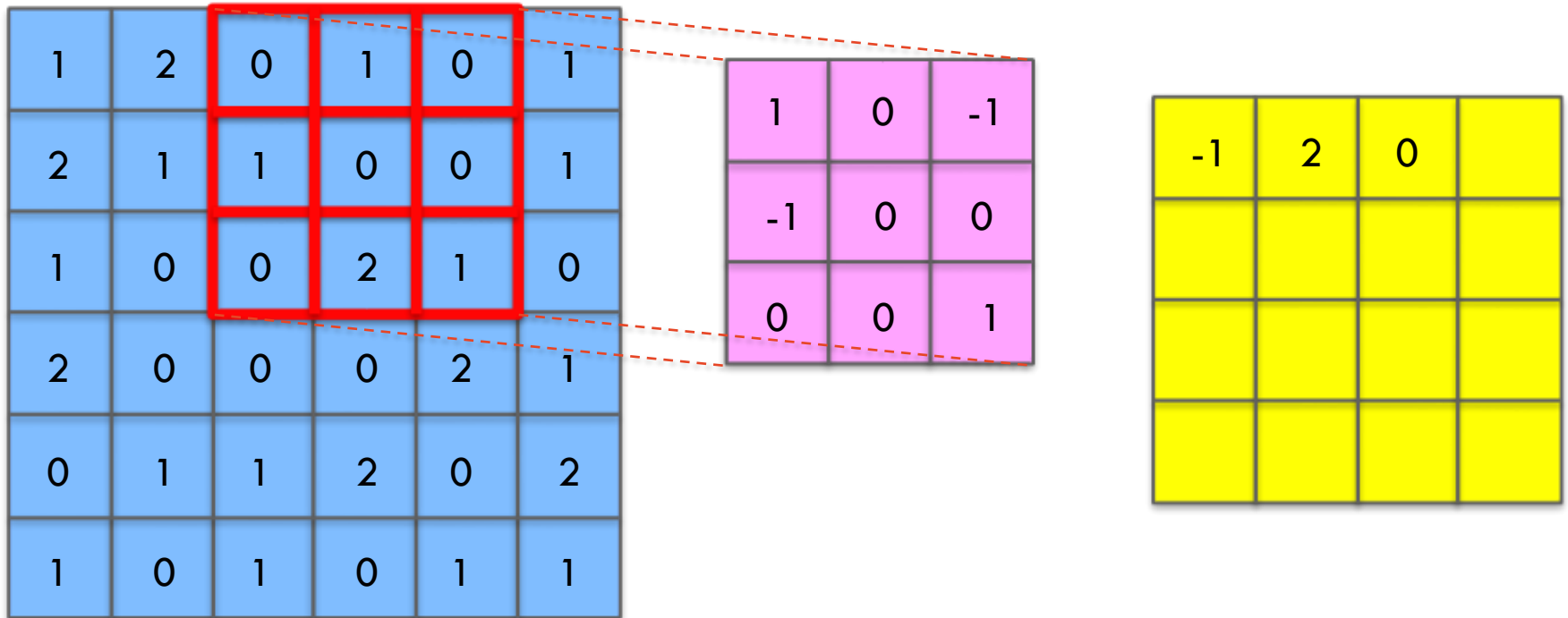
Convolutional Layer

– Convolution



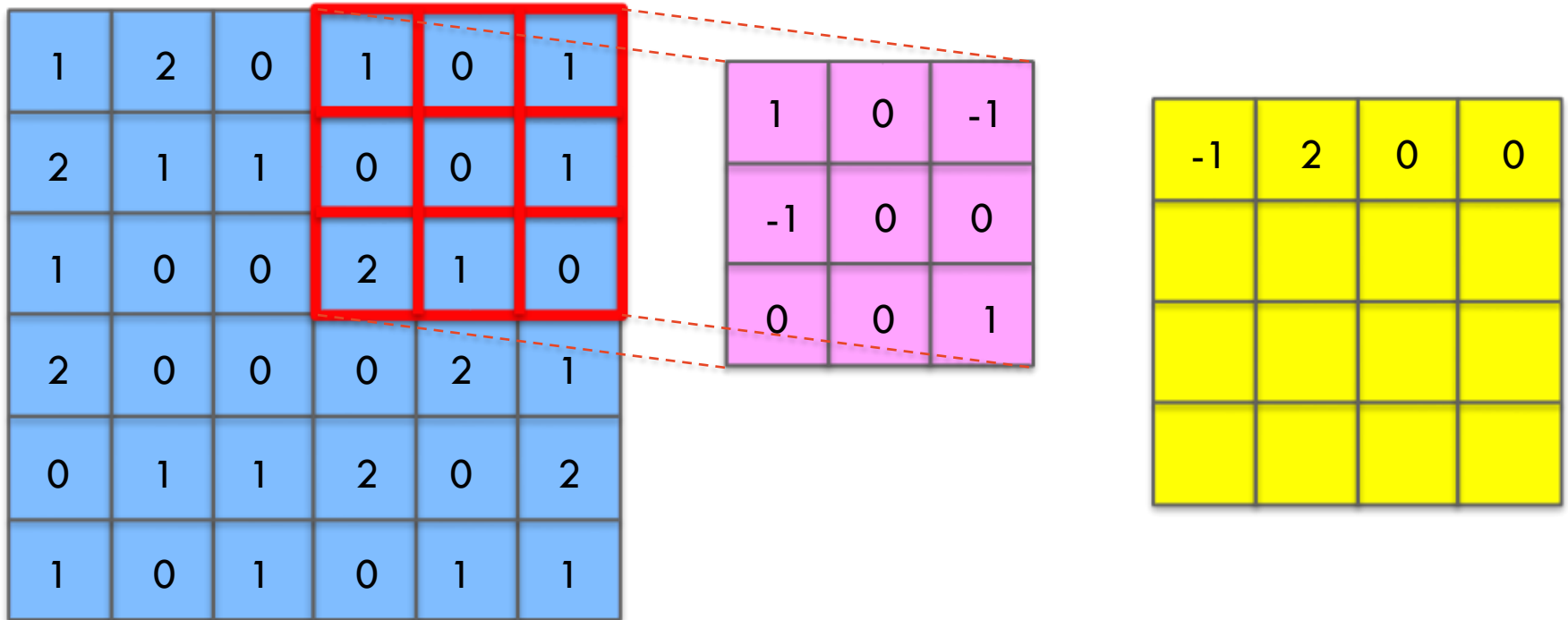
Convolutional Layer

– Convolution



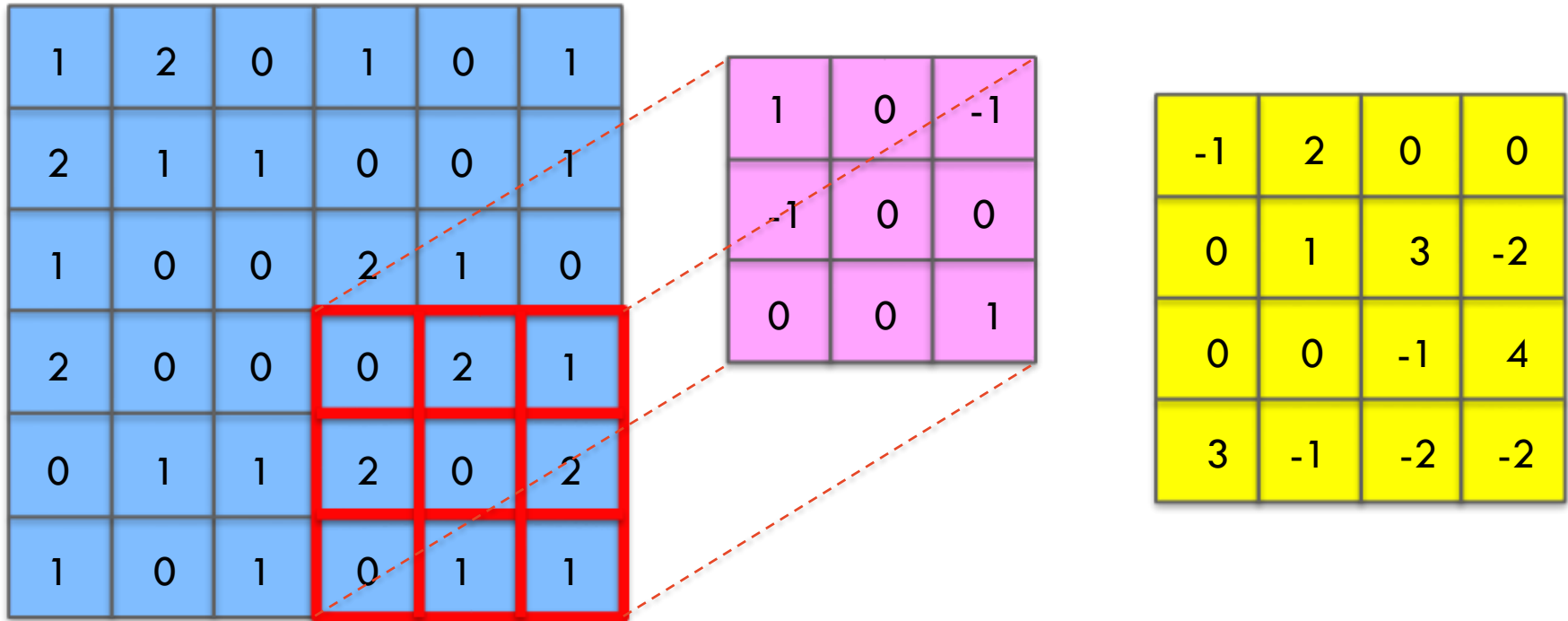
Convolutional Layer

– Convolution



Convolutional Layer

– Convolution



Convolutional Layer

– Stride

1	2	0	1	0	1
2	1	1	0	0	1
1	0	0	2	1	0
2	0	0	0	2	1
0	1	1	2	0	2
1	0	1	0	1	1

1	0	-1
-1	0	0
0	0	1

Stride = 1

-1	2		

The *stride* size is defined by how much you want to shift your filter at each step.

Convolutional Layer

– Stride

1	2	0	1	0	1
2	1	1	0	0	1
1	0	0	2	1	0
2	0	0	0	2	1
0	1	1	2	0	2
1	0	1	0	1	1

1	0	-1
-1	0	0
0	0	1

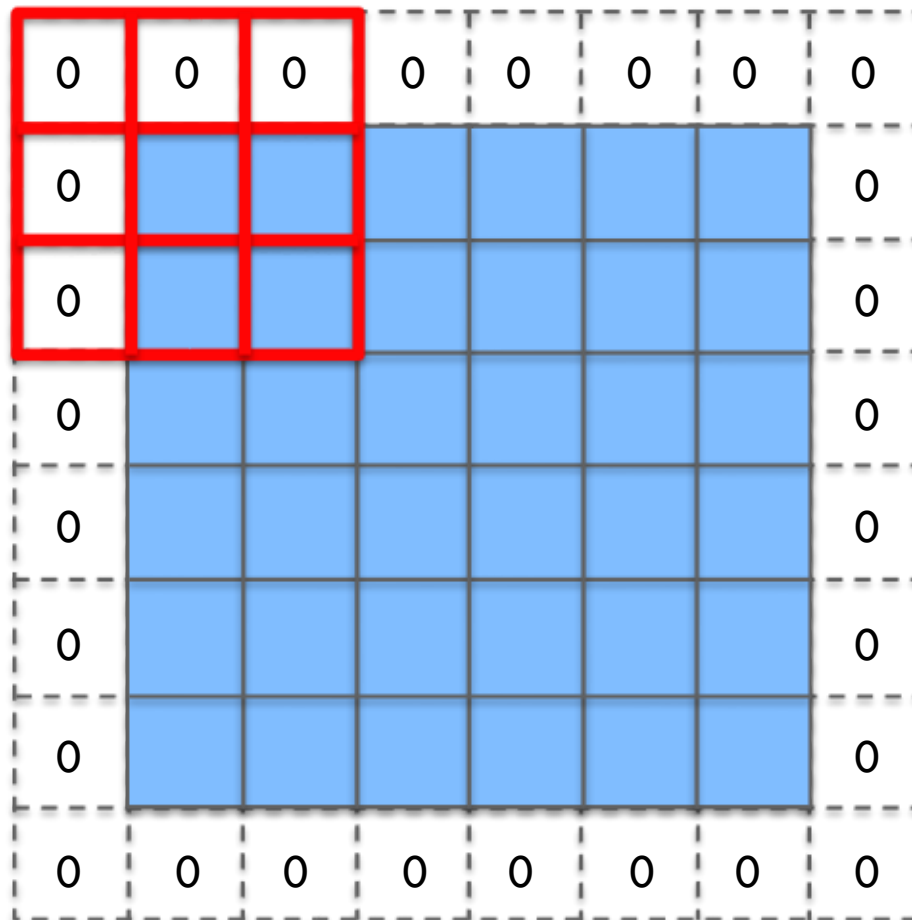
-1	0

Stride = 3

Convolutional Layer

- Zero padding (pad = 1)

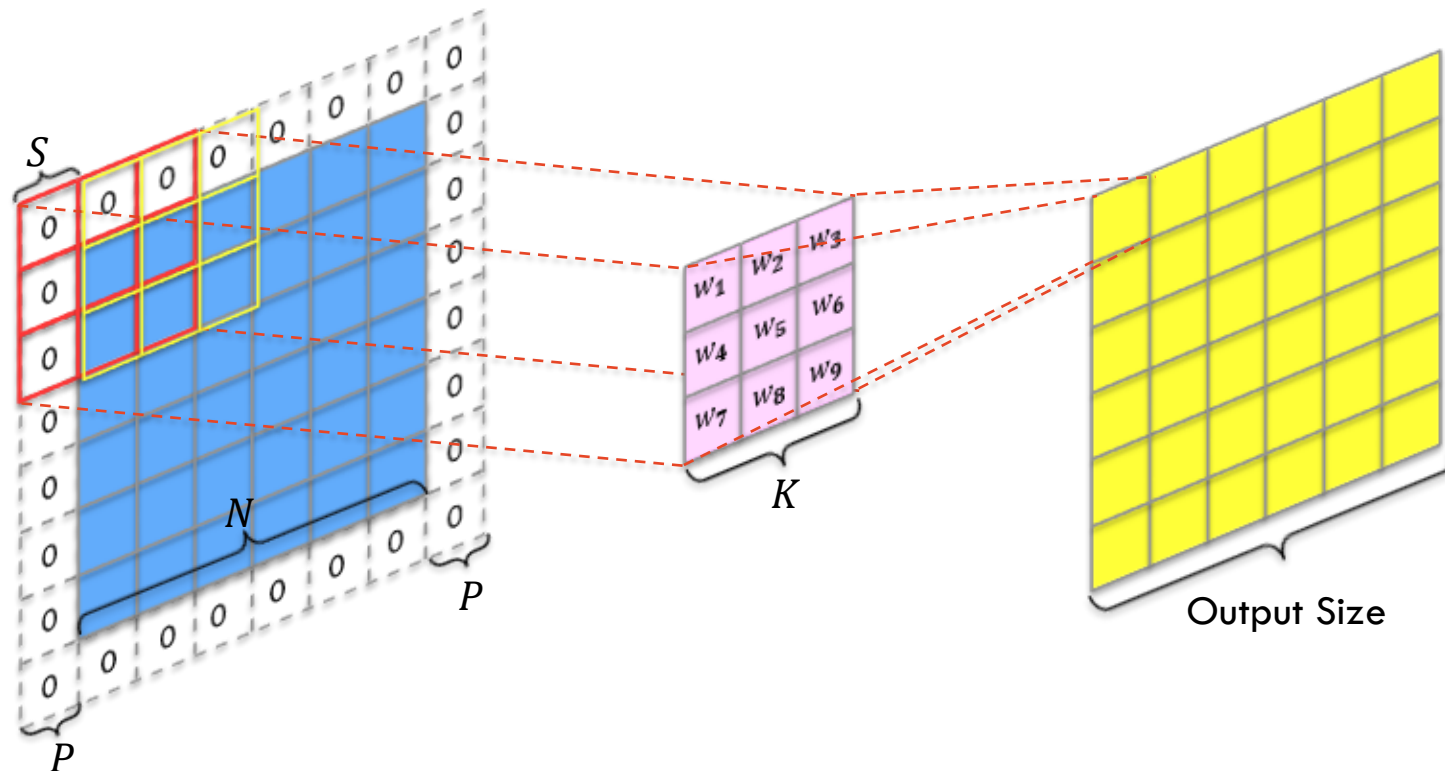
By doing this you can apply the filter to every element of your input matrix.



Convolutional Layer

– Output Size

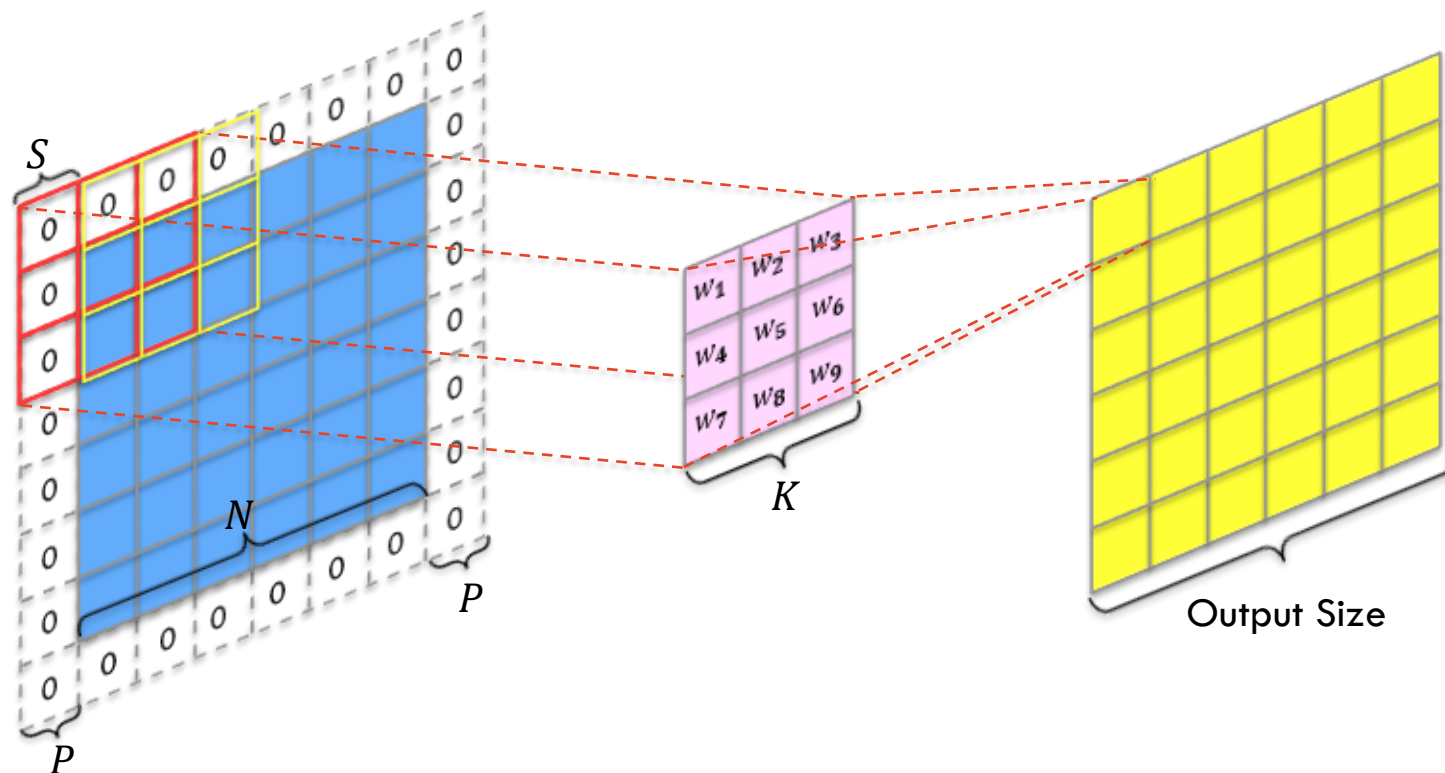
Output Size = ?



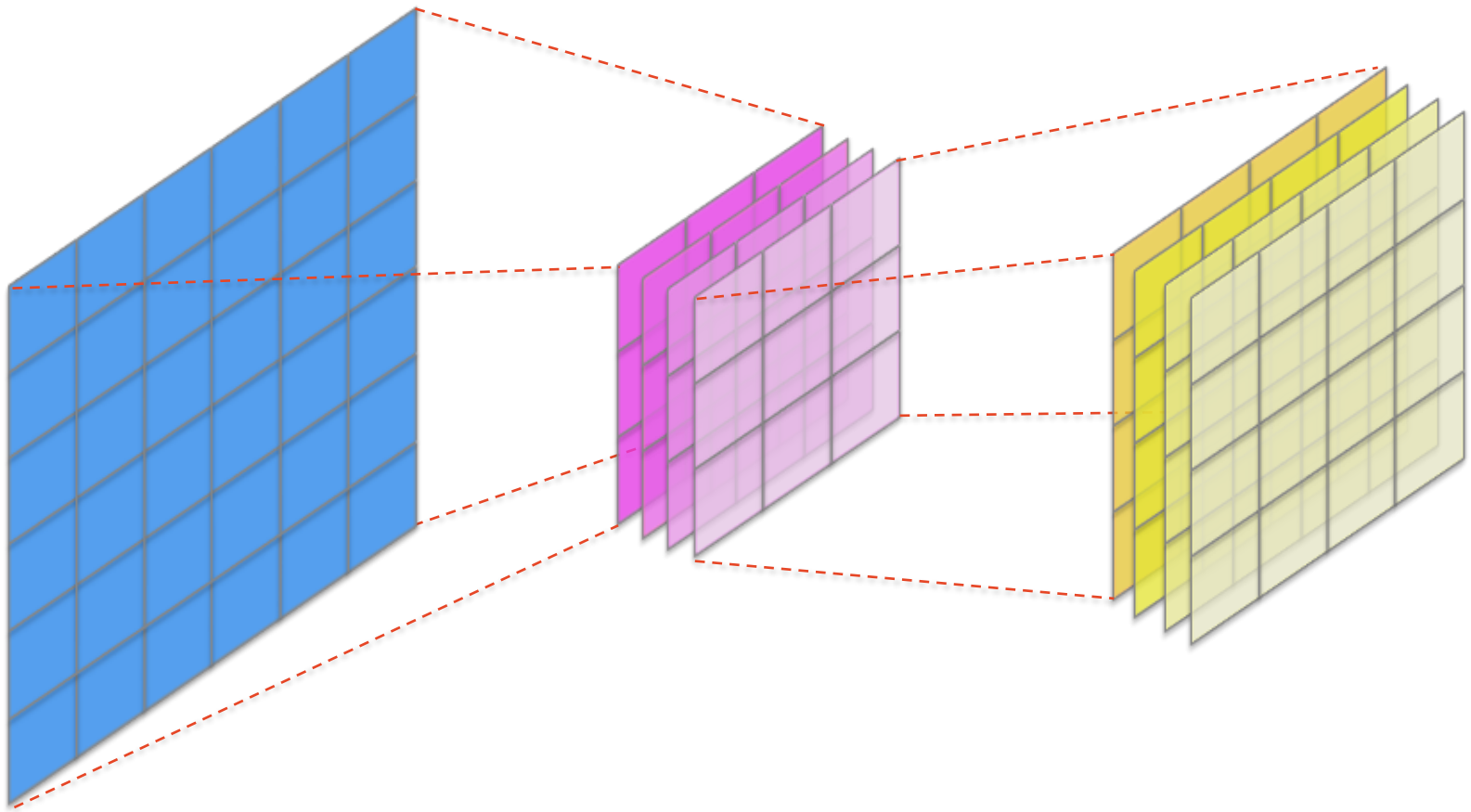
Convolutional Layer

– Output Size

$$\text{Output Size} = \frac{N+2P-K}{S} + 1$$



Learn multiple filters



Learn multiple filters

1	2	0	1	0	1
2	1	1	0	0	1
1	0	0	2	1	0
2	0	0	0	2	1
0	1	1	2	0	2
1	0	1	0	1	1

1	0	-1
-1	0	0
0	0	1

Filter 1

0	2	1
0	1	-1
-1	1	0

Filter 2

⋮

-1	2	0	0
0	1	3	-2
0	0	-1	4
3	-1	-2	-2

⋮

Learn multiple filters

1	2	0	1	0	1
2	1	1	0	0	1
1	0	0	2	1	0
2	0	0	0	2	1
0	1	1	2	0	2
1	0	1	0	1	1

1	0	-1
-1	0	0
0	0	1

Filter 1

0	2	1
0	1	-1
-1	1	0

Filter 2

⋮

-1	2	0	0
0	1	3	-2
0	0	-1	4
3	-1	-2	-2

3			

⋮

Learn multiple filters

1	2	0	1	0	1
2	1	1	0	0	1
1	0	0	2	1	0
2	0	0	0	2	1
0	1	1	2	0	2
1	0	1	0	1	1

1	0	-1
-1	0	0
0	0	1

Filter 1

0	2	1
0	1	-1
-1	1	0

Filter 2

⋮

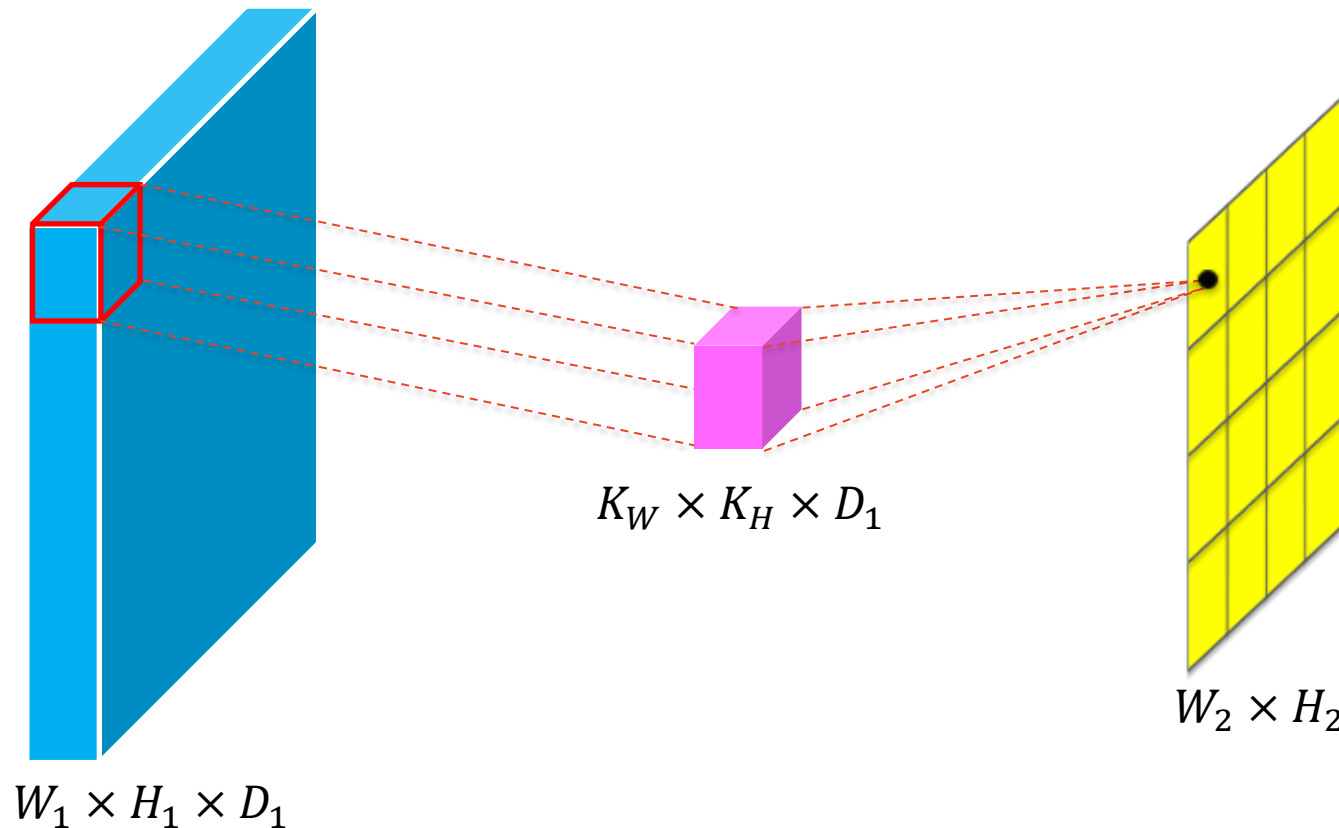
-1	2	0	0
0	1	3	-2
0	0	-1	4
3	-1	-2	-2

3	2	4	-1
1	0	1	4
1	2	4	1
-1	0	3	4

⋮

Convolutional Layer

- Above, we have only considered a 2-D image as input
- When the input has depth (e.g. RGB images), the convolution ops should be...



Convolutional Layer

Input Volume (+pad 1) (7x7x3)

$x[:, :, 0]$

0	0	0	0	0	0	0
0	0	1	1	2	2	0
0	0	1	1	0	0	0

0	1	1	0	1	0	0
0	1	0	1	1	1	0
0	0	2	0	1	0	0
0	0	0	0	0	0	0

$x[:, :, 1]$

0	0	0	0	0	0	0
0	1	1	1	2	0	0
0	0	2	1	1	2	0

0	1	2	0	0	2	0
0	0	2	1	2	1	0
0	2	0	1	2	0	0
0	0	0	0	0	0	0

$x[:, :, 2]$

0	0	0	0	0	0	0
0	2	0	2	0	2	0
0	0	0	1	2	1	0

0	1	0	2	2	1	0
0	2	0	2	0	0	0
0	0	0	1	1	2	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

$w0[:, :, 0]$

1	1	-1
-1	0	1
-1	-1	0

$w0[:, :, 1]$

-1	0	-1
0	0	-1
1	-1	0

$w0[:, :, 2]$

0	1	0
1	0	1
0	-1	1

Bias b0 (1x1x1)

$b0[:, :, 0]$

1

Filter W1 (3x3x3)

$w1[:, :, 0]$

-1	-1	0
-1	1	0
-1	1	0

$w1[:, :, 1]$

1	-1	0
-1	0	-1
-1	0	0

$w1[:, :, 2]$

-1	0	1
1	0	1
0	-1	0

Bias b1 (1x1x1)

$b1[:, :, 0]$

0

Output Volume (3x3x2)

?

Two filters

Stride=2

Zero-padding=1

Convolutional Layer

Input Volume (+pad 1) (7x7x3)

$x[:, :, 0]$

0	0	0	0	0	0	0
0	0	1	1	2	2	0
0	0	1	1	0	0	0

0	1	1	0	1	0	0
0	1	0	1	1	1	0
0	0	2	0	1	0	0
0	0	0	0	0	0	0

$x[:, :, 1]$

0	0	0	0	0	0	0
0	1	1	1	2	0	0
0	0	2	1	1	2	0

0	1	2	0	0	2	0
0	0	2	1	2	1	0
0	2	0	1	2	0	0
0	0	0	0	0	0	0

$x[:, :, 2]$

0	0	0	0	0	0	0
0	2	0	2	0	2	0
0	0	0	1	2	1	0

0	1	0	2	2	1	0
0	2	0	2	0	0	0
0	0	0	1	1	2	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

$w0[:, :, 0]$

1	1	-1
-1	0	1
-1	-1	0

$w0[:, :, 1]$

-1	0	-1
0	0	-1
1	-1	0

$w0[:, :, 2]$

0	1	0
1	0	1
0	-1	1

Bias b0 (1x1x1)

$b0[:, :, 0]$

1

Filter W1 (3x3x3)

$w1[:, :, 0]$

-1	-1	0
-1	1	0
-1	1	0

$w1[:, :, 1]$

1	-1	0
-1	0	-1
-1	0	0

$w1[:, :, 2]$

-1	0	1
1	0	1
0	-1	0

Bias b1 (1x1x1)

$b1[:, :, 0]$

0

Output Volume (3x3x2)

$o[:, :, 0]$

1	0	-3
-6	1	1
4	-3	1

$o[:, :, 1]$

-1	-6	-4
-2	-3	-4
-1	-3	-3

Two filters
Stride=2
Zero-padding=1

Convolutional Layer

Input Volume (+pad 1) (7x7x3) Filter W0 (3x3x3)

$x[:, :, 0]$

0	0	0	0	0	0	0
0	0	1	1	2	2	0
0	0	1	1	0	0	0
0	1	1	0	1	0	0
0	1	0	1	1	1	0
0	0	2	0	1	0	0
0	0	0	0	0	0	0

$w0[:, :, 0]$

1	1	-1
-1	0	1
-1	-1	0

$w0[:, :, 1]$

-1	0	-1
0	0	-1
1	-1	0

$w0[:, :, 2]$

0	1	0
1	0	1
0	-1	1

Bias $b0$ (1x1x1)

$b0[:, :, 0]$

1

$x[:, :, 1]$

0	0	0	0	0	0	0
0	1	1	1	2	0	0
0	0	2	1	1	2	0
0	1	2	0	0	2	0
0	0	2	1	2	1	0
0	2	0	1	2	0	0
0	0	0	0	0	0	0

$x[:, :, 2]$

0	0	0	0	0	0	0
0	2	0	2	0	2	0
0	0	0	1	2	1	0
0	1	0	2	2	1	0
0	2	0	2	0	0	0
0	0	0	1	1	2	0
0	0	0	0	0	0	0

Filter W1 (3x3x3)

$w1[:, :, 0]$

-1	-1	0
-1	1	0
-1	1	0

$w1[:, :, 1]$

1	-1	0
-1	0	-1
-1	0	0

$w1[:, :, 2]$

-1	0	1
1	0	1
0	-1	0

Bias $b1$ (1x1x1)

$b1[:, :, 0]$

0

Output Volume (3x3x2)

$o[:, :, 0]$

1	0	-3
-6	1	1
4	-3	1

$o[:, :, 1]$

-1	-6	-4
-2	-3	-4
-1	-3	-3

Two filters
Stride=2
Zero-padding=1

Convolutional Layer

Input Volume (+pad 1) (7x7x3) Filter W0 (3x3x3)

$x[:, :, 0]$

0	0	0	0	0	0	0
0	0	1	1	2	2	0
0	0	1	1	0	0	0
0	1	1	0	1	0	0
0	1	0	1	1	1	0
0	0	2	0	1	0	0
0	0	0	0	0	0	0

$w0[:, :, 0]$

1	1	-1
-1	0	1
-1	-1	0

$w0[:, :, 1]$

-1	0	-1
0	0	-1
1	-1	0

$w0[:, :, 2]$

0	1	0
1	0	1
0	-1	1

Bias $b0$ (1x1x1)

$b0[:, :, 0]$

1

$x[:, :, 1]$

0	0	0	0	0	0	0
0	1	1	1	2	0	0
0	0	2	1	1	2	0
0	1	2	0	0	2	0
0	0	2	1	2	1	0
0	2	0	1	2	0	0
0	0	0	0	0	0	0

$x[:, :, 2]$

0	0	0	0	0	0	0
0	2	0	2	0	2	0
0	0	0	1	2	1	0
0	1	0	2	2	1	0
0	2	0	2	0	0	0
0	0	0	1	1	2	0
0	0	0	0	0	0	0

Filter W1 (3x3x3)

$w1[:, :, 0]$

-1	-1	0
-1	1	0
-1	1	0

$w1[:, :, 1]$

1	-1	0
-1	0	-1
-1	0	0

$w1[:, :, 2]$

-1	0	1
1	0	1
0	-1	0

Bias $b1$ (1x1x1)

$b1[:, :, 0]$

0

Output Volume (3x3x2)

$o[:, :, 0]$

1	0	-3
-6	1	1
4	-3	1

$o[:, :, 1]$

-1	-6	-4
-2	-3	-4
-1	-3	-3

Two filters
Stride=2
Zero-padding=1

Convolutional Layer

Input Volume (+pad 1) (7x7x3)

$x[:, :, 0]$

0	0	0	0	0	0	0
0	0	1	1	2	2	0
0	0	1	1	0	0	0
0	1	1	0	1	0	0
0	1	0	1	1	1	0
0	0	2	0	1	0	0
0	0	0	0	0	0	0

$x[:, :, 1]$

0	0	0	0	0	0	0
0	1	1	1	2	0	0
0	0	2	1	1	2	0
0	1	2	0	0	2	0
0	0	2	1	2	1	0
0	2	0	1	2	0	0
0	0	0	0	0	0	0

$x[:, :, 2]$

0	0	0	0	0	0	0
0	2	0	2	0	2	0
0	0	0	1	2	1	0
0	1	0	2	2	1	0
0	2	0	2	0	0	0
0	0	0	1	1	2	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

$w0[:, :, 0]$

1	1	-1
-1	0	1
-1	-1	0

$w0[:, :, 1]$

-1	0	-1
0	0	-1
1	-1	0

$w0[:, :, 2]$

0	1	0
1	0	1
0	-1	1

Bias b0 (1x1x1)

$b0[:, :, 0]$
1

Filter W1 (3x3x3)

$w1[:, :, 0]$

-1	-1	0
-1	1	0
-1	1	0

$w1[:, :, 1]$

1	-1	0
-1	0	-1
-1	0	0

$w1[:, :, 2]$

-1	0	1
1	0	1
0	-1	0

Bias b1 (1x1x1)

$b1[:, :, 0]$
0

Output Volume (3x3x2)

$o[:, :, 0]$

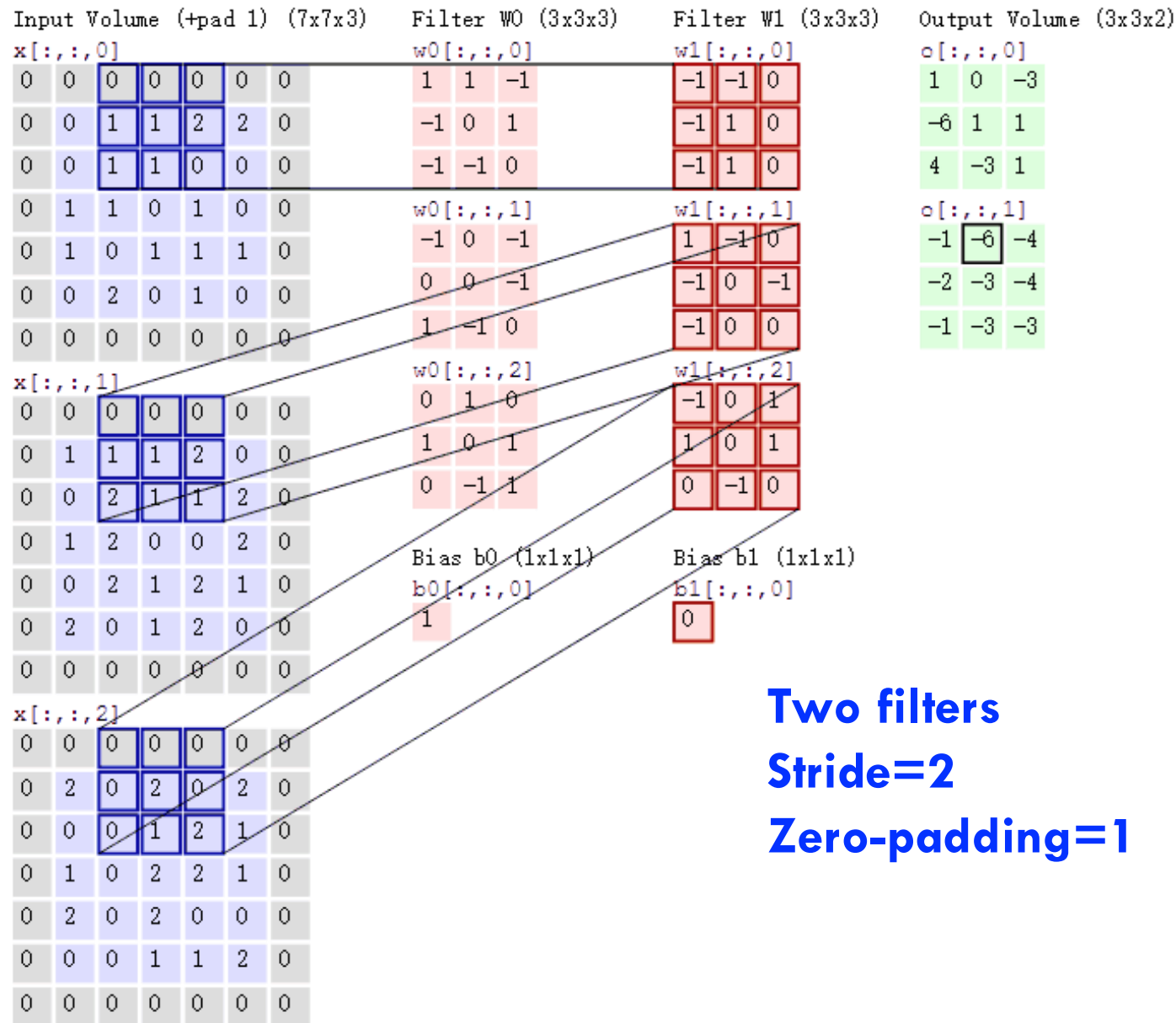
1	0	-3
-6	1	1
4	-3	1

$o[:, :, 1]$

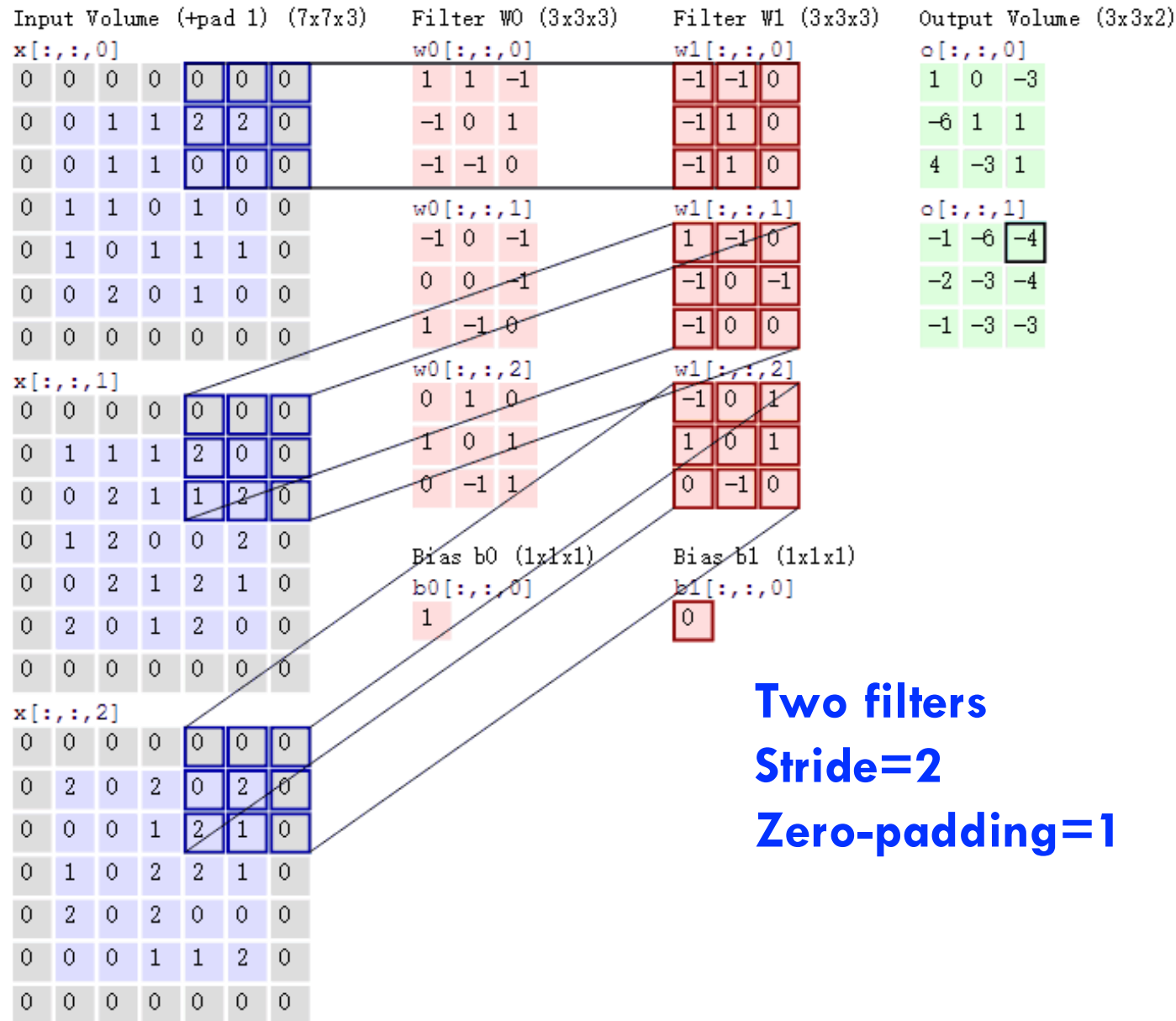
-1	-6	-4
-2	-3	-4
-1	-3	-3

Two filters
Stride=2
Zero-padding=1

Convolutional Layer



Convolutional Layer



Two filters
Stride=2
Zero-padding=1

Convolutional Layer

Input Volume (+pad 1) (7x7x3) Filter W0 (3x3x3)

$x[:, :, 0]$

0	0	0	0	0	0	0
0	0	1	1	2	2	0
0	0	1	1	0	0	0

$w0[:, :, 0]$

1	1	-1
-1	0	1
-1	-1	0

0	1	1	0	1	0	0
0	1	0	1	1	1	0
0	0	2	0	1	0	0
0	0	0	0	0	0	0

$w0[:, :, 1]$

-1	0	-1
0	0	-1
1	-1	0

$x[:, :, 1]$

0	0	0	0	0	0	0
0	1	1	1	2	0	0
0	0	2	1	1	2	0
0	1	2	0	0	2	0
0	0	2	1	2	1	0
0	2	0	1	2	0	0
0	0	0	0	0	0	0

$w0[:, :, 2]$

0	1	0
1	0	1
0	-1	1

Bias $b0$ (1x1x1)

$b0[:, :, 0]$

1

$x[:, :, 2]$

0	0	0	0	0	0	0
0	2	0	2	0	2	0
0	0	0	1	2	1	0
0	1	0	2	2	1	0
0	2	0	2	0	0	0
0	0	0	1	1	2	0
0	0	0	0	0	0	0

Filter W1 (3x3x3)

$w1[:, :, 0]$

-1	-1	0
-1	1	0
-1	1	0

$w1[:, :, 1]$

1	-1	0
-1	0	-1
-1	0	0

$w1[:, :, 2]$

-1	0	1
1	0	1
0	-1	0

Bias $b1$ (1x1x1)

$b1[:, :, 0]$

0

Output Volume (3x3x2)

$o[:, :, 0]$

1	0	-3
-6	1	1
4	-3	1

$o[:, :, 1]$

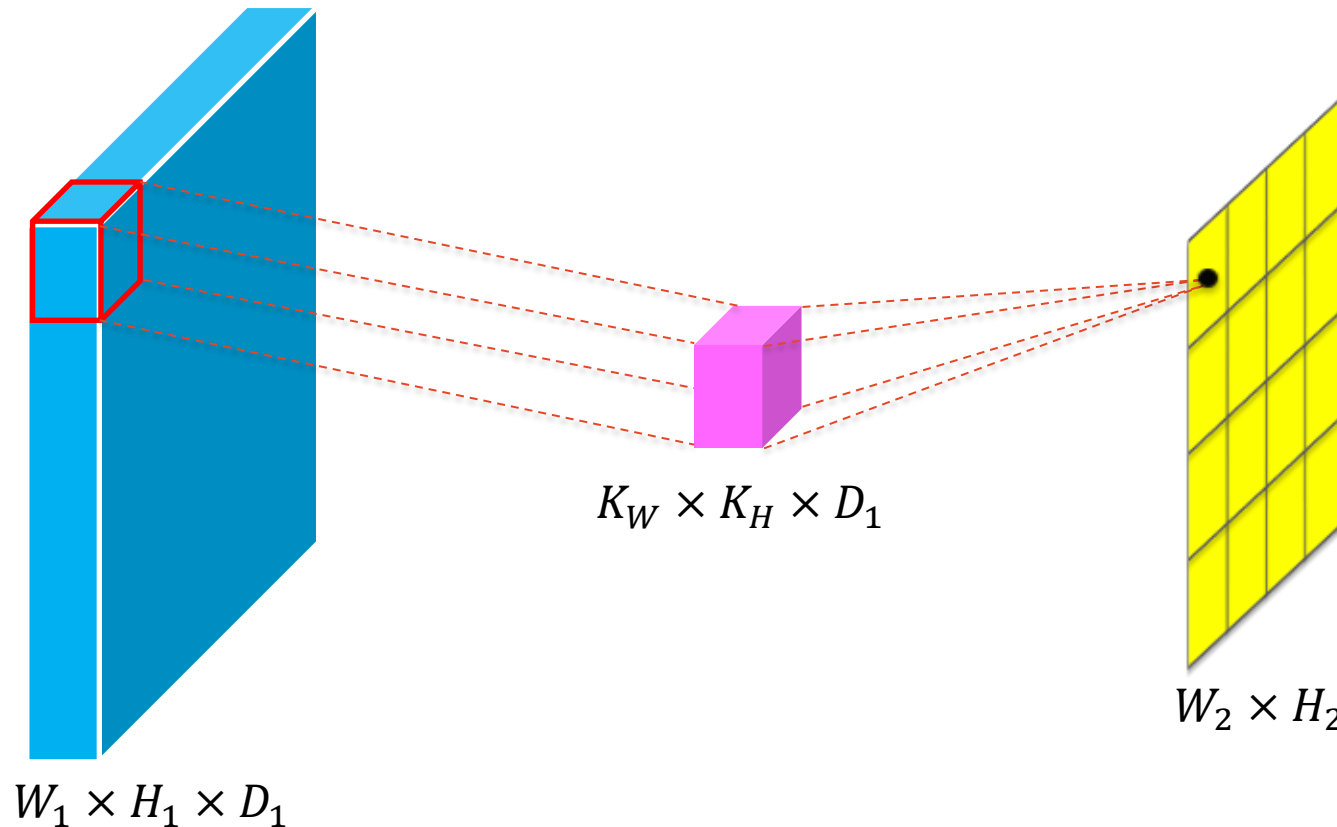
-1	-6	-4
-2	-3	-4
-1	-3	-3

Two filters
Stride=2
Zero-padding=1

Convolutional Layer

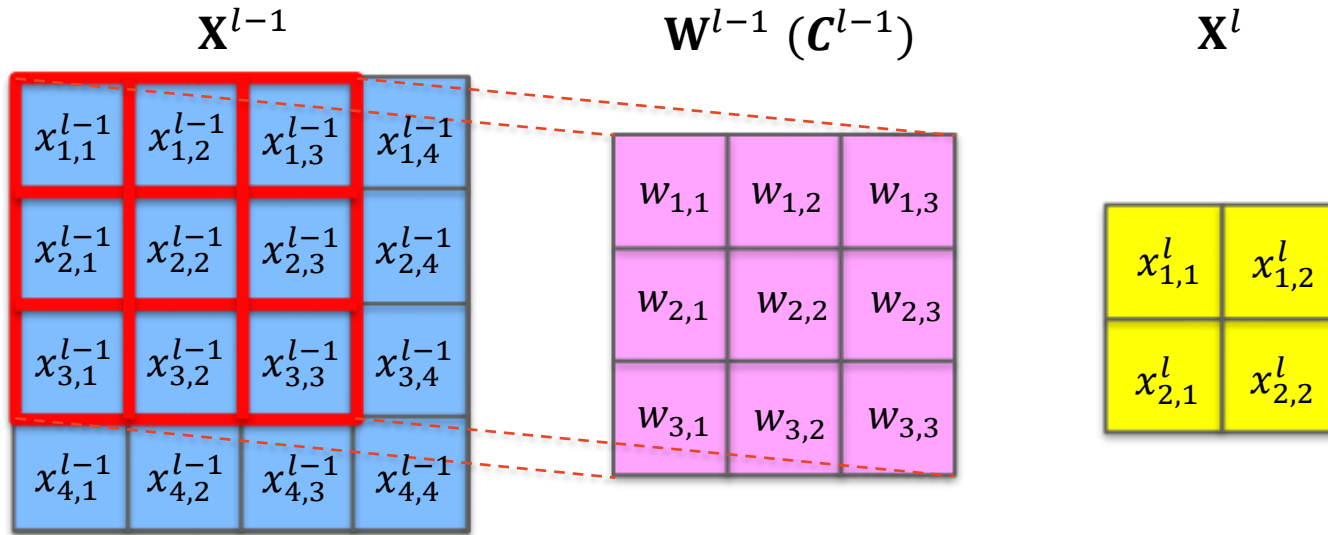
- Suppose stride is (S_W, S_H) and pad is (P_W, P_H)

$$W_2 = \frac{W_1 + 2P_W - K_W}{S_W} + 1 \quad \text{and} \quad H_2 = \frac{H_1 + 2P_H - K_H}{S_H} + 1$$



Convolution as a matrix operation

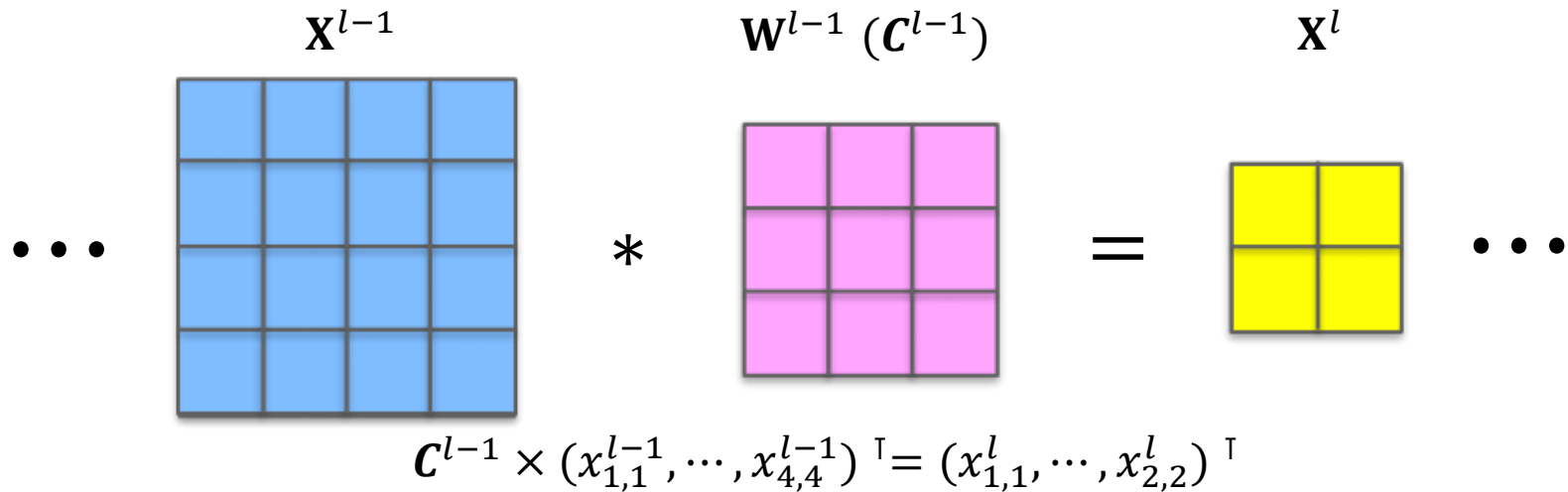
- If the input $\mathbf{X}^{l-1}$ and output $\mathbf{X}^l$ were to be unrolled into vectors, the convolution could be represented as a sparse matrix $\mathbf{C}^{l-1}$ where the non-zero elements are the elements $w_{m,n}$ of the kernel.



$$\mathbf{C}^{l-1} \times (x_{1,1}^{l-1}, \dots, x_{4,4}^{l-1})^\top = (x_{1,1}^l, \dots, x_{2,2}^l)^\top$$

$$\mathbf{C}^{l-1} = \begin{pmatrix} w_{1,1} & w_{1,2} & w_{1,3} & 0 & w_{2,1} & w_{2,2} & w_{2,3} & 0 & w_{3,1} & w_{3,2} & w_{3,3} & 0 & 0 & 0 & 0 & 0 \\ 0 & w_{1,1} & w_{1,2} & w_{1,3} & 0 & w_{2,1} & w_{2,2} & w_{2,3} & 0 & w_{3,1} & w_{3,2} & w_{3,3} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & w_{1,1} & w_{1,2} & w_{1,3} & 0 & w_{2,1} & w_{2,2} & w_{2,3} & 0 & w_{3,1} & w_{3,2} & w_{3,3} & 0 \\ 0 & 0 & 0 & 0 & 0 & w_{1,1} & w_{1,2} & w_{1,3} & 0 & w_{2,1} & w_{2,2} & w_{2,3} & 0 & w_{3,1} & w_{3,2} & w_{3,3} \end{pmatrix}$$

Back-propagation in convolutional layer



– Backward pass

$$\frac{\partial \text{Loss}}{\partial w_{m,n}} = \sum_{h,k} \frac{\partial \text{Loss}}{\partial x_{h,k}^l} \frac{\partial x_{h,k}^l}{\partial w_{m,n}},$$

,where

$$\frac{\partial x_{h,k}^l}{\partial w_{m,n}} = x_{h+m-1, k+n-1}^{l-1},$$

$$\frac{\partial \text{Loss}}{\partial x_{h,k}^l} = \frac{\partial \text{Loss}}{\partial x_i^l} = \sum_j \frac{\partial \text{Loss}}{\partial x_j^{l+1}} \frac{\partial x_j^{l+1}}{\partial x_i^l} = \sum_j \frac{\partial \text{Loss}}{\partial x_j^{l+1}} \mathbf{C}_{j,i}^l = \frac{\partial \text{Loss}}{\partial \mathbf{x}^{l+1}} \mathbf{C}_{*,i}^l = \mathbf{C}_{*,i}^{l\top} \frac{\partial \text{Loss}}{\partial \mathbf{x}^{l+1}}.$$

(Note that x_i^l represent i -th element in the $\mathbf{X}^l$. Here, $i = (h - 1) \times H + k$.)

A toy example

<https://github.com/pytorch/examples/blob/master/mnist/main.py>

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1)
        self.conv2 = nn.Conv2d(32, 64, 3, 1)
        self.dropout1 = nn.Dropout2d(0.25)
        self.dropout2 = nn.Dropout2d(0.5)
        self.fc1 = nn.Linear(9216, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.conv1(x)
        x = F.relu(x)
        x = self.conv2(x)
        x = F.max_pool2d(x, 2)
        x = self.dropout1(x)
        x = torch.flatten(x, 1)
        x = self.fc1(x)
        x = F.relu(x)
        x = self.dropout2(x)
        x = self.fc2(x)
        output = F.log_softmax(x, dim=1)
        return output
```

<https://pytorch.org/docs/stable/nn.html#convolution-layers>

<https://pytorch.org/docs/stable/nn.html#linear>

Convolution layers in PyTorch

<https://pytorch.org/docs/stable/nn.html#convolution-layers>

Conv2d

```
CLASS torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1,  
groups=1, bias=True, padding_mode='zeros')
```

[\[SOURCE\]](#)

Applies a 2D convolution over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C_{in}, H, W) and output $(N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ can be precisely described as:

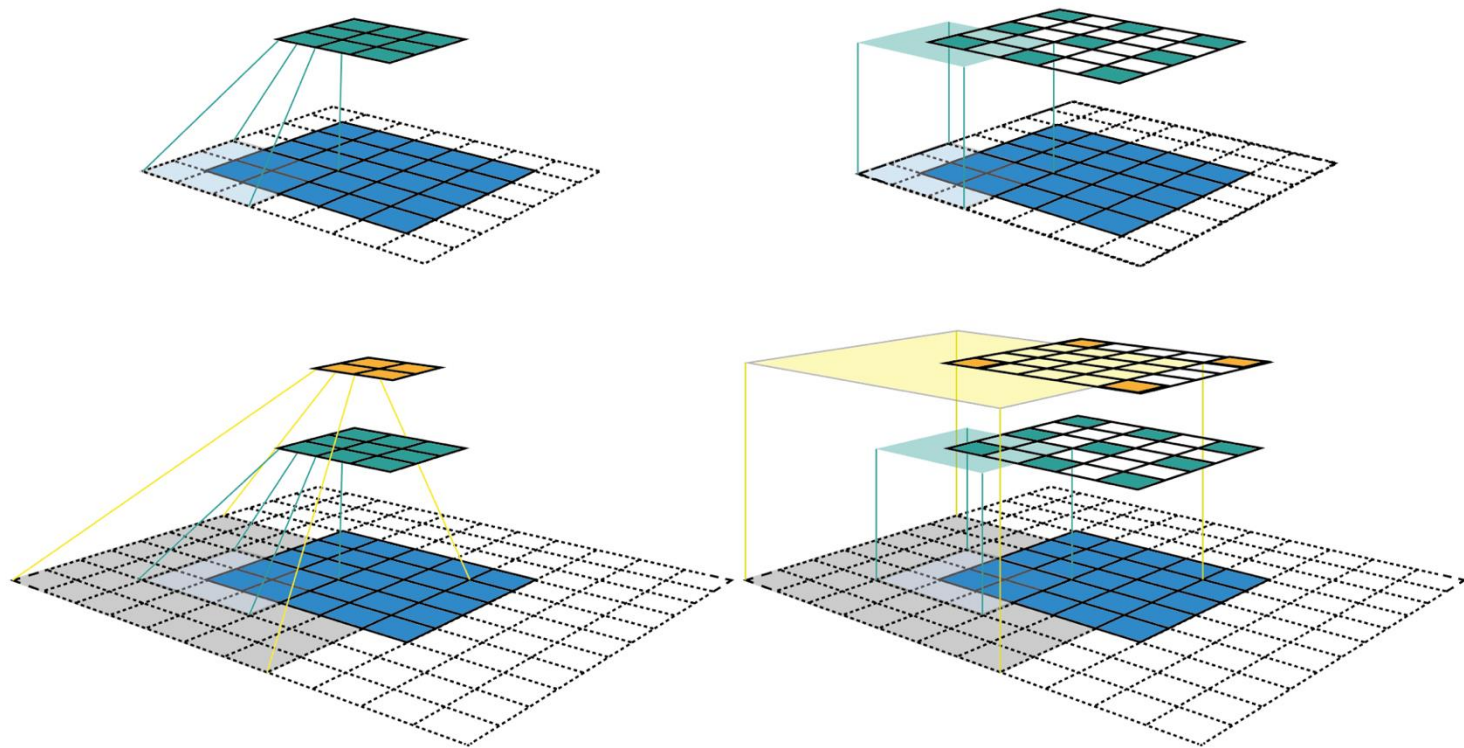
$$\text{out}(N_i, C_{\text{out}_j}) = \text{bias}(C_{\text{out}_j}) + \sum_{k=0}^{C_{\text{in}}-1} \text{weight}(C_{\text{out}_j}, k) \star \text{input}(N_i, k)$$

where $\star$ is the valid 2D **cross-correlation** operator, N is a batch size, C denotes a number of channels, H is a height of input planes in pixels, and W is width in pixels.

Receptive Field

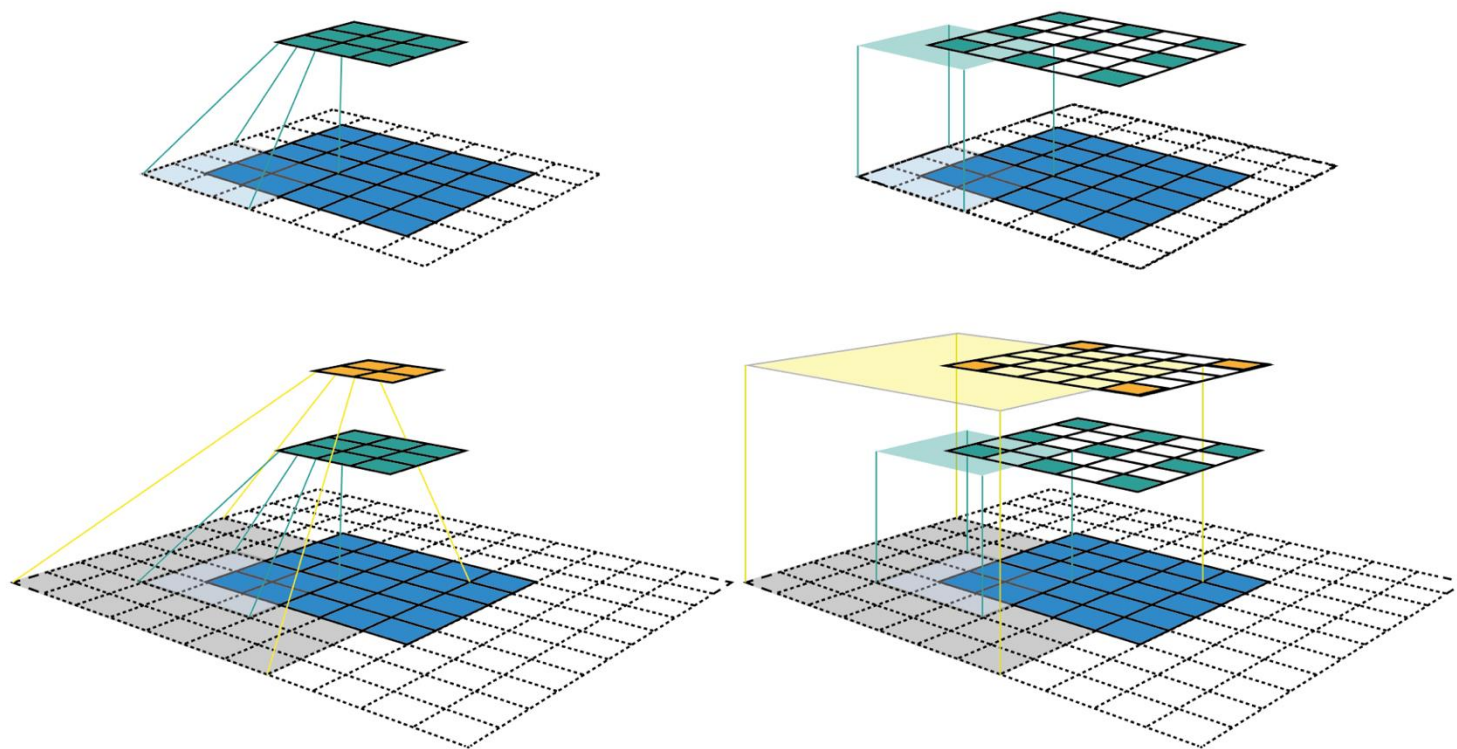
Receptive Field

- The receptive field in Convolutional Neural Networks (CNN) is **the region of the input space that affects a particular unit of the network.**
- In this example, we use the convolution filter $\mathbf{W}$ with size $K = 3 \times 3$, padding $P = 1$, stride $S = 2 \times 2$.



Receptive Field

- From the left column, we are hard to tell the receptive field size, especially for deep CNNs.
- The right column shows the fixed-sized CNN visualization, which solves the problem by keeping the size of all feature maps constant and equal to the input map. Each feature is then marked at the center of its receptive field location.



Convolution layers in PyTorch

<https://pytorch.org/docs/stable/nn.html#convolution-layers>

Conv2d

```
CLASS torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1,  
groups=1, bias=True, padding_mode='zeros') [SOURCE]
```

Applies a 2D convolution over an input signal composed of several input planes.

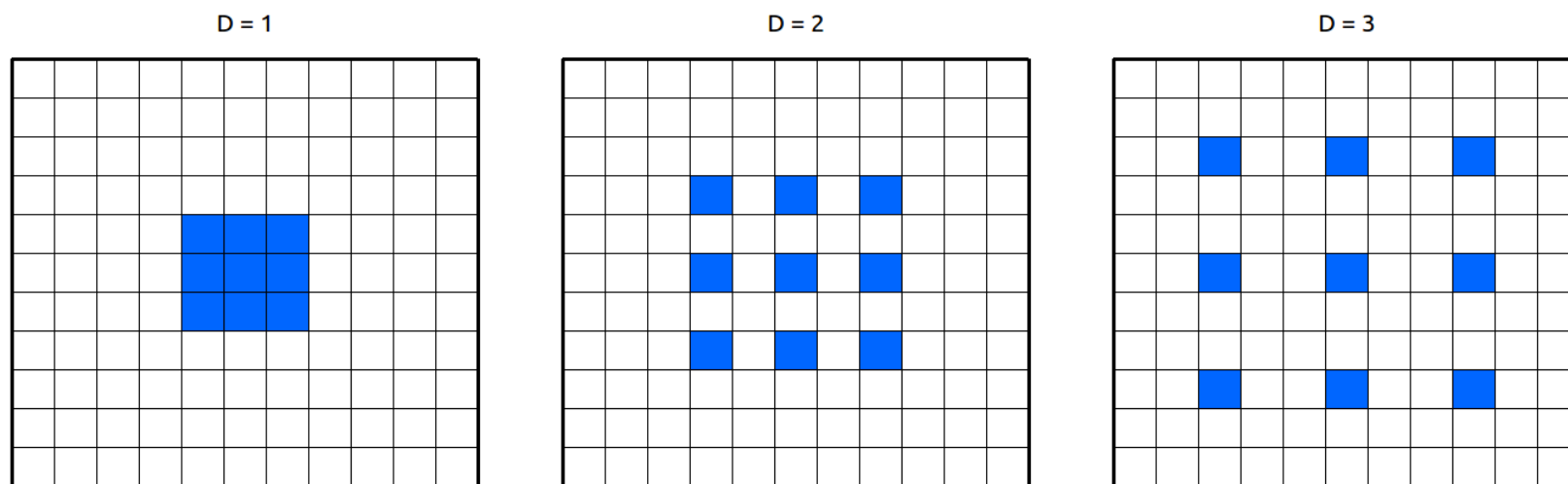
In the simplest case, the output value of the layer with input size (N, C_{in}, H, W) and output $(N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ can be precisely described as:

$$\text{out}(N_i, C_{\text{out}_j}) = \text{bias}(C_{\text{out}_j}) + \sum_{k=0}^{C_{\text{in}}-1} \text{weight}(C_{\text{out}_j}, k) \star \text{input}(N_i, k)$$

where $\star$ is the valid 2D **cross-correlation** operator, N is a batch size, C denotes a number of channels, H is a height of input planes in pixels, and W is width in pixels.

Dilated Convolution

- In simple terms, dilated convolution is just a convolution applied to input with defined gaps.
- Dilation: Spacing between kernel elements. Default: 1.
- $D=2$ means skipping one pixel per input
- The receptive field grows exponentially while the number of parameters grows linearly.



(Yu et al, 2015)

Pooling

Pooling

Max pooling

- Filter size: (2,2)
- Stride: (2,2)
- Pooling ops: $\max(\cdot)$

-1	2	0	0
0	1	3	-2
0	0	-1	4
3	-1	-2	-2

Feature map

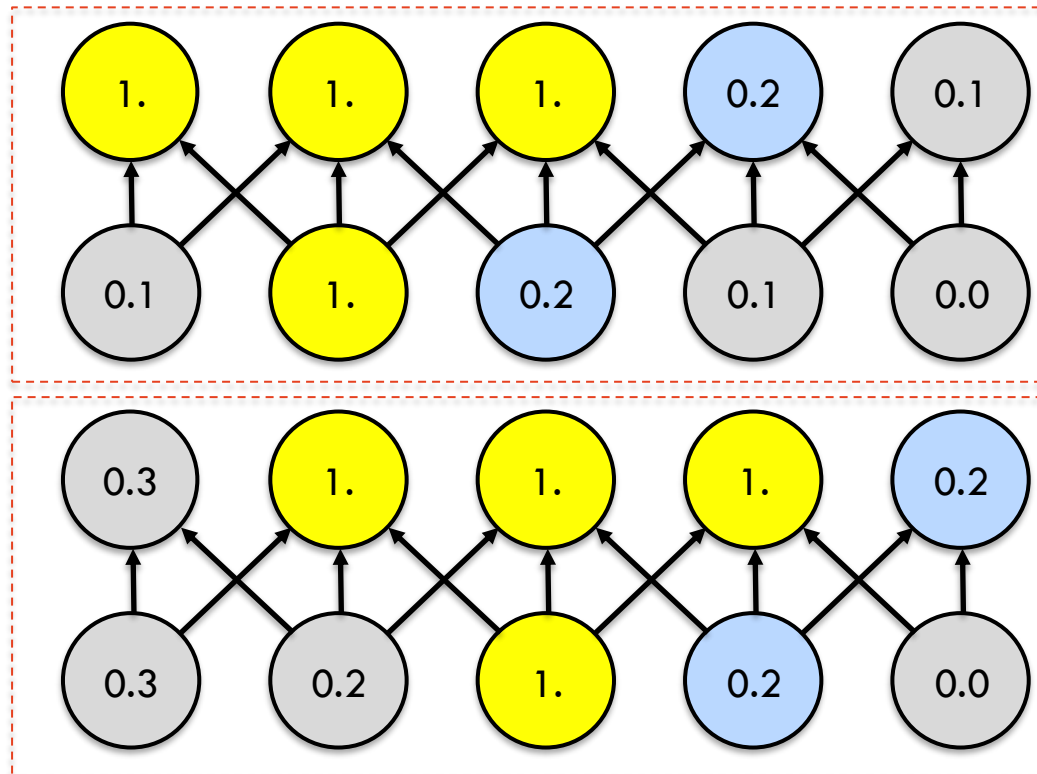


2	3
3	4

Subsample map

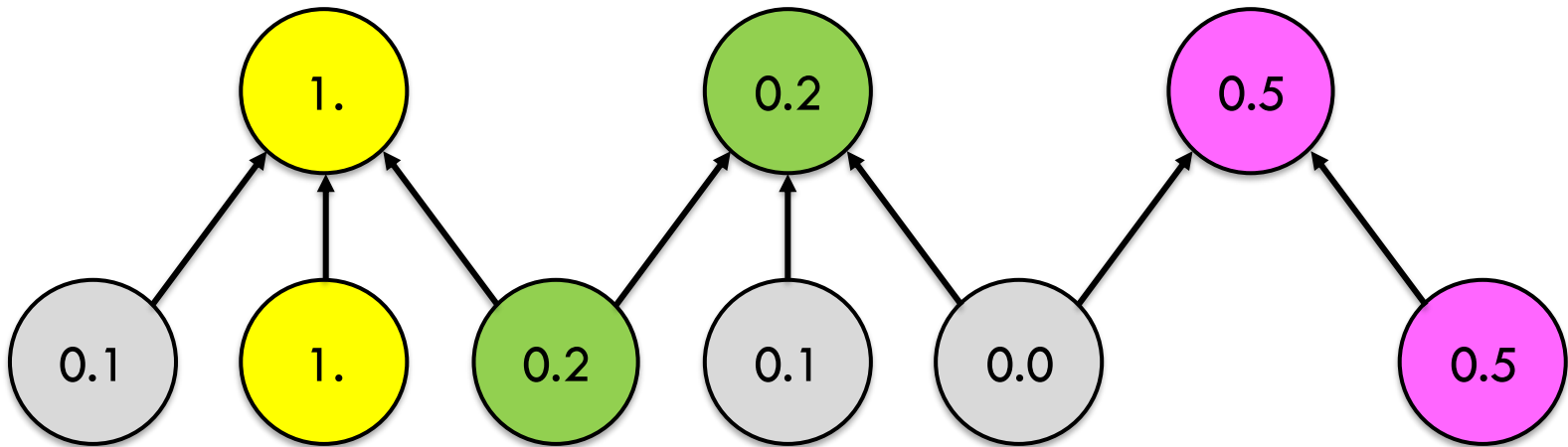
Motivation: Pooling

- Pooling helps the representation become slightly invariant to small translations of the input
 - Invariance to local translation can be a very useful property if we care more about whether some feature is present than exactly where it is
 - Taking max pooling as an example:



Motivation: Pooling

- Because pooling summarizes the responses over a whole neighbourhood, it is possible to use fewer pooling units than detector units
- Since pooling is used for down sampling, it can be used to handle inputs of varying sizes



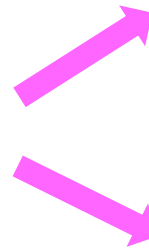
Pooling

Average pooling

- Filter size: (2,2)
- Stride: (2,2)
- Pooling ops: $\text{mean}(\cdot)$

-1	4	1	2
0	1	3	-2
1	5	-2	6
3	-1	-2	-2

Feature map



4	3
5	6

Max pooling

1	1
2	0

Average pooling

Pooling

L_2 norm pooling

- Filter size (Gaussian kernel size): (2,2)
- Stride: (2,2)

- Pooling ops: $y_i = \sqrt{\sum_j w_j x_{i,j}^2}$

$x_{1,1}$	$x_{1,2}$	$x_{2,1}$	$x_{2,2}$
$x_{1,3}$	$x_{1,4}$	$x_{2,3}$	$x_{2,4}$
$x_{3,1}$	$x_{3,2}$	$x_{4,1}$	$x_{4,2}$
$x_{3,3}$	$x_{3,4}$	$x_{4,3}$	$x_{4,4}$

Feature map

w_1	w_2
w_3	w_4

Gaussian window

y_1	y_2
y_3	y_4

Output

Pooling

- Other pooling

- **L_p pooling** (preserves the class-specific spatial/geometric information in the pooled features)

$$y_i = \left(\sum_j w_j x_{i,j}^p \right)^{1/p}$$

- **Mixed pooling** (addresses the over-fitting problem)

$$y_i = a \max(x_{i,1}, \dots, x_{i,n}) + (1 - a) \text{mean}(x_{i,1}, \dots, x_{i,n})$$

- **Stochastic pooling** (hyper-parameter free, regularizes large CNNs)

$$y_i = x_l, \text{ where } l \sim P(p_1, \dots, p_n) \text{ and } p_j = \frac{x_{i,j}}{\sum_j x_{i,j}}$$

- **Spectral pooling** (preserves considerably more information per parameter than other pooling strategies)

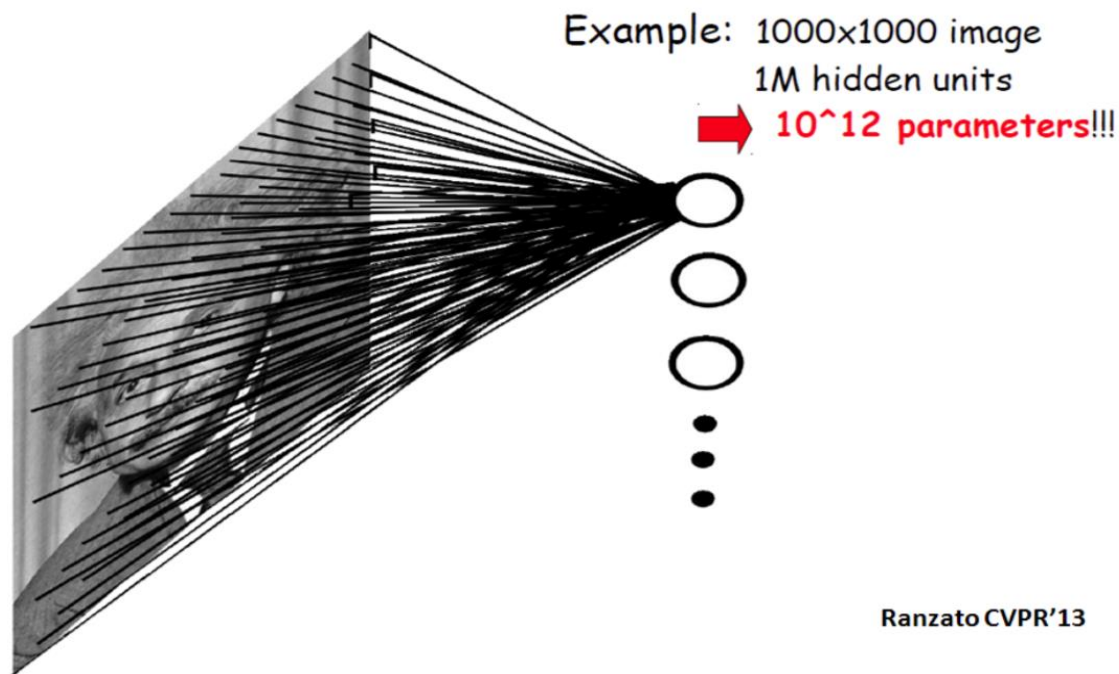
$$\mathbf{y} = \mathcal{F}(\mathbf{x}) \in \mathbb{C}^{M \times N}, \hat{\mathbf{y}} = \mathcal{F}^{-1}(\mathbf{y}) \in \mathbb{C}^{H \times W}$$

- ...

Why CNNs ?

Motivation: convolution

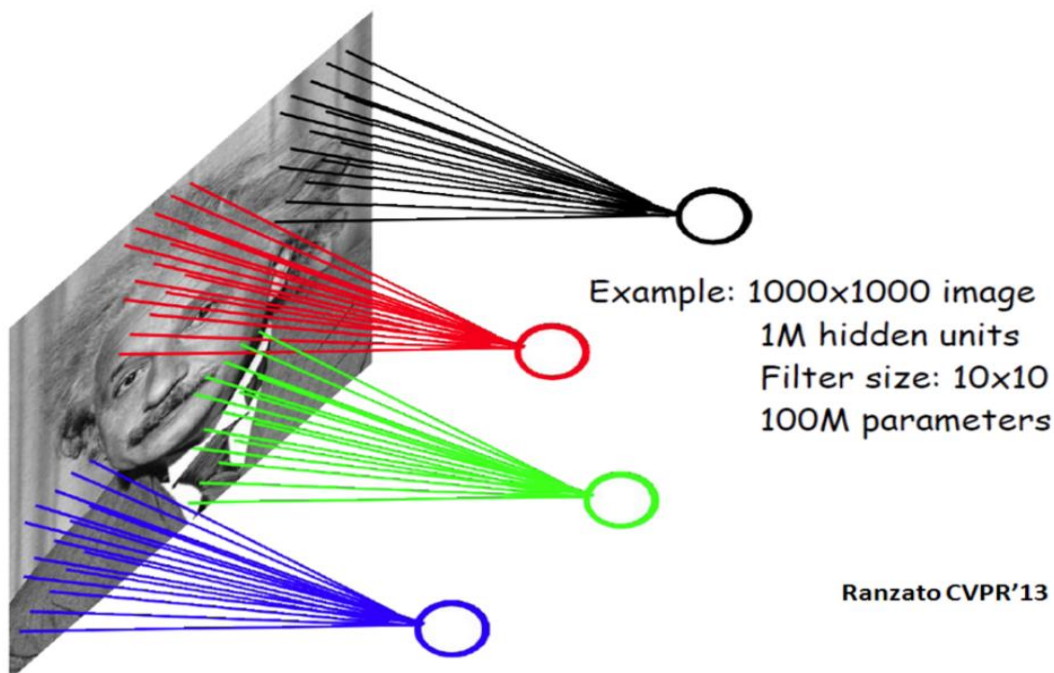
- Problems of fully connected neural networks
 - Every output unit interacts with every input unit
 - The number of weights grows fast with the size of the input image
 - Ignores the fact that far-apart pixels should be less correlated



Ranzato CVPR'13

Motivation: convolution

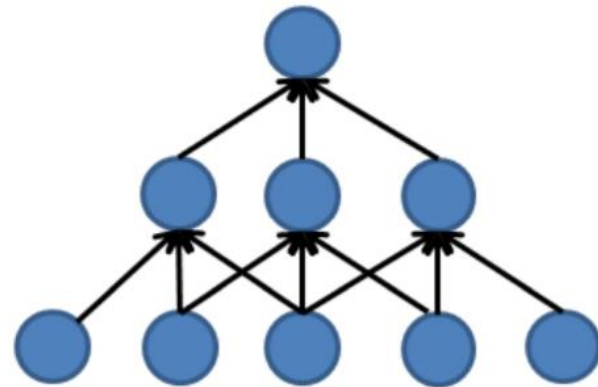
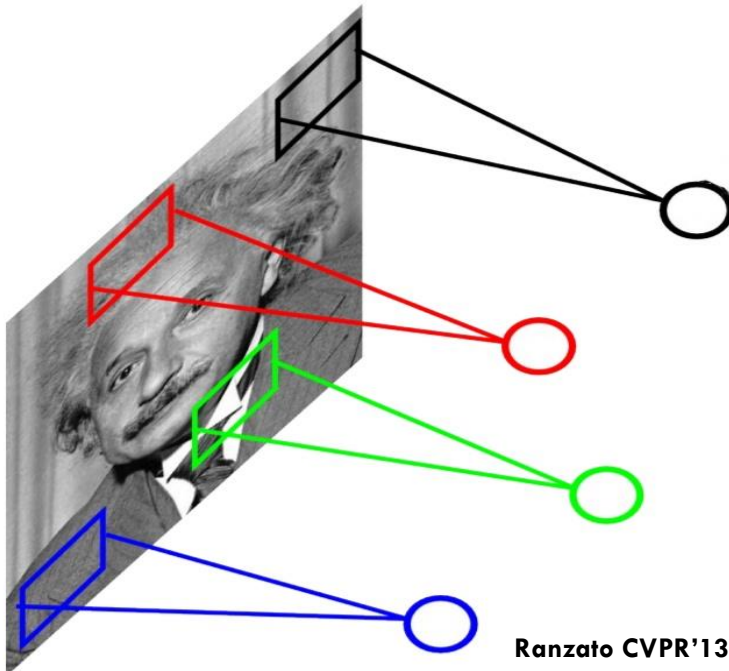
- Locally connected neural net
 - Sparse connectivity: a hidden unit is only connected to a local patch
 - It is inspired by biological systems, where a cell is sensitive to a small sub-region, called a receptive field.
 - Here, the receptive field can be called as filter or kernel



Ranzato CVPR'13

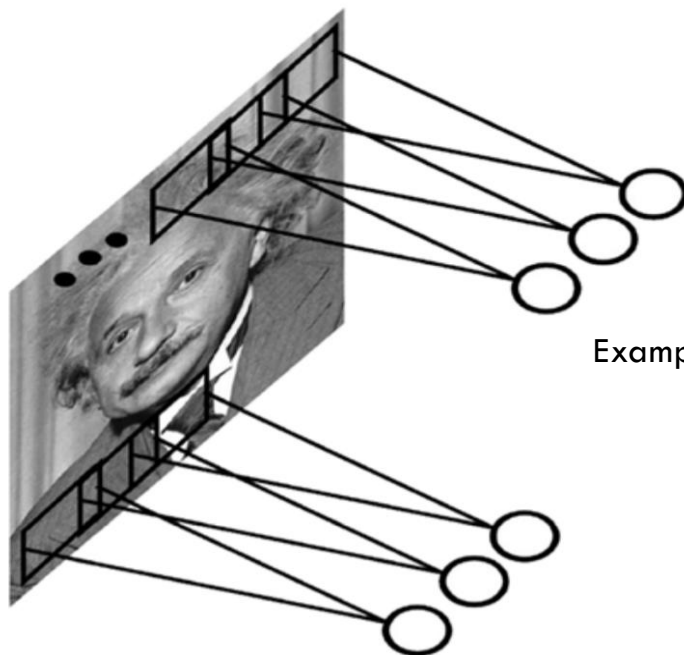
Motivation: convolution

- Problems of locally connected neural net
 - The learned filter is a spatially local pattern, i.e. lack of global view. Can stack layers of filter to increase receptive field but features will become highly non-linear.
 - Local pattern extraction makes learning inefficient
 - Lacks translation invariance



Motivation: convolution

- Convolution: shared weights
 - Translation invariance: capture statistics in local patches and they are independent of locations
 - Hidden nodes at different locations share the same weights. It greatly reduces the number of parameters to learn

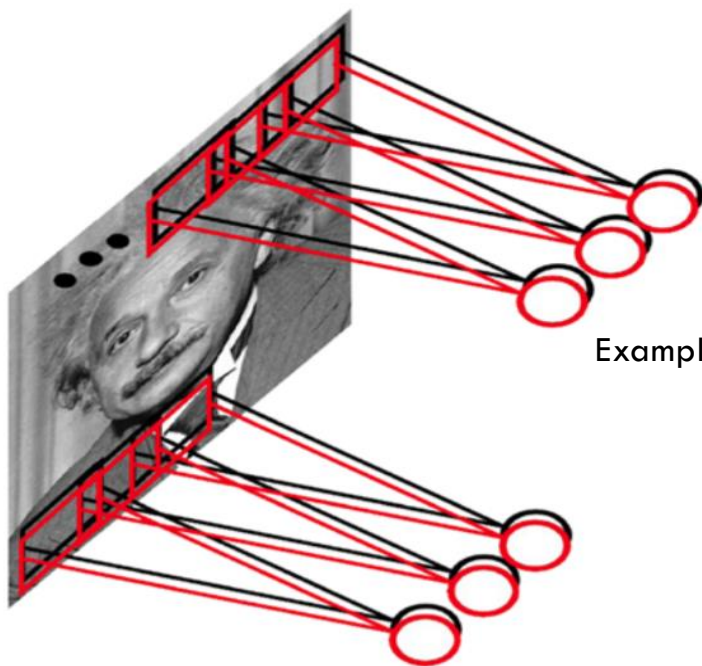


Example: 1000x1000 image
1 Filters
Filter size: 10x10
100 parameters

Ranzato CVPR'13

Motivation: convolution

- Multiple filters
 - Multiple filters provide the probability of detecting the spatial distributions of multiple visual patterns
 - One filter can build a feature map, multiple filters will build a stack of feature maps



Example: 1000x1000 image
100 Filters
Filter size: 10x10
10k parameters

Ranzato CVPR'13

Motivation: convolution

- Multiple filters: intuitive examples



Input

Image
blurring

0	0	0	0	0
0	1	1	1	0
0	1	1	1	0
0	1	1	1	0
0	0	0	0	0



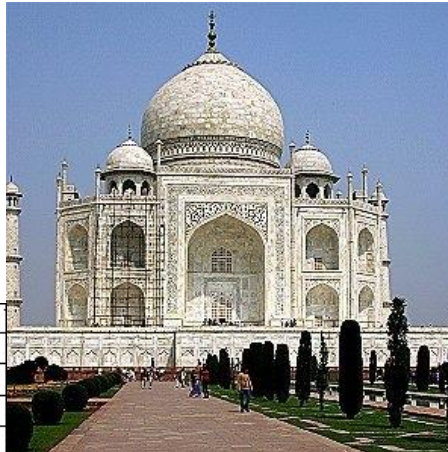
Edge
detection

0	0	0	0	0
0	0	1	0	0
0	1	-4	1	0
0	0	1	0	0
0	0	0	0	0



Image
enhancement

0	0	0	0	0
0	0	-1	0	0
0	-1	5	-1	0
0	0	-1	0	0
0	0	0	0	0



Vertical
detection

0	0	0	0	0
0	0	0	0	0
0	-1	1	0	0
0	0	0	0	0
0	0	0	0	0



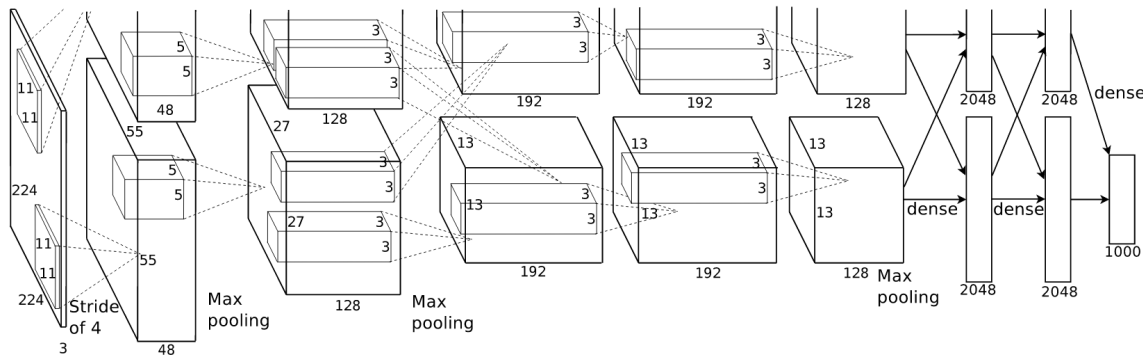
Visualize features

Visualize features

Hierarchical Convolution,
Nonlinear operations (ReLU, max pooling...)



Input image



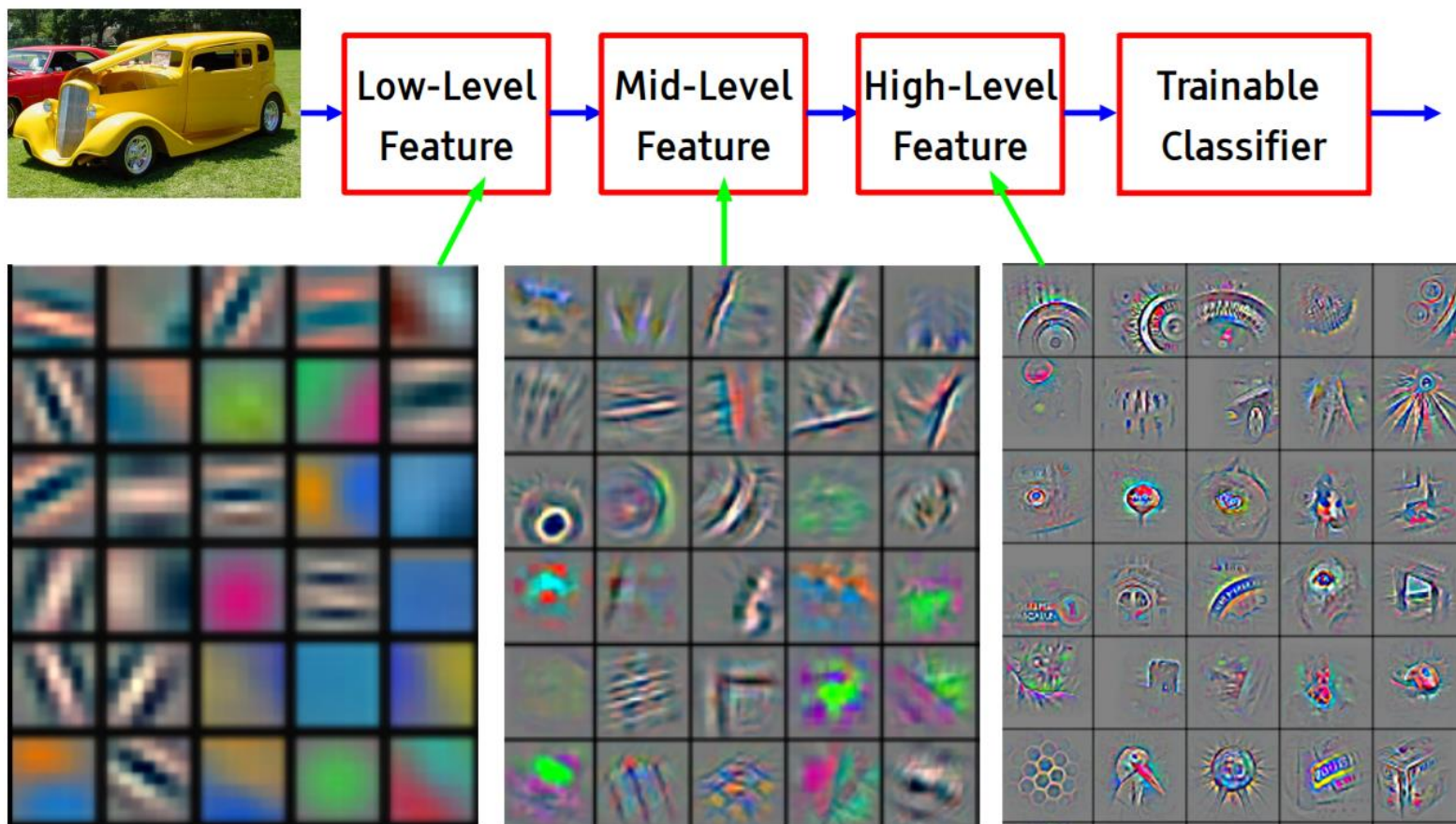
Class Scores:
1000 numbers



What happens inside hidden layers?

Visualize features

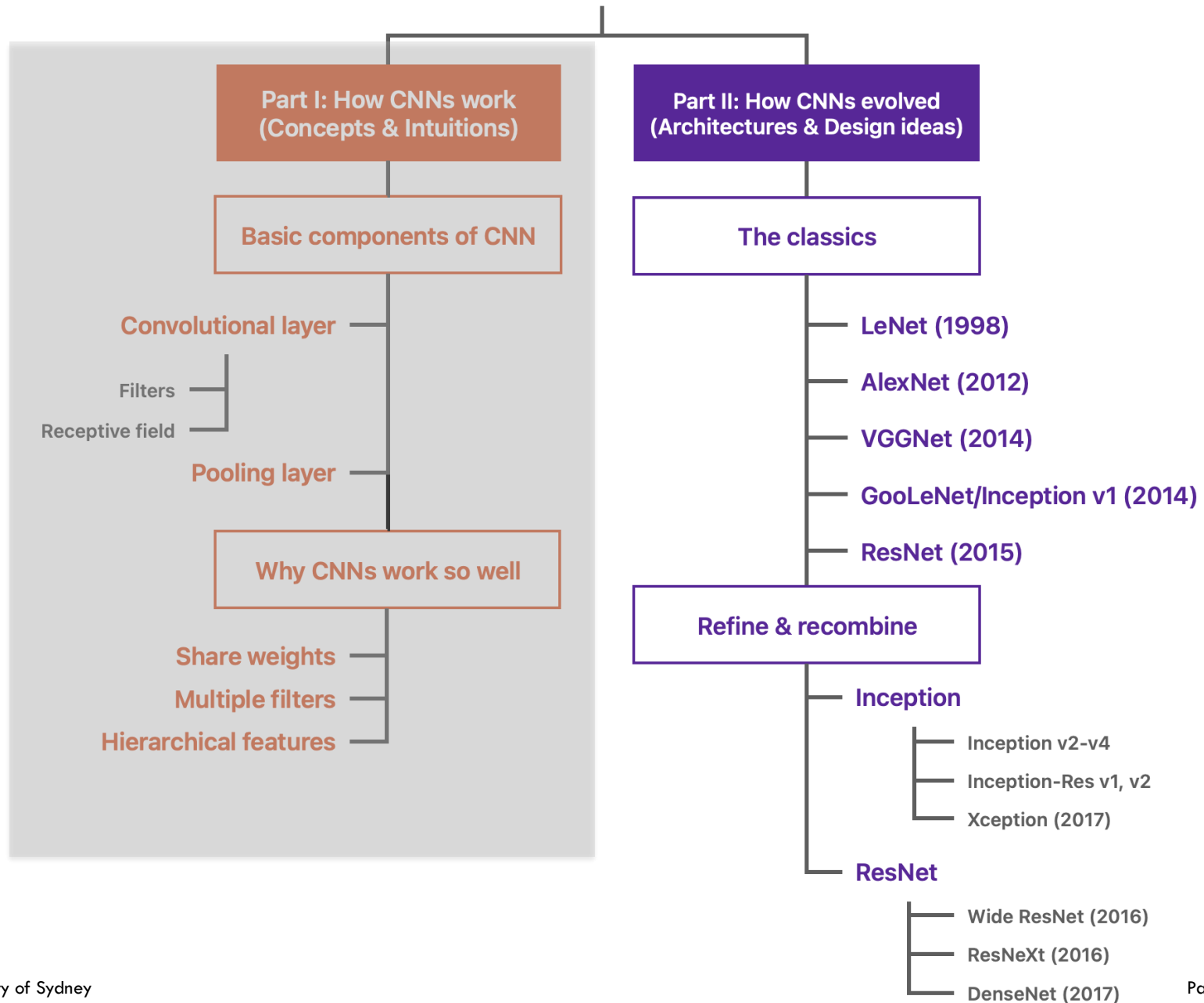
- Give insight into the function of intermediate feature layers and the operation of the classifier



(Zeiler and Fergus, 2014)

Neural Network Architectures

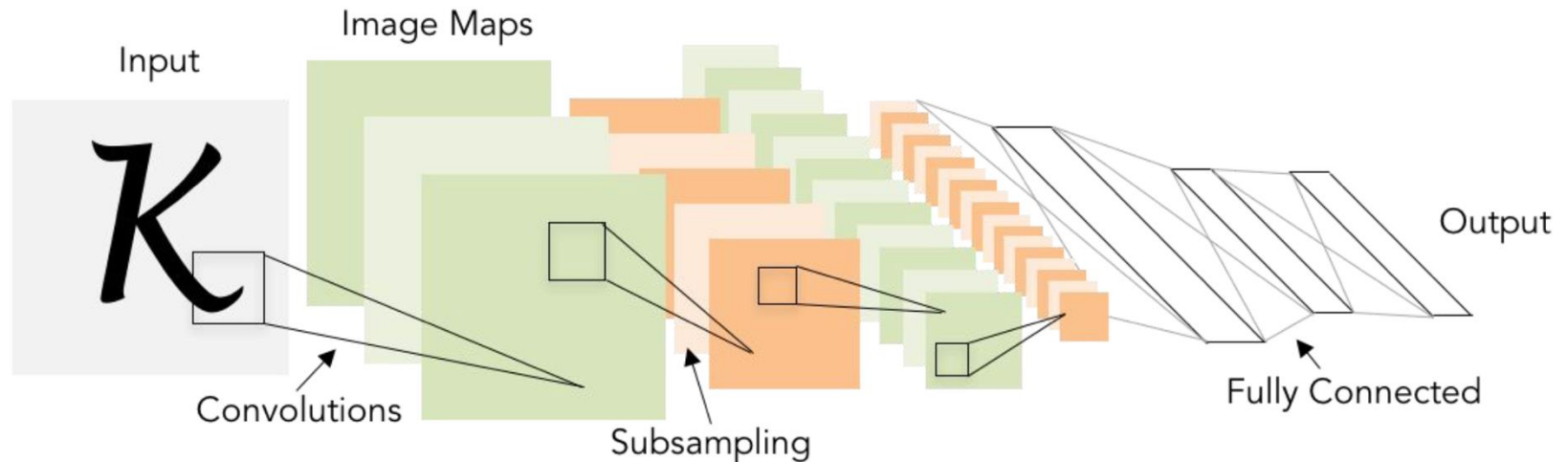
Convolutional Neural Network



CNN Architecture: Part I

□ LeNet [LeCun et al., 1998]

“Does CNN work at all?”

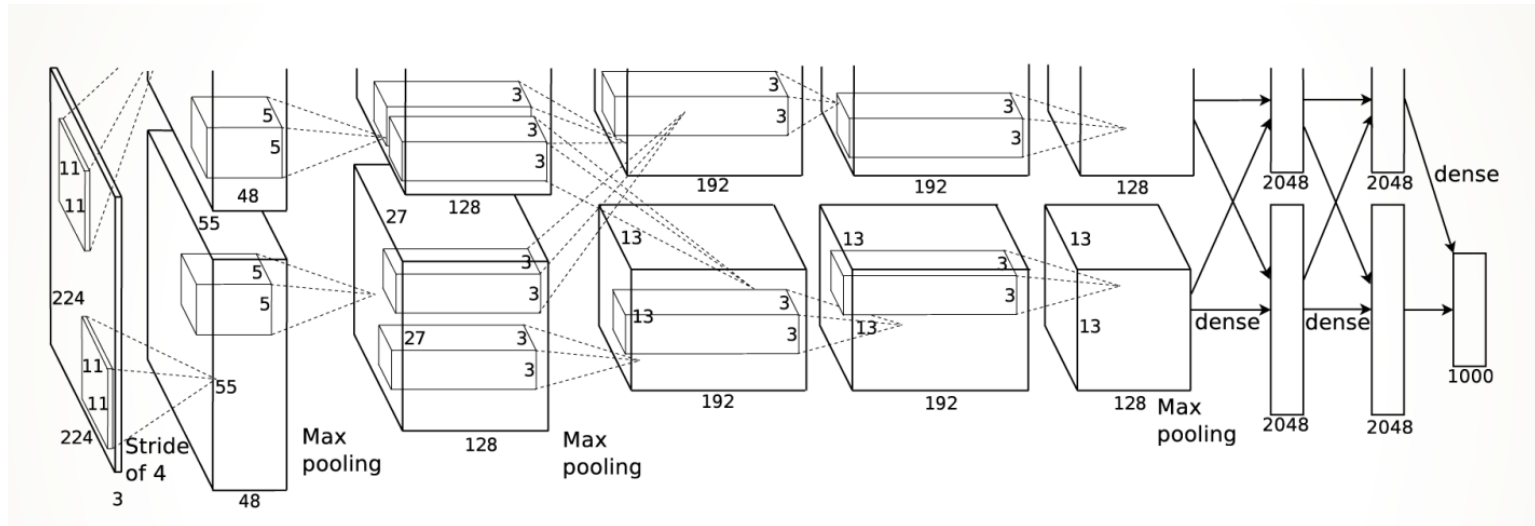


- Architecture is [CONV-POOL-CONV-POOL-FC-FC]
- CONV: 5x5 filter, stride=1
- POOL: 2x2 filter, stride=2

CNN Architecture: Part I

□ AlexNet [Krizhevsky et al. 2012]

“Can CNN scale up?”



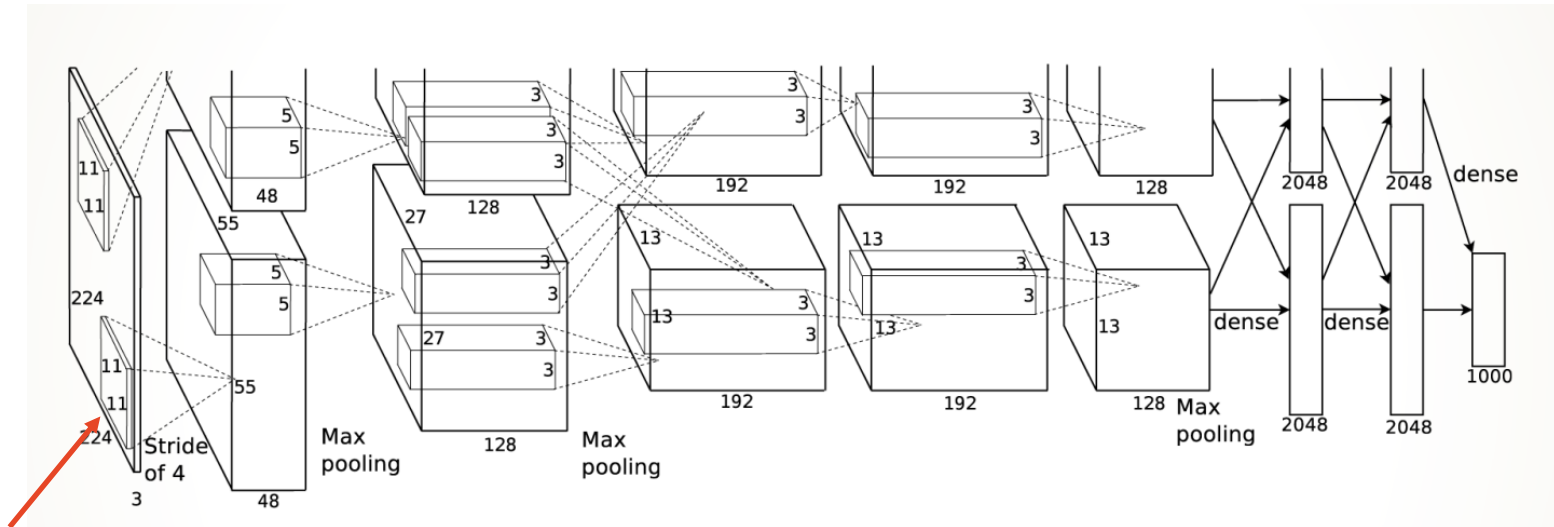
ILSVRC2010: 28.2%

ILSVRC2011: 25.8%

ILSVRC2012: 16.4% (runner-up 26.2%)

CNN Architecture: Part I

□ **AlexNet** [Krizhevsky et al. 2012] -5 conv layers, 3 fully connected layers.



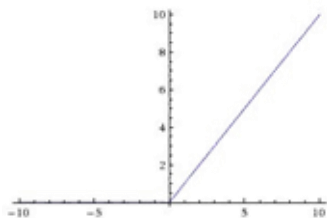
11x11 filters

- Activation function: ReLU
- Data Augmentation
- Dropout (drop rate=0.5)
- Local Response Normalization
- Overlapping Pooling

CNN Architecture: Part I

□ **AlexNet** [Krizhevsky et al. 2012] -5 conv layers, 3 fully connected layers.

- Activation function: ReLU
- Data Augmentation
- Dropout
- Local Response Normalization
- Overlapping Pooling



$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$$

$$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$$

Pros:

- Easy to feed forward
- Easy to back propagate
- Help prevent saturation
- Sparse

Cons:

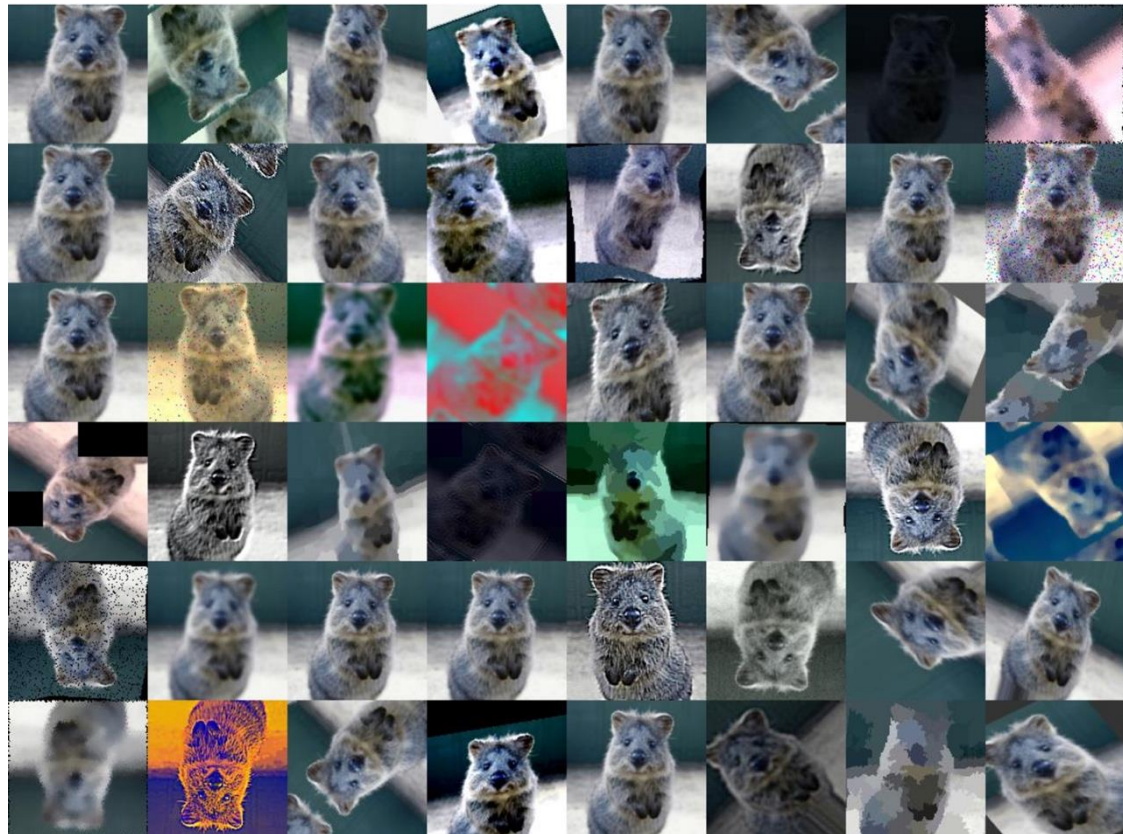
- Dead ReLU

CNN Architecture: Part I

□ AlexNet [Krizhevsky et al. 2012]

- Activation function: ReLU
- Data Augmentation
- Dropout
- Local Response Normalization
- Overlapping Pooling

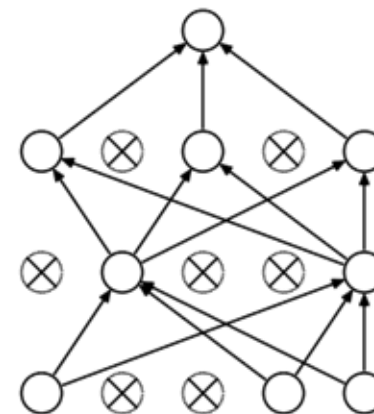
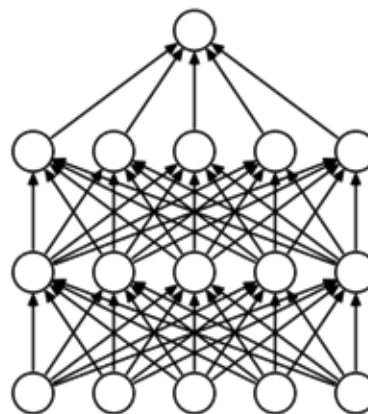
- Randomly Rotate Images
- Flip Images
- Shift Images
- Contrast Stretching
- Adaptive Equalization
- etc.



CNN Architecture: Part I

□ AlexNet [Krizhevsky et al. 2012]

- Activation function: ReLU
- Data Augmentation
- Dropout
- Local Response Normalization
- Overlapping Pooling



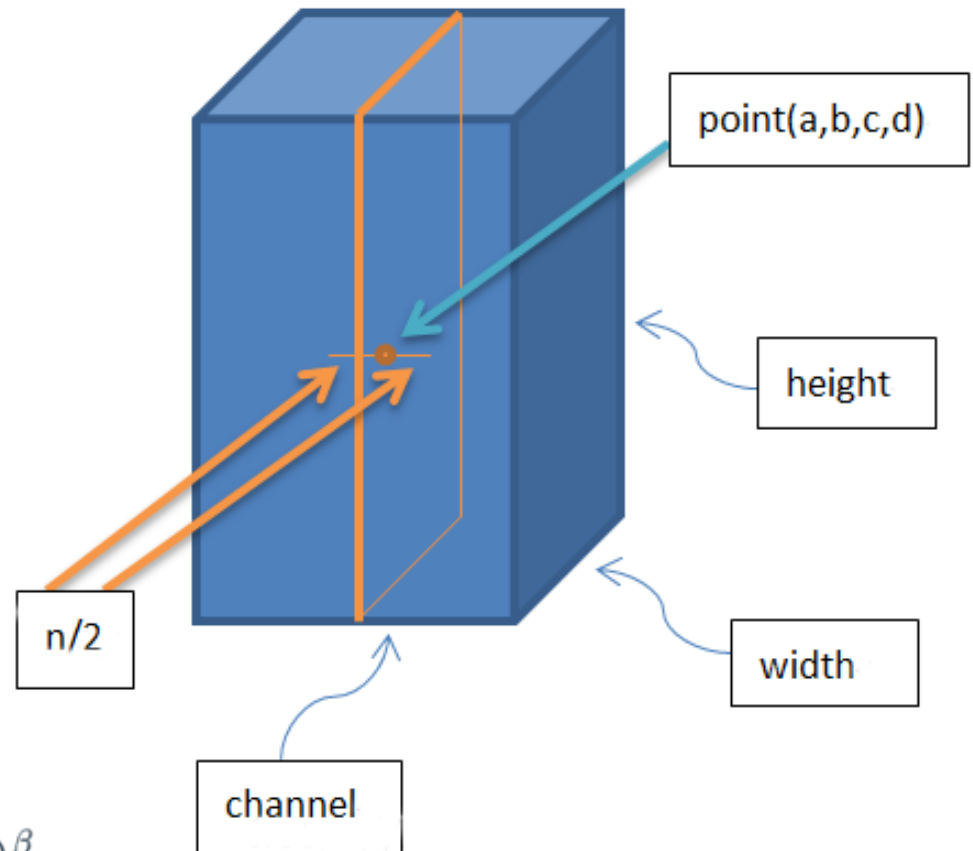
In practice, dropout trains 2^n networks (n – number of units).

Dropout is a technique which deal with overfitting by combining the predictions of many different large neural nets at test time.

CNN Architecture: Part I

□ AlexNet [Krizhevsky et al. 2012]

- Activation function: ReLU
- Data Augmentation
- Dropout
- Local Response Normalization
- Overlapping Pooling



$$b_{x,y}^i = a_{x,y}^i / (k + \alpha \sum_{j=\max(0, i-n/2)}^{\min(N-1, i+n/2)} (a_{x,y}^j)^2)^\beta$$

CNN Architecture: Part I

□ AlexNet [Krizhevsky et al. 2012]

- Activation function: ReLU
- Data Augmentation
- Dropout
- Local Response Normalization
- **Overlapping Pooling**

-1	4	1	2
0	1	3	-2
1	5	-2	6
3	-1	-2	-2

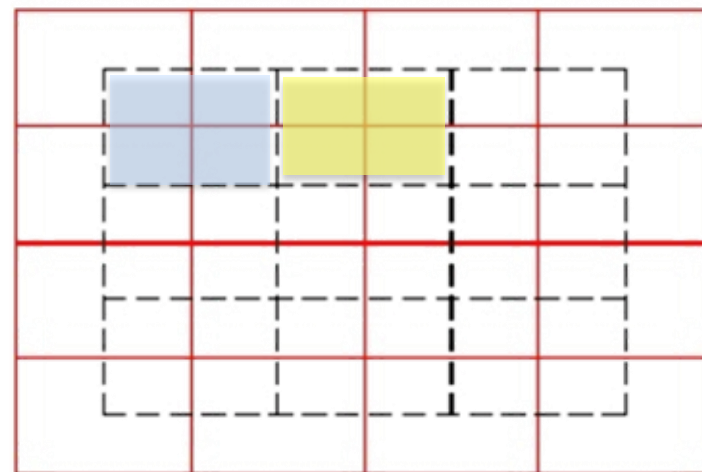
Feature map

4	3
5	6

Max pooling

1	1
2	0

Average pooling



Pooling stride < Pooling kernel size

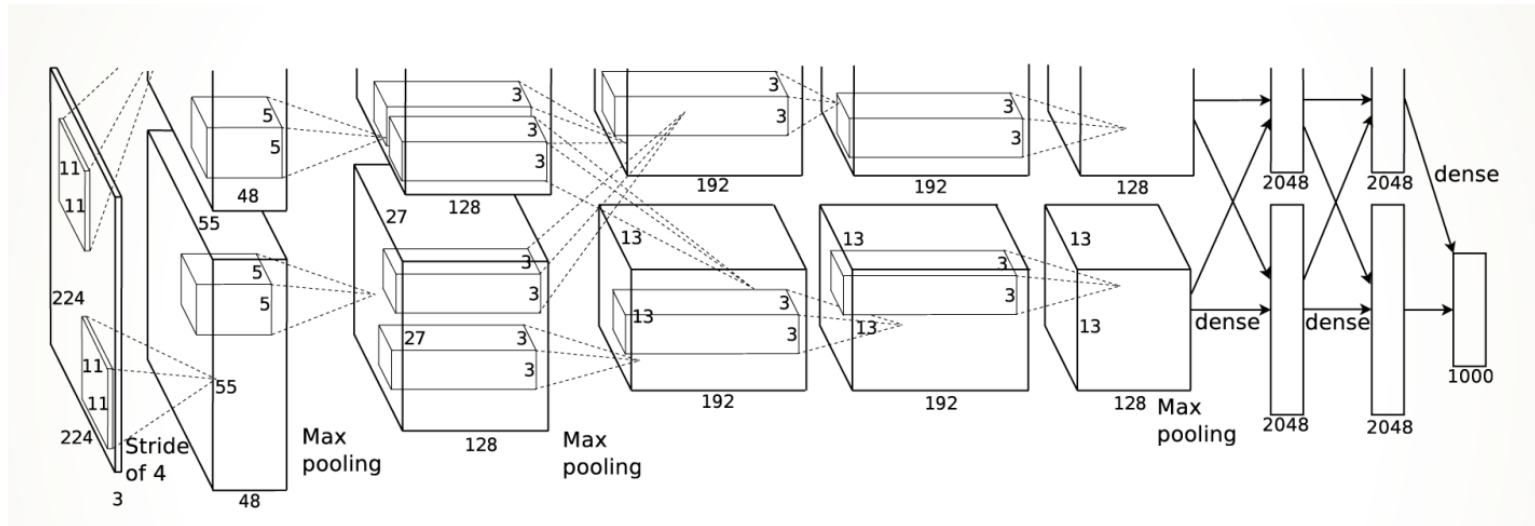
Overlapping Pooling

Standard Pooling

Pooling stride = Pooling kernel size

CNN Architecture: Part I

□ AlexNet [Krizhevsky et al. 2012]



The problem set is divided into 2 parts, half executing on GPU 1 & another half on GPU 2.

CNN Architecture: Part I

❑ VGGNet [Simonyan and Zisserman, 2014]

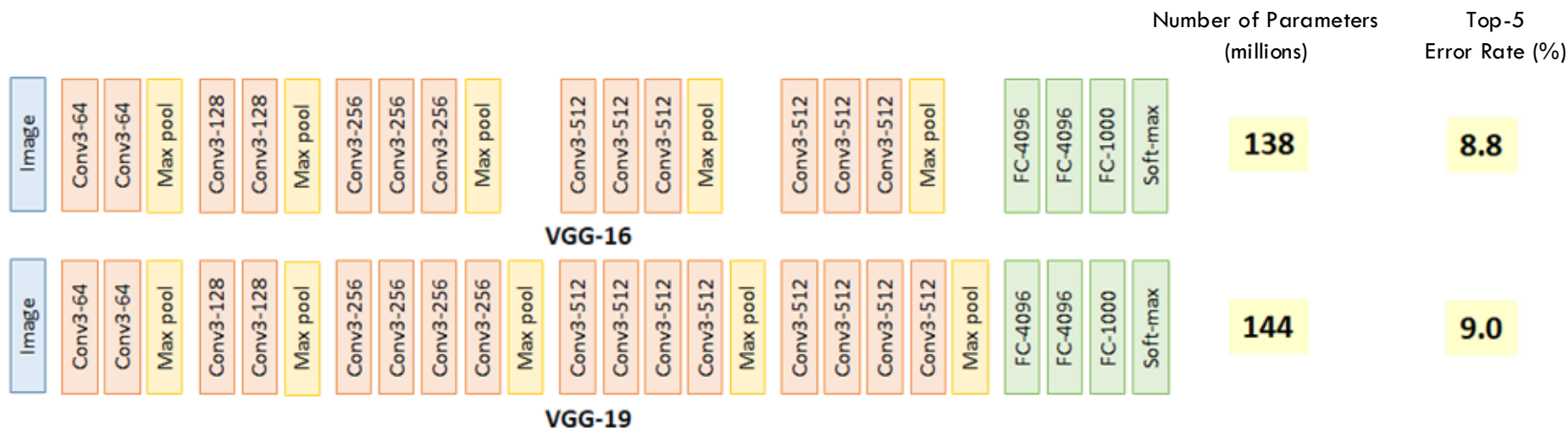
“Does going deeper help?”

Small filters:

- 3x3 convolutional layers (stride 1, pad 1)
- 2x2 max-pooling, stride 2

Deeper networks:

- AlexNet: 8 layers
- VGGNet: 16 or 19 layers

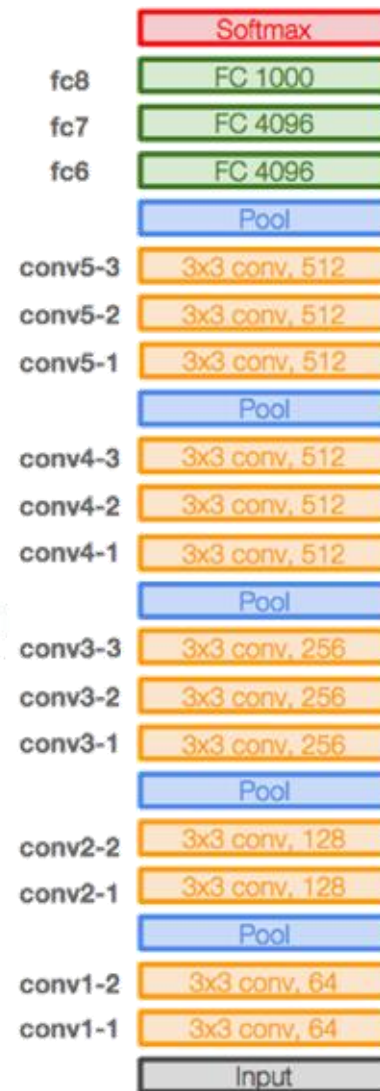
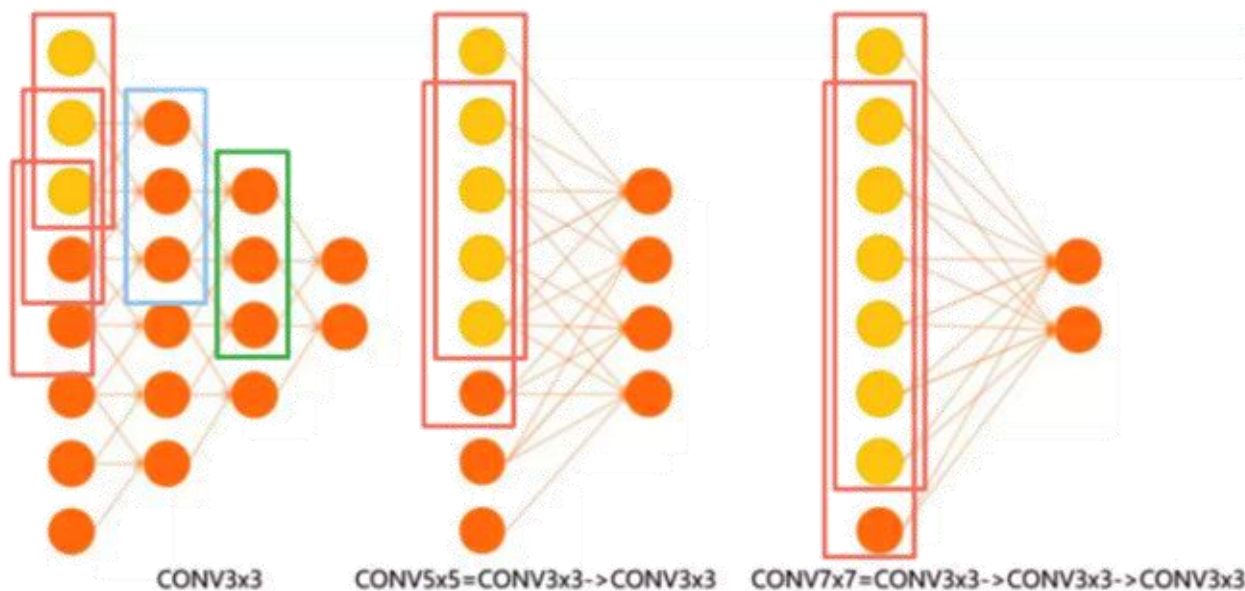


CNN Architecture: Part I

Why small filters?

- Two 3x3 layer = 5x5 layer
Three 3x3 layer = 7x7 layer
- Deeper, more non-linearity
- Less parameters

Receptive Field (RF): the region in the input space that a particular CNN's feature is looking at (i.e. be affected by).



VGG16

CNN Architecture: Part I

Why small filters?

- Two 3x3 layer = 5x5 layer
Three 3x3 layer = 7x7 layer
- Deeper, more non-linearity
- Less parameters

Why popular?

- FC7 features generalize well to other tasks
- Plain network structure

The rule for network design

- Prefer a stack of small filters to a large filter



VGG16

Image credit to: http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture9.pdf

CNN Architecture: Part I

❑ GoogLeNet [Szegedy et al., 2014]

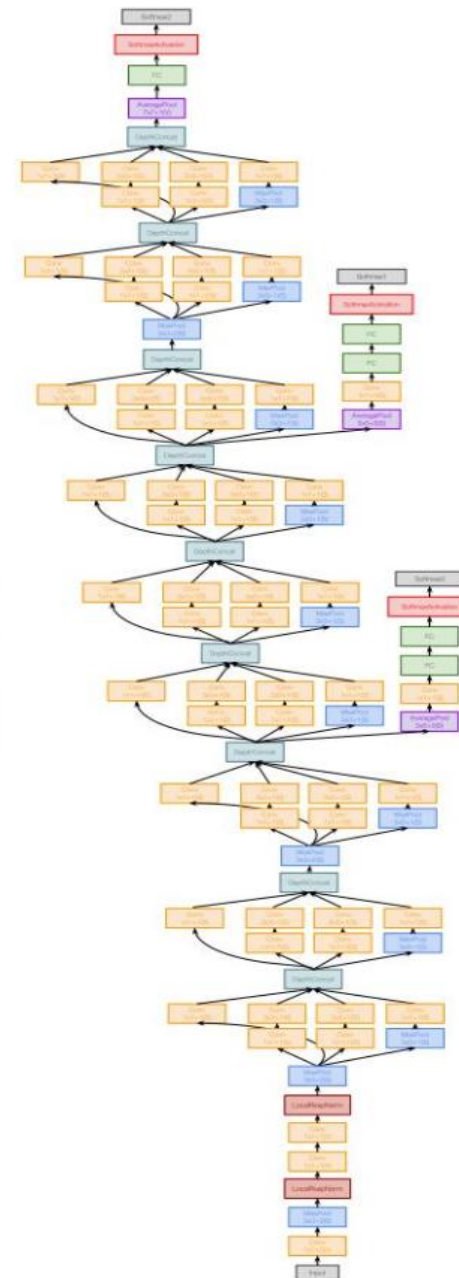
“Can we go deeper efficiently?”

Deeper networks

- 22 layers.
- Auxiliary loss

Computational efficiency

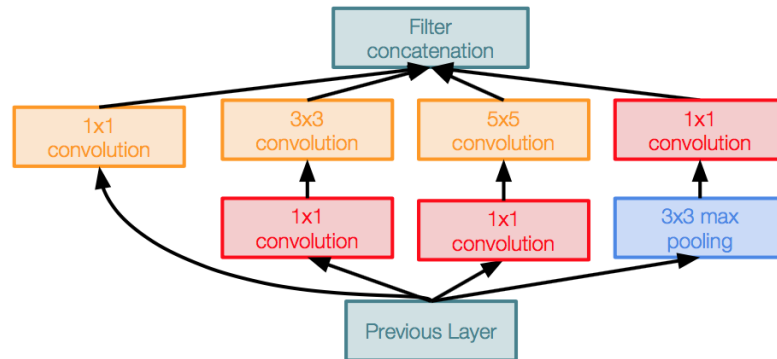
- Inception module
- Remove FC layer, use global average pooling
- 12x less parameters than AlexNet



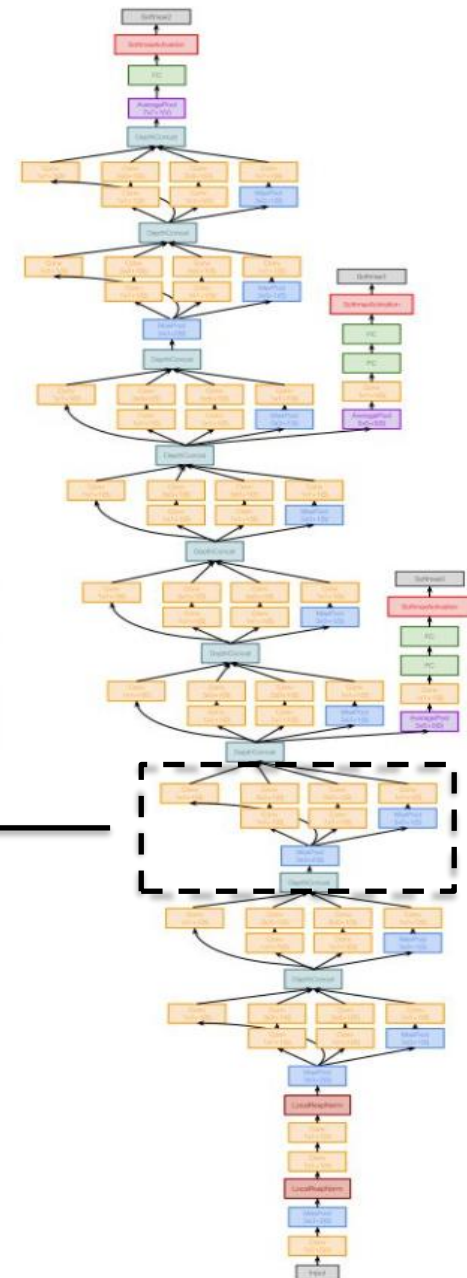
CNN Architecture: Part I

GoogLeNet [Szegedy et al., 2014]

Design a good local network topology (NIN, [Lin et al., 2013]) and then stack these modules on top of each other.



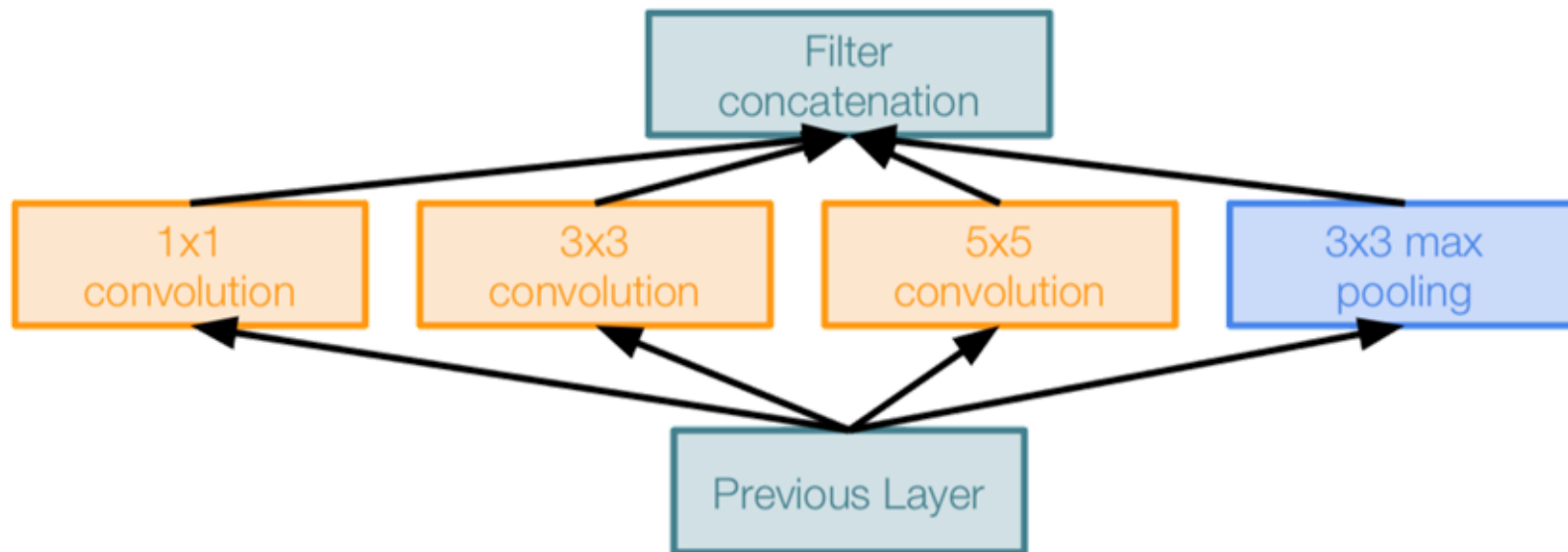
Inception module



CNN Architecture: Part I

❑ GoogLeNet [Szegedy et al., 2014]

- Different receptive fields
- The same feature dimension
- The effectiveness of pooling

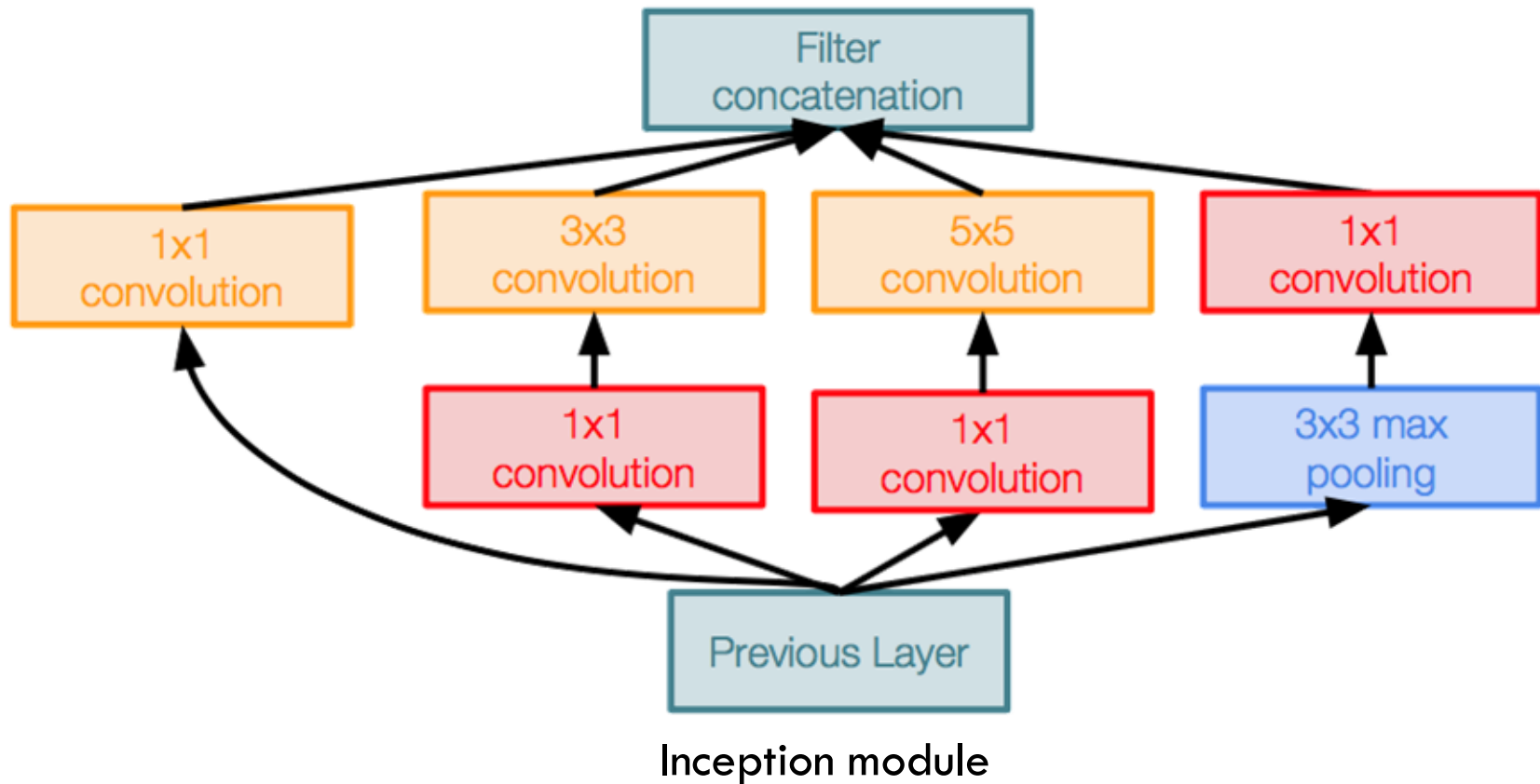


Naive Inception module

$$\text{Output Size} = \frac{N + 2P - K}{S} + 1$$

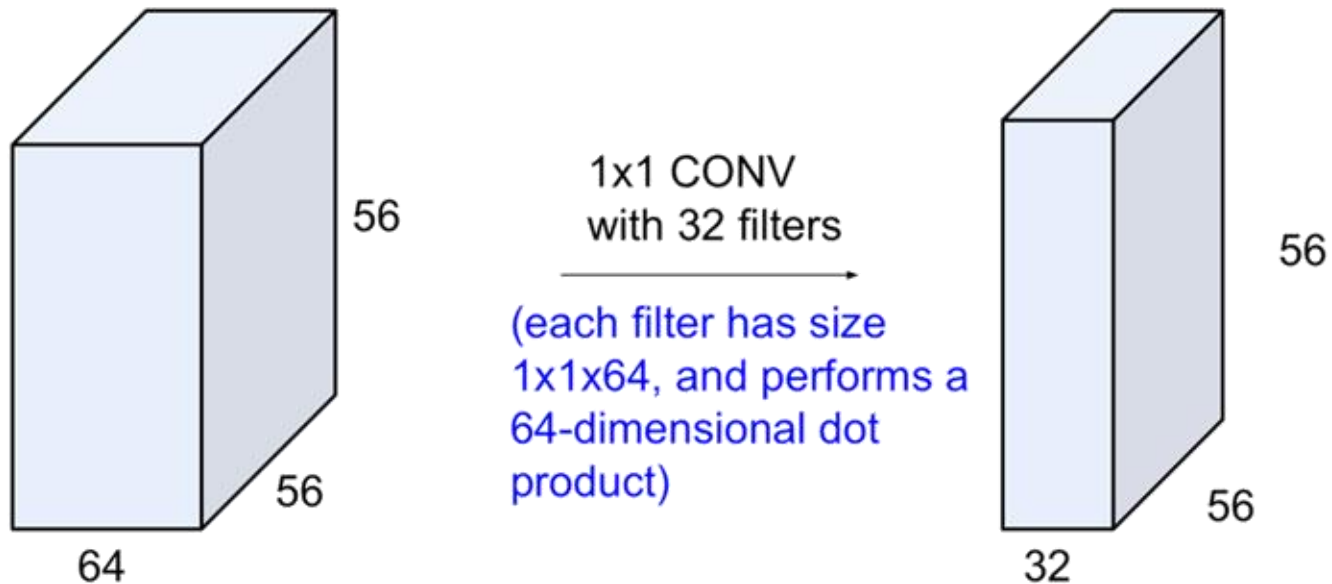
CNN Architecture: Part I

❑ GoogLeNet [Szegedy et al., 2014]



CNN Architecture: Part I

❑ GoogLeNet [Szegedy et al., 2014]



1x1 convolutional layer

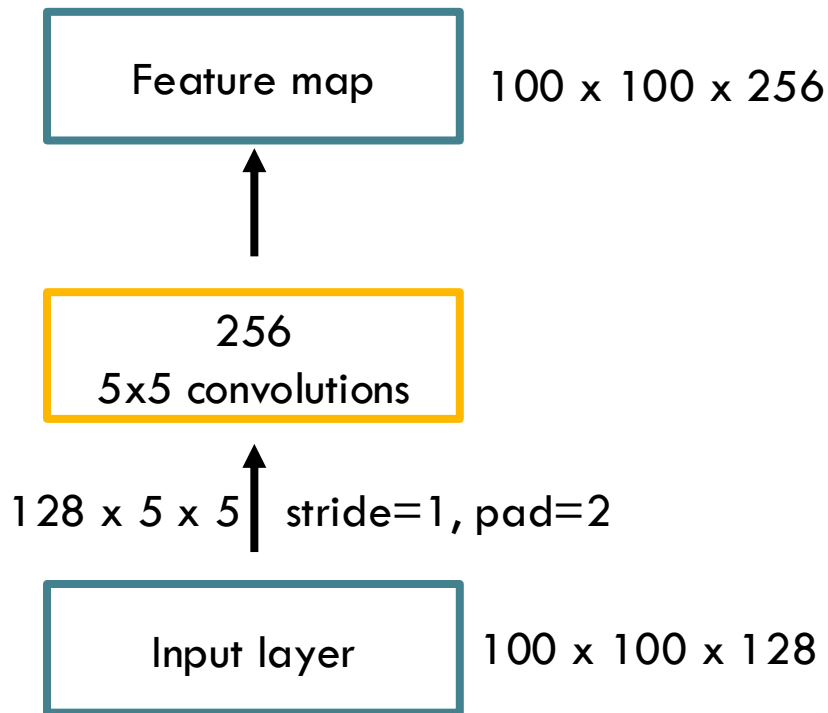
- preserve spatial dimensions (56x56)
- reduces depth (64 $\rightarrow$ 32)

CNN Architecture: Part I

❑ GoogLeNet [Szegedy et al., 2014]

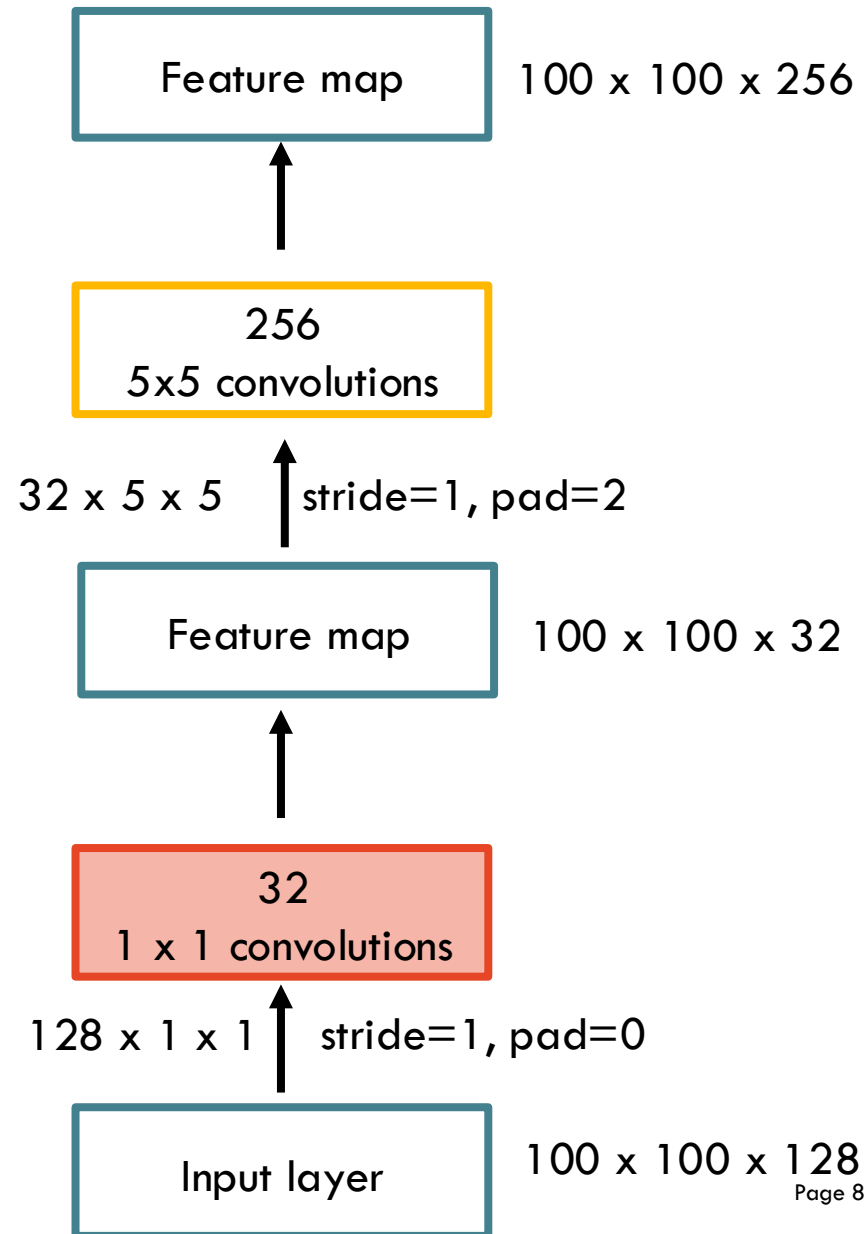
Without 1x1 filters

#parameter: $(128 \times 5 \times 5) \times 256$



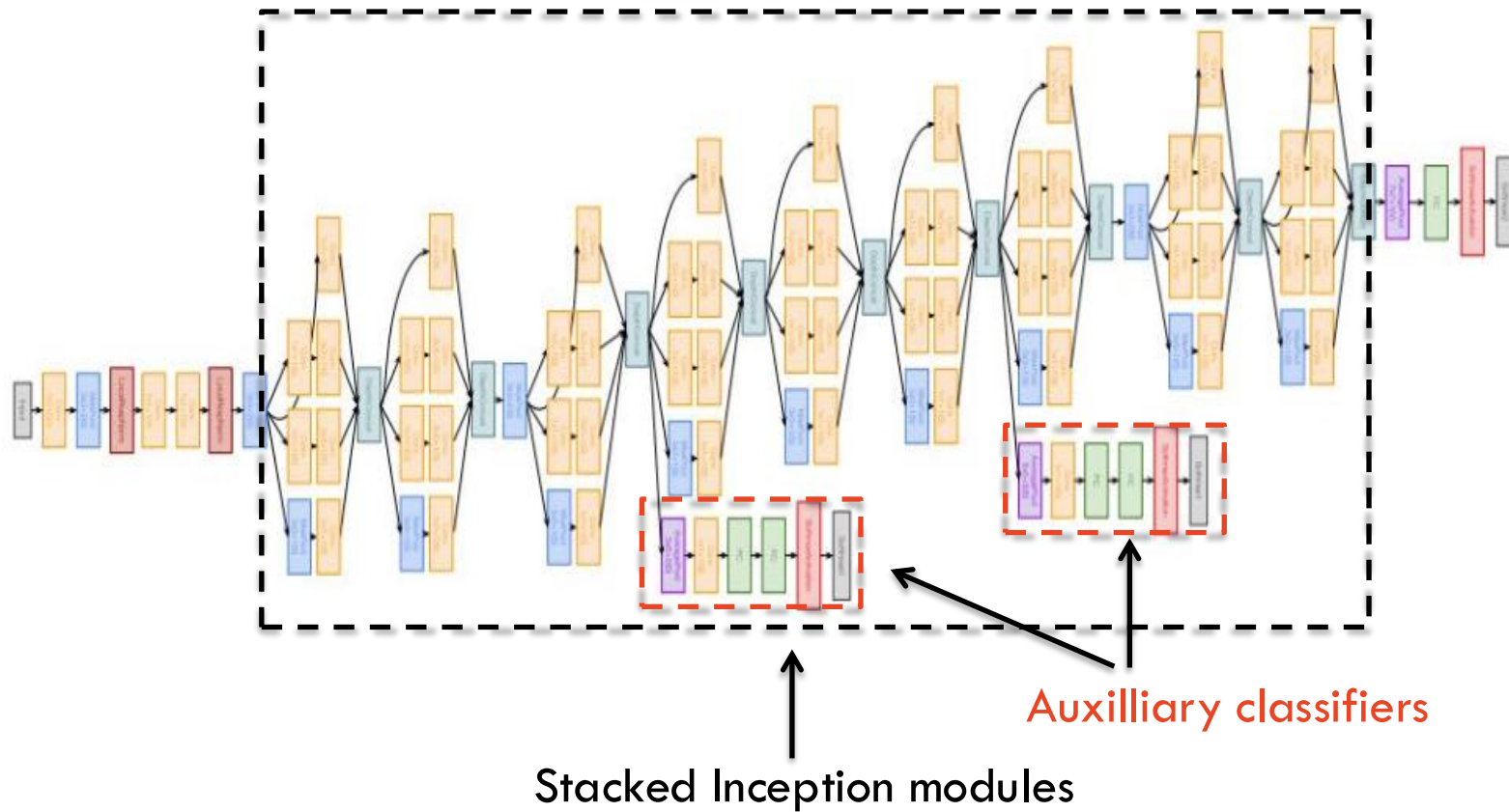
With 1x1 filters

#parameter: $(128 \times 1 \times 1) \times 32 + (32 \times 5 \times 5) \times 256$



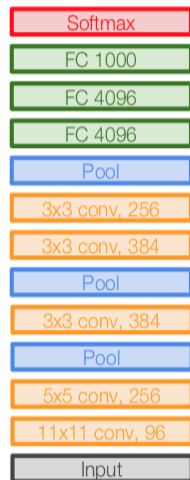
CNN Architecture: Part I

□ GoogLeNet [Szegedy et al., 2014]



CNN Architecture: Part I

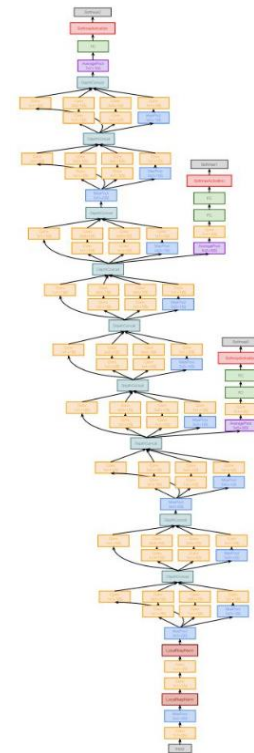
The deeper model should be able to perform at least as well as the shallower model.



AlexNet (9 layers)



VGG16 (16 layers)



GoogLeNet (22 layers)

CNN Architecture: Part I

❑ ResNet [He et al., 2015]

“Why does very deep networks break?”

Stacking deeper layers on a “plain” convolutional neural network
=> the deeper model performs worse. But it's not overfitting.

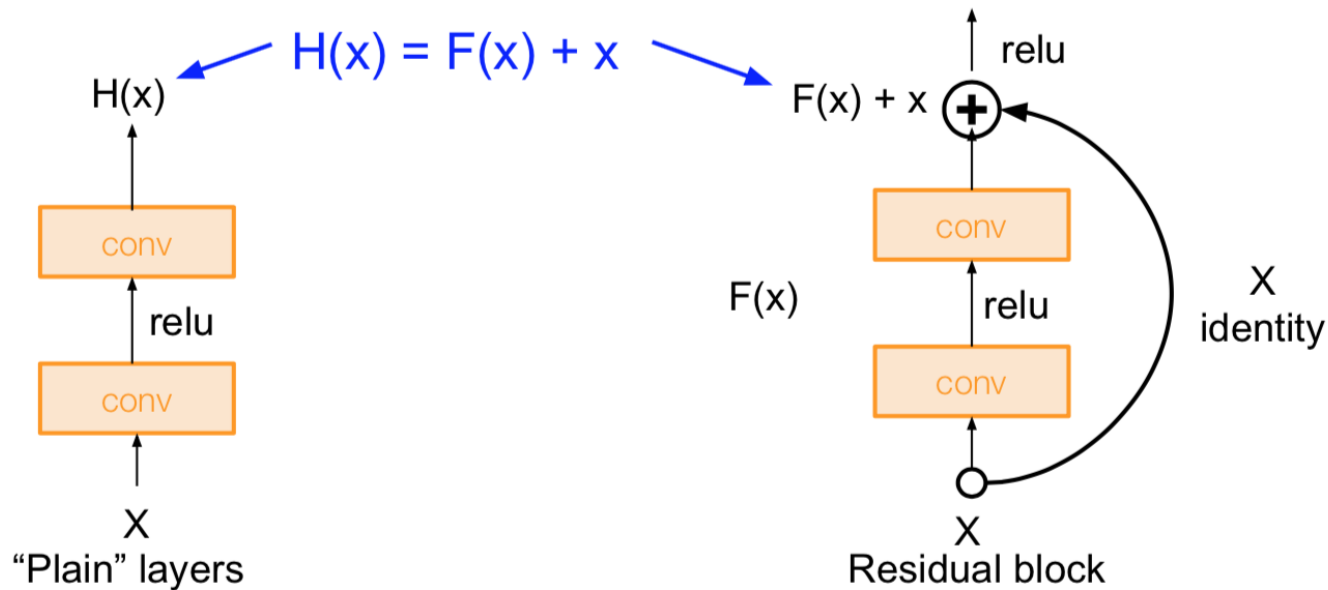


Hypothesis: deeper models are harder to optimize.

CNN Architecture: Part I

❑ ResNet [He et al., 2015]

Solution: use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping



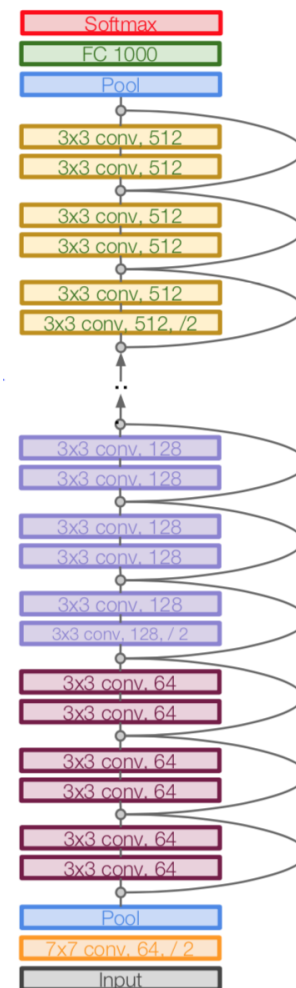
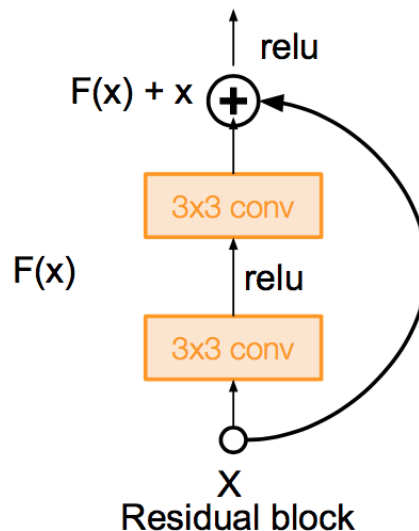
CNN Architecture: Part I

ResNet [He et al., 2016]

Very deep networks using residual connections

Basic design:

- all 3x3 conv (almost)
- spatial size /2 => #filter x2
- just deep
- average pooling (remove FC)
- no dropout

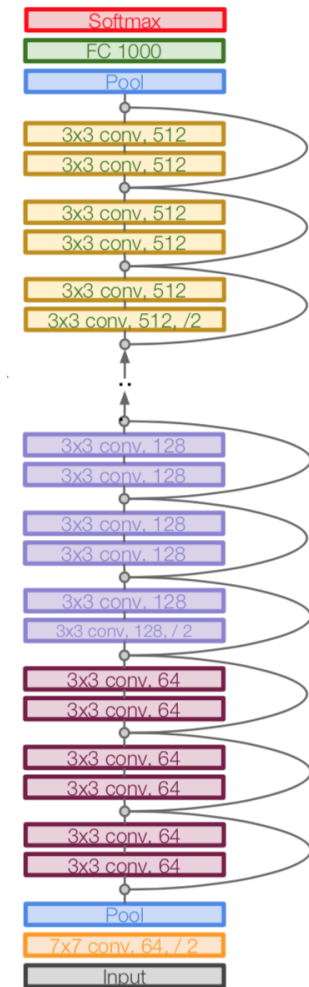
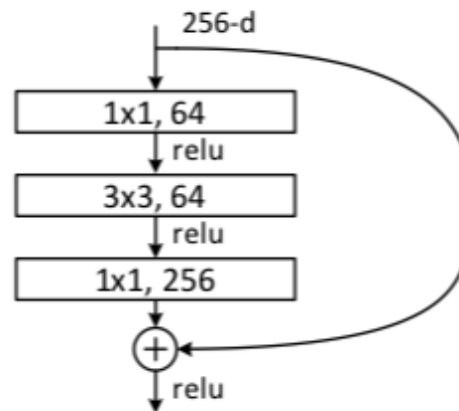
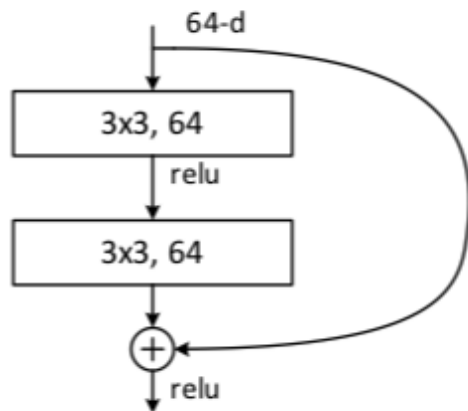


ResNet

CNN Architecture: Part I

❑ ResNet [He et al., 2015]

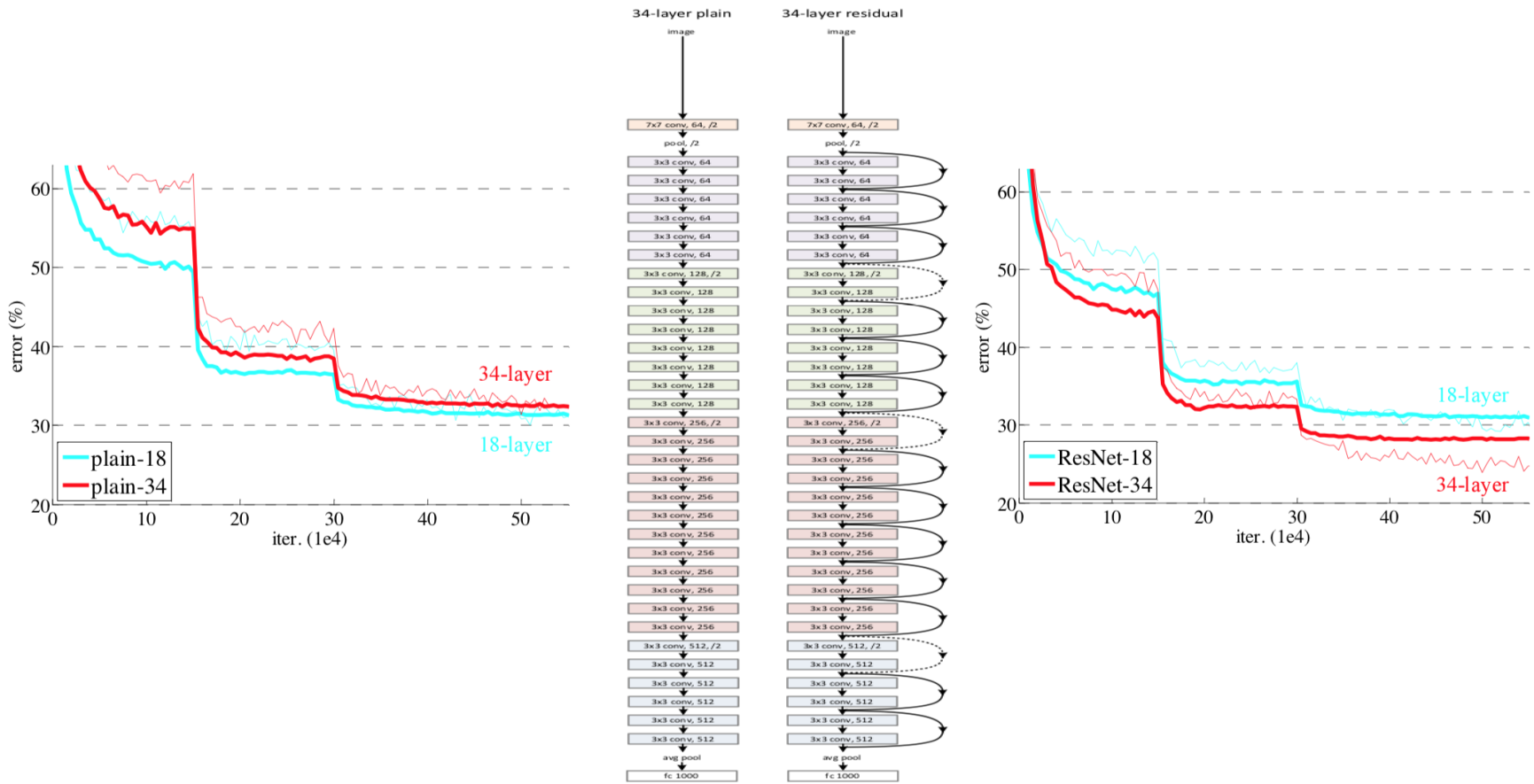
Use “**bottleneck**” layer to improve efficiency (similar to GoogLeNet).



ResNet

CNN Architecture: Part I

ResNet [He et al., 2015]



CNN Architecture: Part I

❑ ResNet [He et al., 2015]

- **1st places in all five main tracks**
 - ImageNet Classification: “*Ultra-deep*” 152-layer nets
 - ImageNet Detection: 16% better than 2nd
 - ImageNet Localization: 27% better than 2nd
 - COCO Detection: 11% better than 2nd
 - COCO Segmentation: 12% better than 2nd

Features matter: deeper features are well transferrable

CNN Architecture II: Refine and recombine

CNN Architecture: Part II

Inception: Inception v1,v2,v3,v4

Inception-res-v1,v2

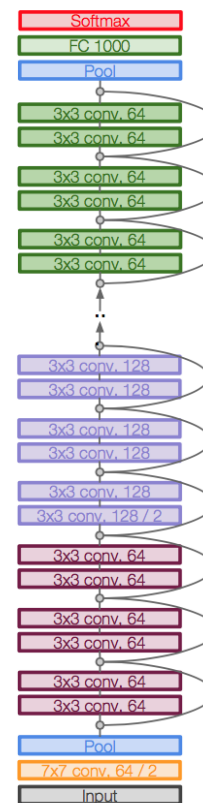
Xception



ResNet: Wide-ResNet

ResNeXt

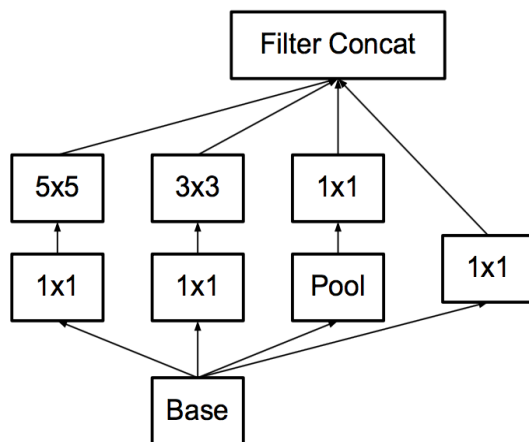
DenseNet



CNN Architecture: Part II

❏ Inception v2

- batch normalization
- remove LRN and dropout
- small kernels (inspired by VGG)



Inception module v1
(In GoogLeNet)

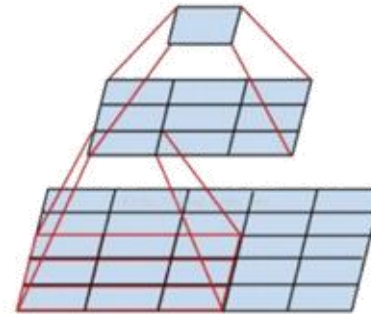
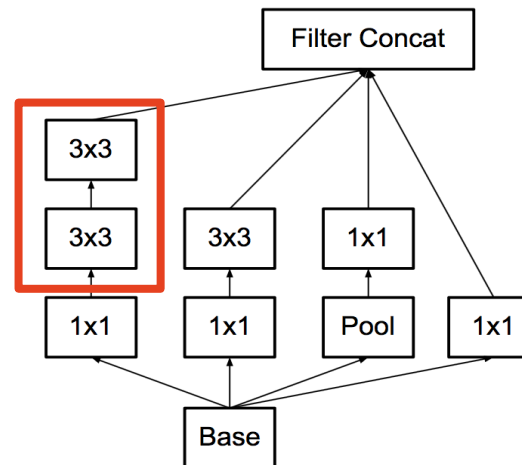


Figure 1. Mini-network replacing the 5×5 convolutions.



CNN Architecture: Part II

❏ Inception v3

Asymmetric convolutions

- 7x7 filter -> 7x1 and 1x7 filters
- 3x3 filter -> 3x1 and 1x3 filters

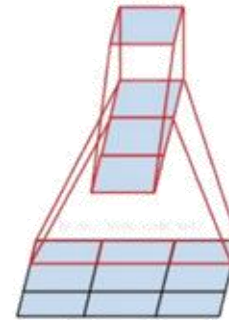
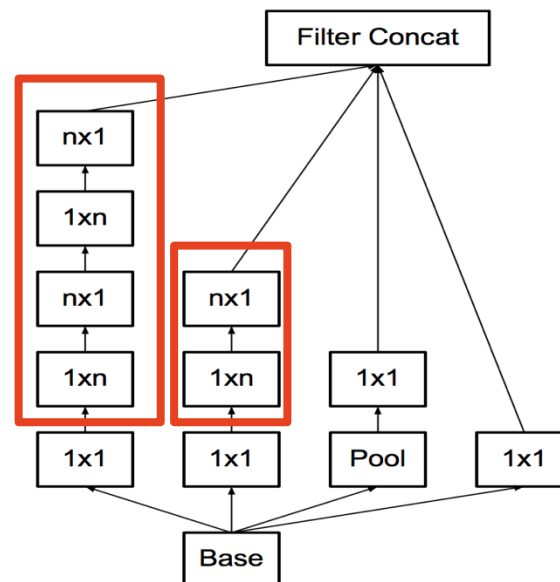
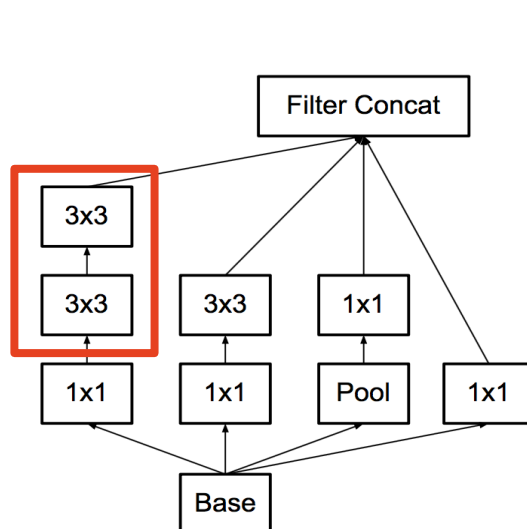


Figure 3. Mini-network replacing the 3×3 convolutions. The lower layer of this network consists of a 3×1 convolution with 3 output units.



CNN Architecture: Part II

❑ Inception v3

Label Smoothing

For a training example with ground-truth label y , we replace the label distribution with

$$q'(k|x) = (1 - \epsilon)q(k|x) + \epsilon u(k)$$

Inception v3



Network	Top-1 Error	Top-5 Error	Cost Bn Ops
GoogLeNet [20]	29%	9.2%	1.5
BN-GoogLeNet	26.8%	-	1.5
BN-Inception [7]	25.2%	7.8	2.0
Inception-v2	23.4%	-	3.8
Inception-v2 RMSProp	23.1%	6.3	3.8
Inception-v2 Label Smoothing	22.8%	6.1	3.8
Inception-v2 Factorized 7×7	21.6%	5.8	4.8
Inception-v2 BN-auxiliary	21.2%	5.6%	4.8

CNN Architecture: Part II

□ Inception v4

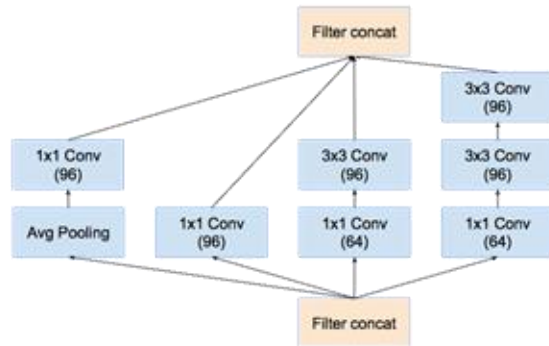


Figure 4. The schema for 35×35 grid modules of the pure Inception-v4 network. This is the Inception-A block of Figure 9.

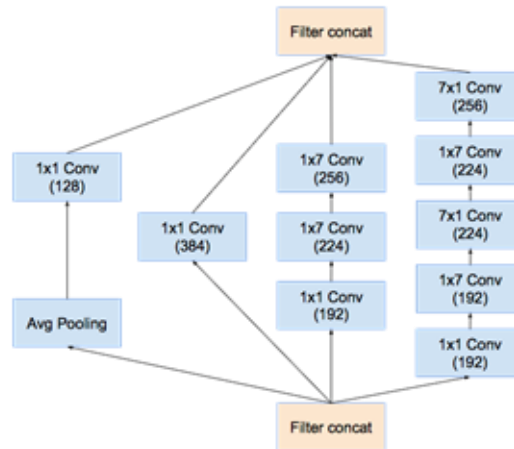


Figure 5. The schema for 17×17 grid modules of the pure Inception-v4 network. This is the Inception-B block of Figure 9.

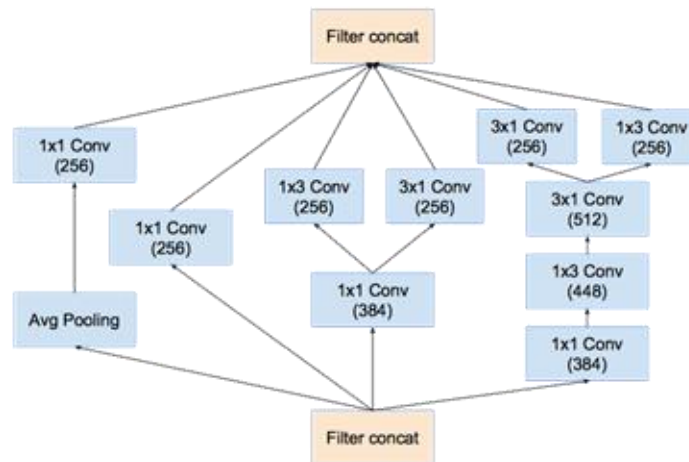
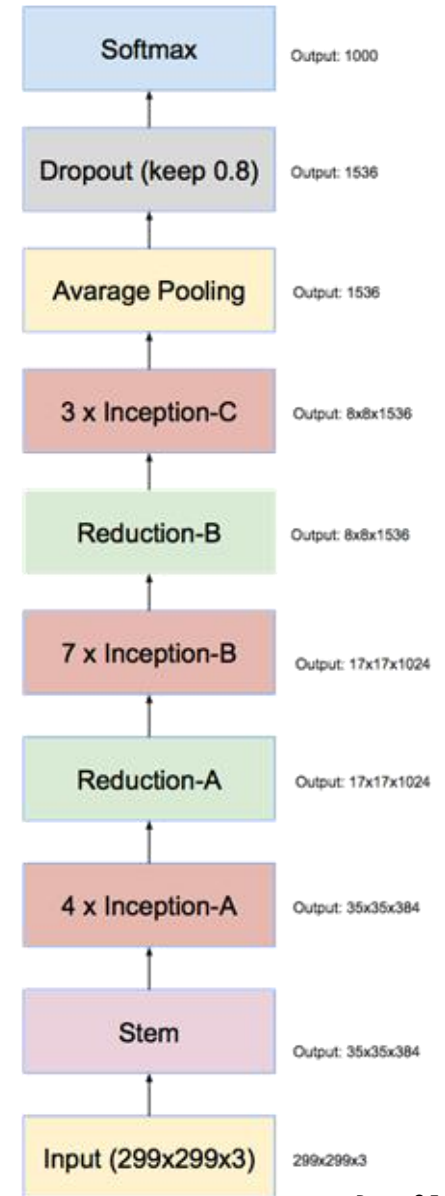


Figure 6. The schema for 8×8 grid modules of the pure Inception-v4 network. This is the Inception-C block of Figure 9.



CNN Architecture: Part II

□ Inception-ResNet v1, v2

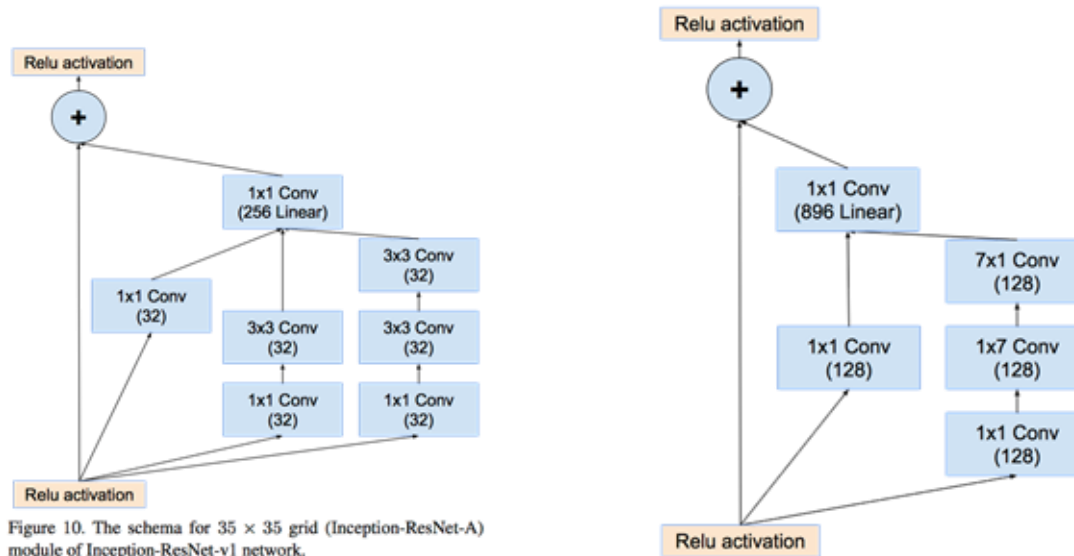


Figure 10. The schema for 35×35 grid (Inception-ResNet-A) module of Inception-ResNet-v1 network.

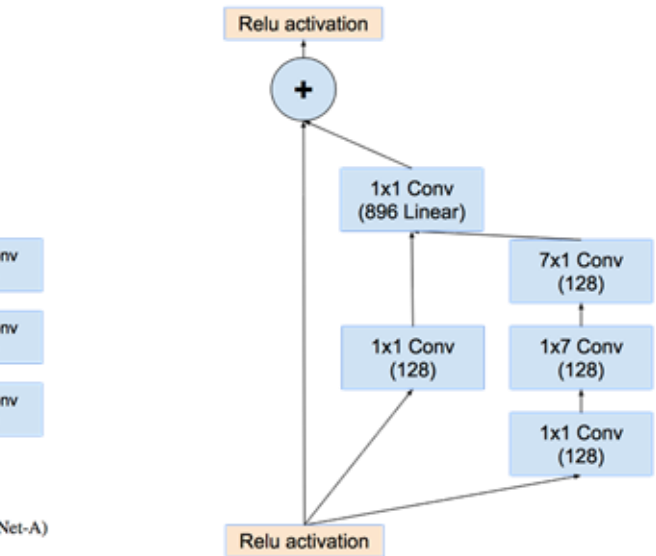


Figure 11. The schema for 17×17 grid (Inception-ResNet-B) module of Inception-ResNet-v1 network.

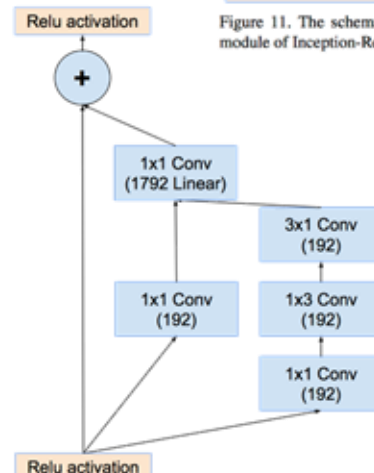


Figure 13. The schema for 8×8 grid (Inception-ResNet-C) module of Inception-ResNet-v1 network.



CNN Architecture: Part II

❑ Error on ImageNet-1000 classification

Network	Top-1 Error	Top-5 Error
BN-Inception [6]	25.2%	7.8%
Inception-v3 [15]	21.2%	5.6%
Inception-ResNet-v1	21.3%	5.5%
Inception-v4	20.0%	5.0%
Inception-ResNet-v2	19.9%	4.9%

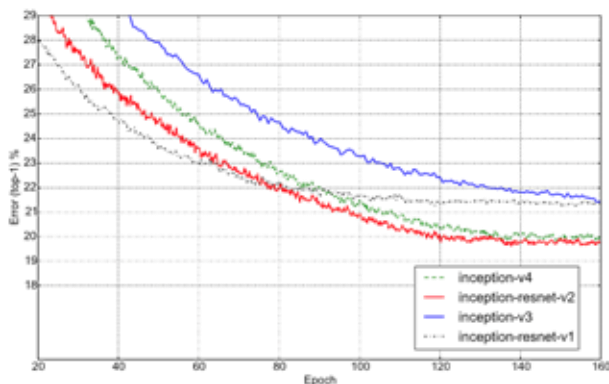


Figure 26. Top-1 error evolution of all four models (single model, single crop). This paints a similar picture as the top-5 evaluation.

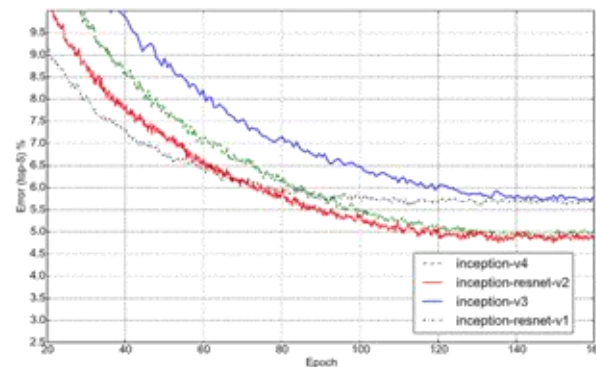
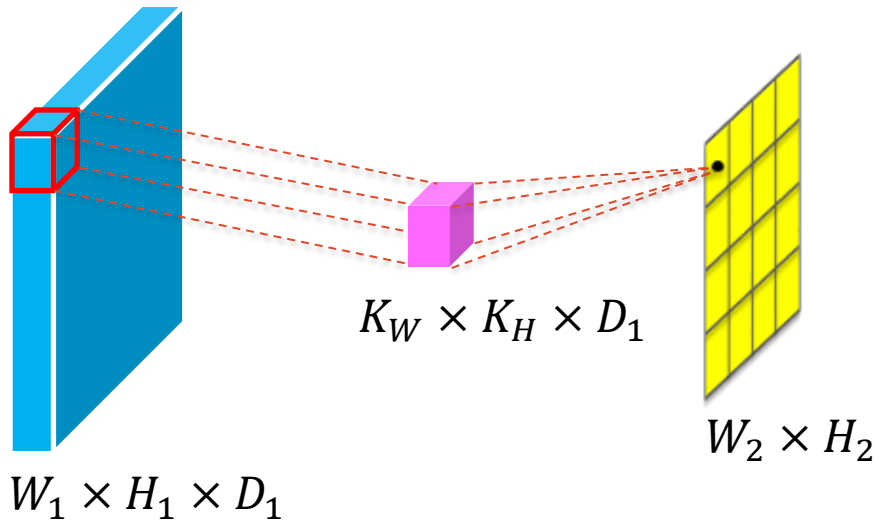


Figure 25. Top-5 error evolution of all four models (single model, single crop). Showing the improvement due to larger model size. Although the residual version converges faster, the final accuracy seems to mainly depend on the model size.

CNN Architecture: Part II

❑ Xception [François Chollet, 2017]



Replace inception module with
depth-wise separable convolution.

Separates spatial correlation (i.e. width, height) and cross-channel correlations.

Figure 1. A canonical Inception module (Inception V3).

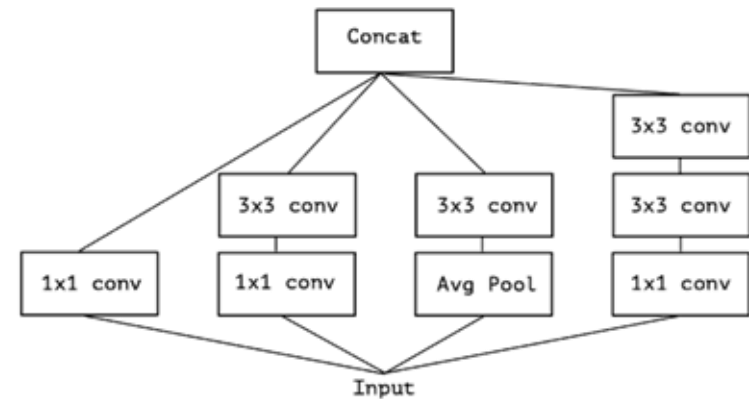
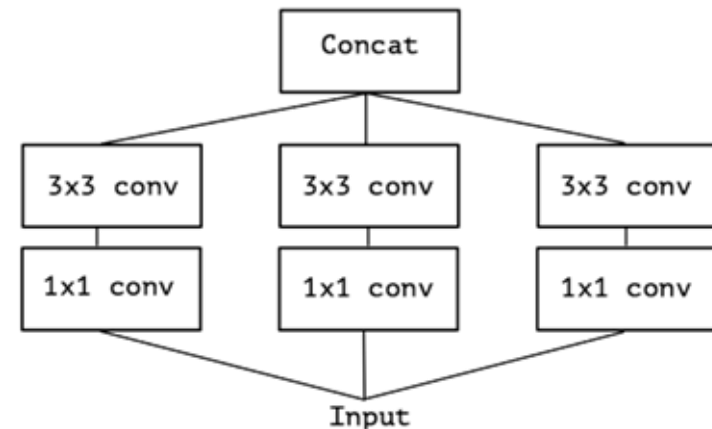
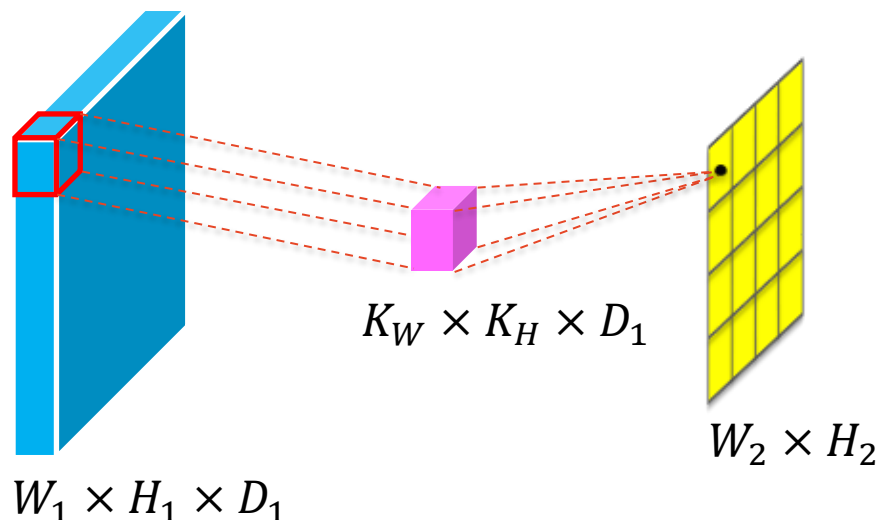


Figure 2. A simplified Inception module.



CNN Architecture: Part II

❑ Xception [François Chollet, 2017]



Inception module is to explicitly factorize it into a series of operations that would independently look at cross-channel correlations and at spatial correlations.

Assume that cross-channel correlations and spatial correlations can be mapped completely separately.

Figure 4. An “extreme” version of our Inception module, with one spatial convolution per output channel of the 1x1 convolution.

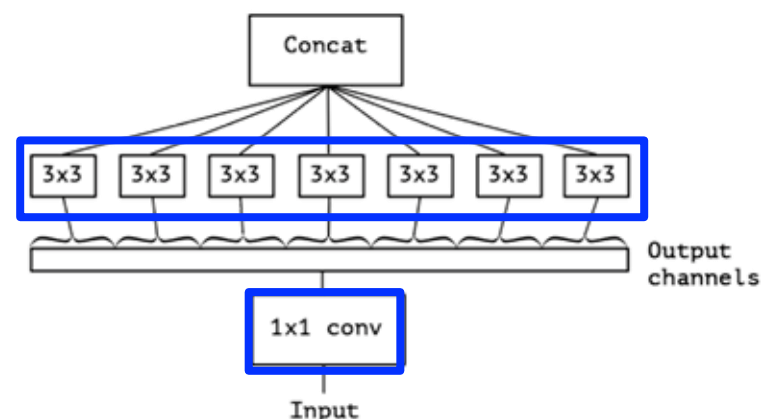
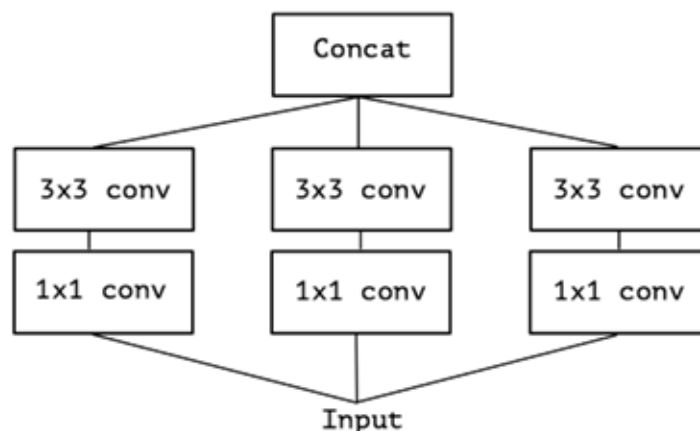


Figure 2. A simplified Inception module.



CNN Architecture: Part II

❑ Wide ResNet (WRNs) [Zagoruyko et al. 2016]

“Is depth really the key variable? Let’s go wider!”

A percent of improved accuracy costs nearly doubling the number of layers.

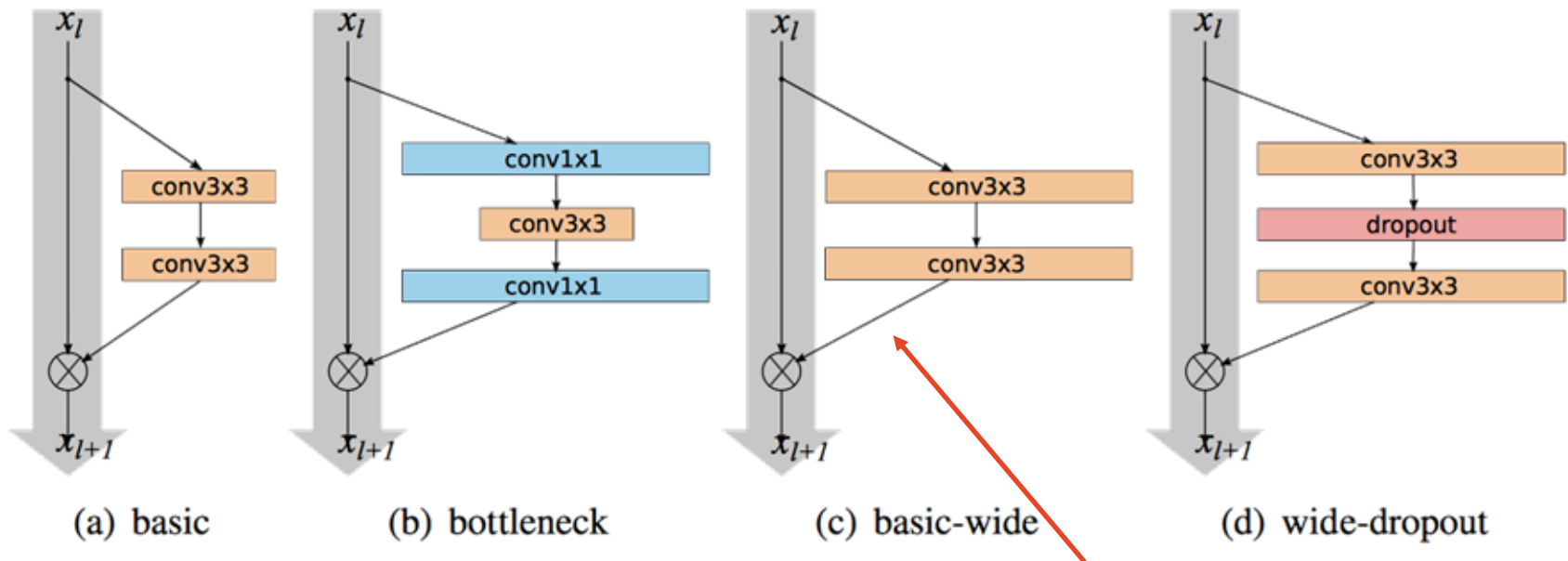
model	top-1	top-5
ResNet-50	22.9%	6.7%
ResNet-101	21.8%	6.1%
ResNet-152	21.4%	5.7%

trade-off: decrease depth and increase width of residual networks

CNN Architecture: Part II

❑ Wide ResNet (WRNs) [Zagoruyko et al. 2016]

- Residuals are important, not depth
- Increasing width instead of depth more computationally efficient



- more convolutional layers per block
- more feature planes

CNN Architecture: Part II

❑ Wide ResNet (WRNs) [Zagoruyko et al. 2016]

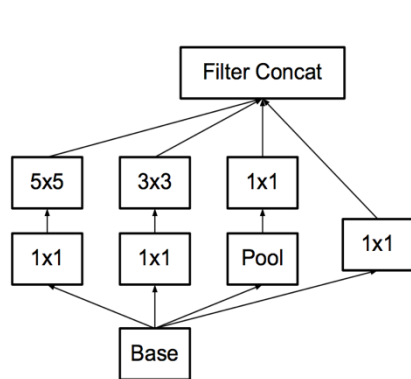
Model	top-1 err, %	top-5 err, %	#params	time/batch 16
ResNet-50	24.01	7.02	25.6M	49
ResNet-101	22.44	6.21	44.5M	82
ResNet-152	22.16	6.16	60.2M	115
WRN-50-2-bottleneck	21.9	6.03	68.9M	93

50-layer wide ResNet outperforms 152-layer original ResNet

CNN Architecture: Part II

❑ ResNeXt [Xie et al. 2016]

“What if we increase parallel paths instead of depth?”



Inception model

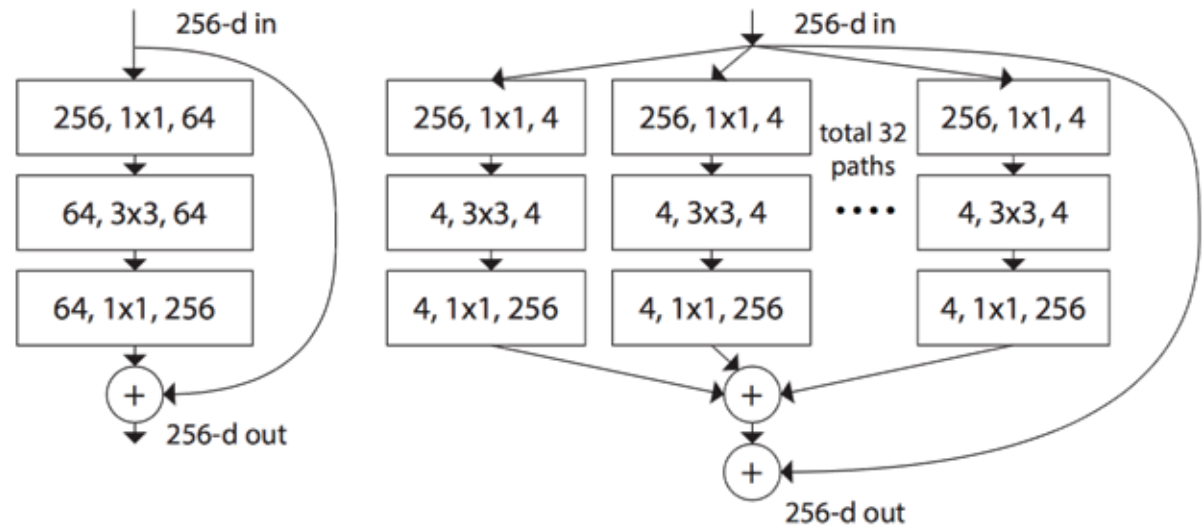


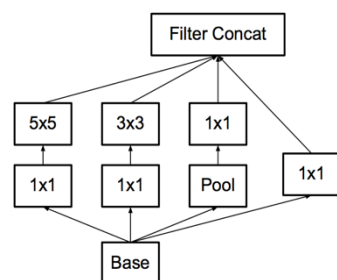
Figure 1. **Left:** A block of ResNet [14]. **Right:** A block of ResNeXt with cardinality = 32, with roughly the same complexity. A layer is shown as (# in channels, filter size, # out channels).

CNN Architecture: Part II

❑ ResNeXt [Xie et al. 2016]

Repeating a building block that aggregates a set of transformations with the same topology.

- **Stack building blocks** of the same shape, e.g. VGG and ResNets.
- **Split-transform-merge** strategy. In an Inception module, the input is split into a few lower-dimensional embeddings (by 1×1 convolutions), transformed by a set of specialized filters (3×3 , 5×5 , etc.), and merged by concatenation.



Inception model

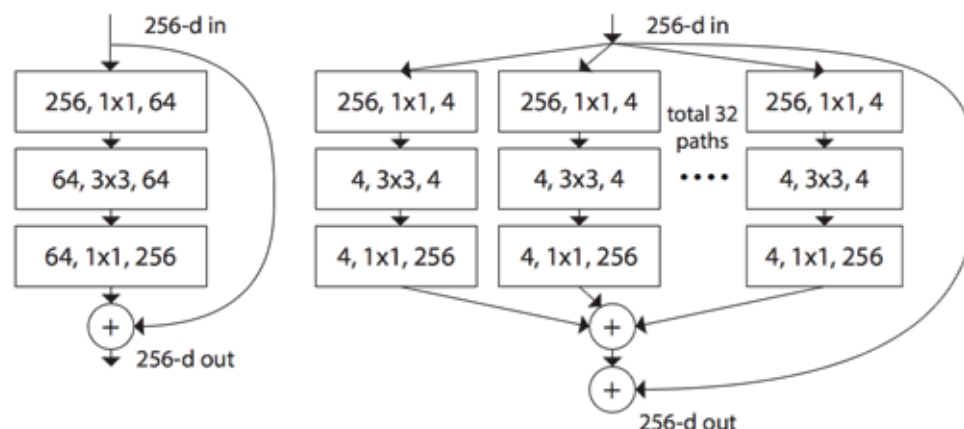


Figure 1. **Left:** A block of ResNet [14]. **Right:** A block of ResNeXt with cardinality = 32, with roughly the same complexity. A layer is shown as (# in channels, filter size, # out channels).

CNN Architecture: Part II

□ ResNeXt [Xie et al. 2016]

stage	output	ResNet-50	ResNeXt-50 (32×4d)
conv1	112×112	7×7, 64, stride 2	7×7, 64, stride 2
conv2	56×56	3×3 max pool, stride 2	3×3 max pool, stride 2
		$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128, C=32 \\ 1\times 1, 256 \end{bmatrix} \times 3$
conv3	28×28	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256, C=32 \\ 1\times 1, 512 \end{bmatrix} \times 4$
conv4	14×14	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512, C=32 \\ 1\times 1, 1024 \end{bmatrix} \times 6$
conv5	7×7	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 1024 \\ 3\times 3, 1024, C=32 \\ 1\times 1, 2048 \end{bmatrix} \times 3$
	1×1	global average pool 1000-d fc, softmax	global average pool 1000-d fc, softmax
# params.		25.5 ×10 ⁶	25.0 ×10 ⁶
FLOPs		4.1 ×10 ⁹	4.2 ×10 ⁹

Similar complexity
ResNet = ResNeXt

CNN Architecture: Part II

□ ResNeXt [Xie et al. 2016]

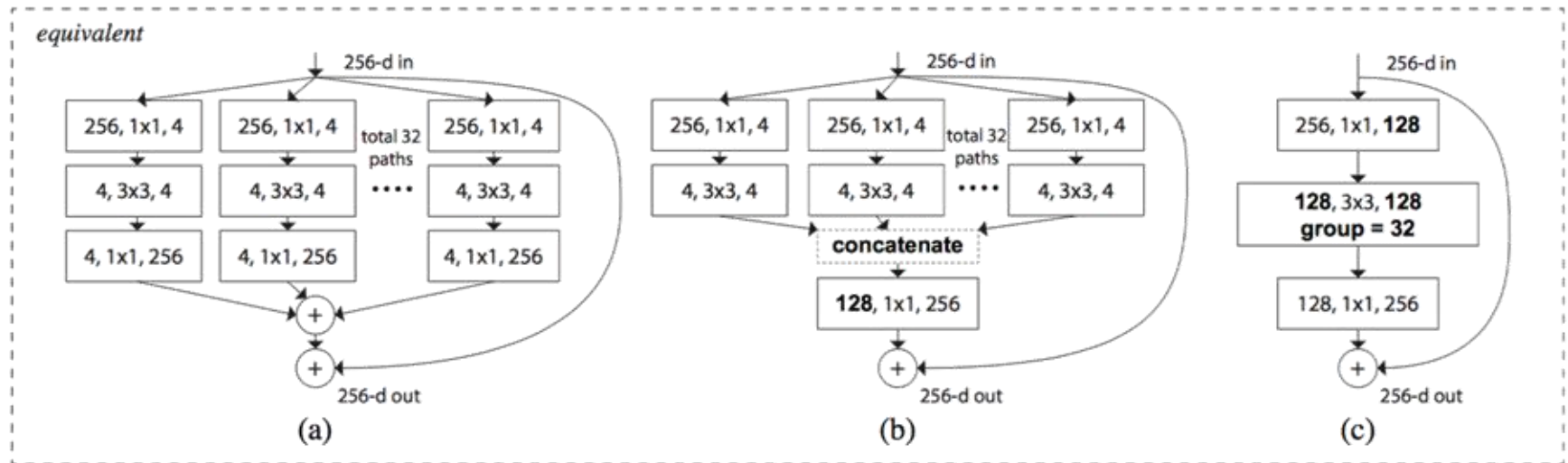


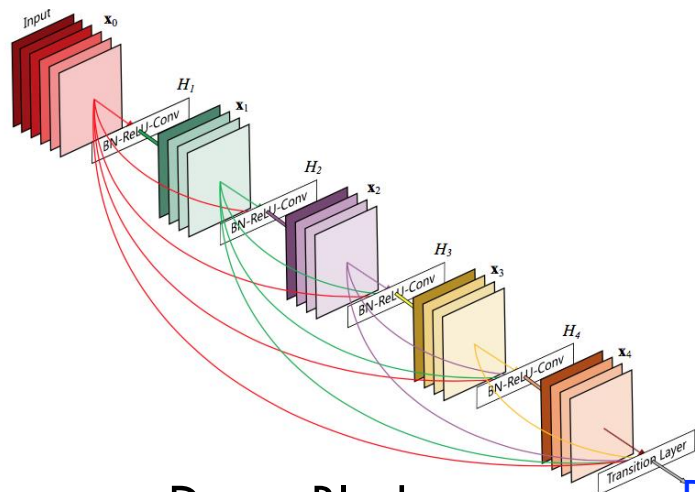
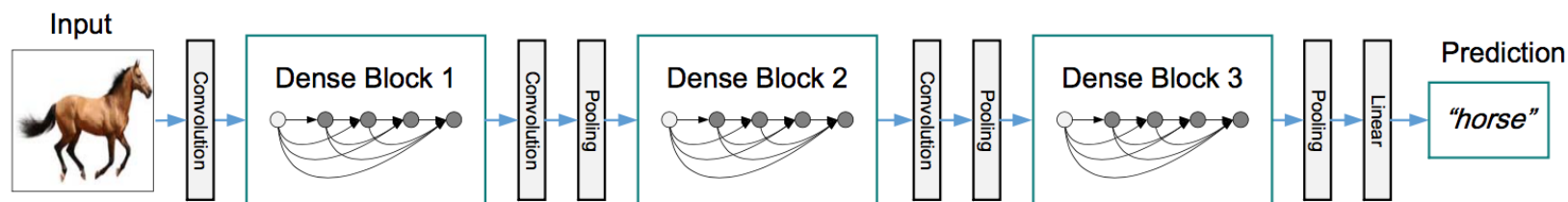
Figure 3. Equivalent building blocks of ResNeXt. **(a)**: Aggregated residual transformations, the same as Fig. 1 right. **(b)**: A block equivalent to (a), implemented as early concatenation. **(c)**: A block equivalent to (a,b), implemented as grouped convolutions [24]. Notations in **bold** text highlight the reformulation changes. A layer is denoted as (# input channels, filter size, # output channels).

CNN Architecture: Part II

□ DenseNet [Huang et al. 2017]

“What if we connect everything to everything?”

Each layer has **direct access** to the gradients from the loss function and the original input signal, leading to an implicit deep supervision.



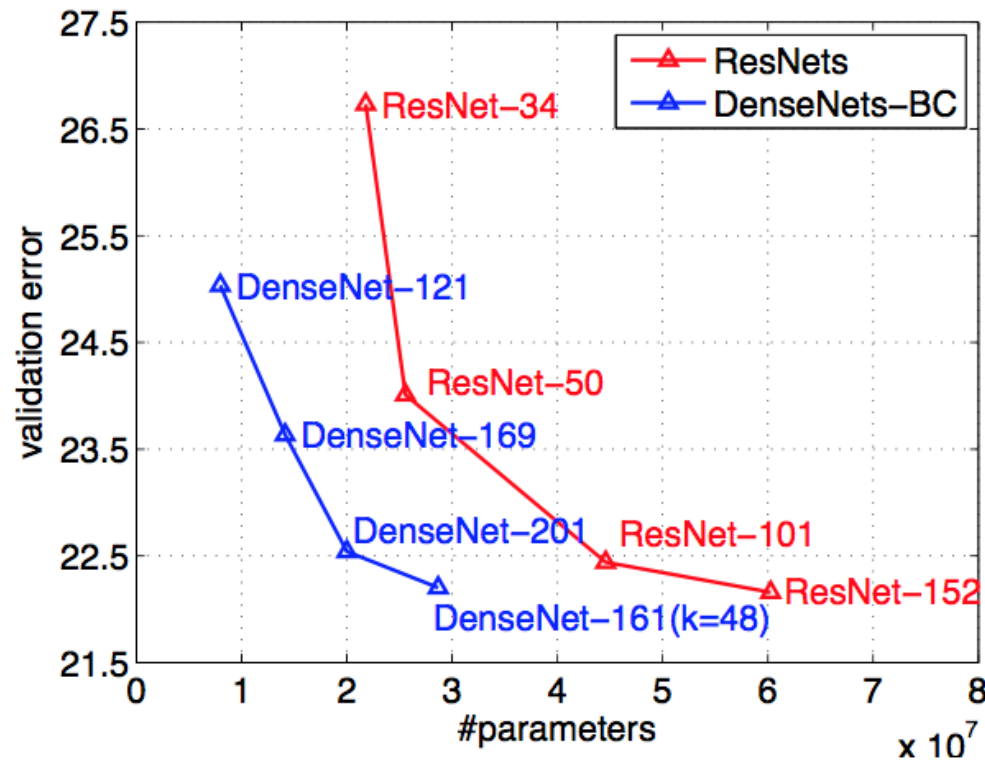
- alleviate the vanishing-gradient problem
- strengthen feature propagation
- encourage feature reuse
- reduce the number of parameters

ResNets: $\mathbf{x}_\ell = H_\ell(\mathbf{x}_{\ell-1}) + \mathbf{x}_{\ell-1}$

DenseNets: $\mathbf{x}_\ell = H_\ell([\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{\ell-1}])$

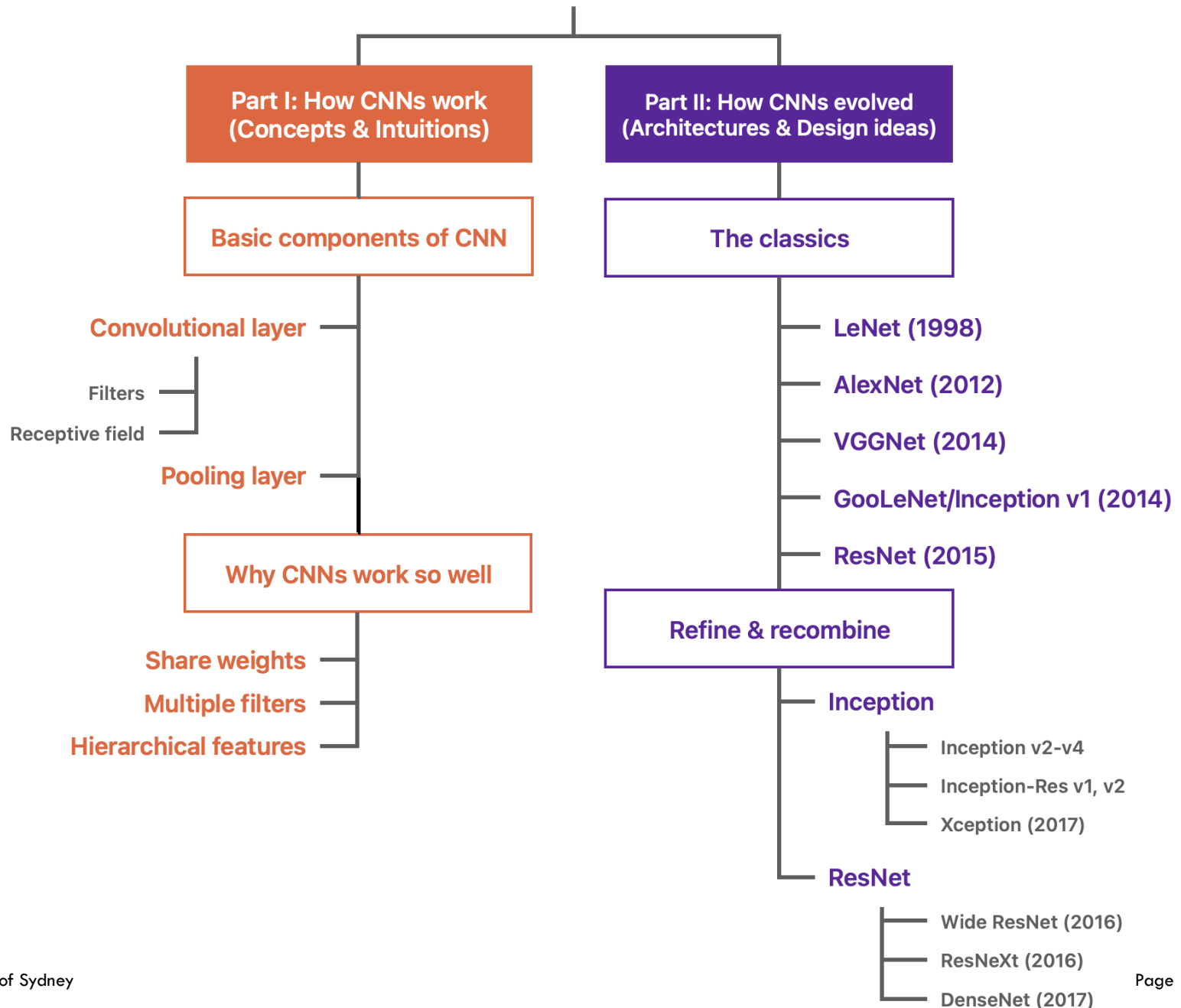
CNN Architecture: Part II

□ DenseNet [Huang et al. 2017]



Comparison of the DenseNet and ResNet on model size

Convolutional Neural Network



Thank you!